Infrastructure Scalability

7. Design a Content Delivery Network

Key Terms:

Term	Full Name	What it does
CDN	Content Delivery Network	Globally distributed edge servers for fast content delivery
POP	Point of Presence	Edge server location close to users
GeoDNS	Geographic DNS	DNS that returns different IPs based on user location
Anycast	Network routing	Same IP announced from multiple locations; BGP routes to nearest
TTL	Time To Live	How long content stays cached before refresh
Origin	Source server	The original server where content lives
Shield	Mid-tier cache	Regional cache layer protecting origin from traffic
Purge	Cache invalidation	Removing stale content from edge servers

Problem Statement

Design a CDN that delivers static content (images, videos, files) with low latency to users worldwide.

Clarifying Questions to Ask

1. **Content types**: Static files, streaming video, or both?
2. **Scale**: How much data? How many requests per second?
3. **Latency target**: Max acceptable latency? (<50ms?)
4. **Cache hit ratio**: Target cache hit rate? (>95%?)
5. **Origin**: Single origin or multiple origins?
6. **Invalidation**: How quickly must cache updates propagate?

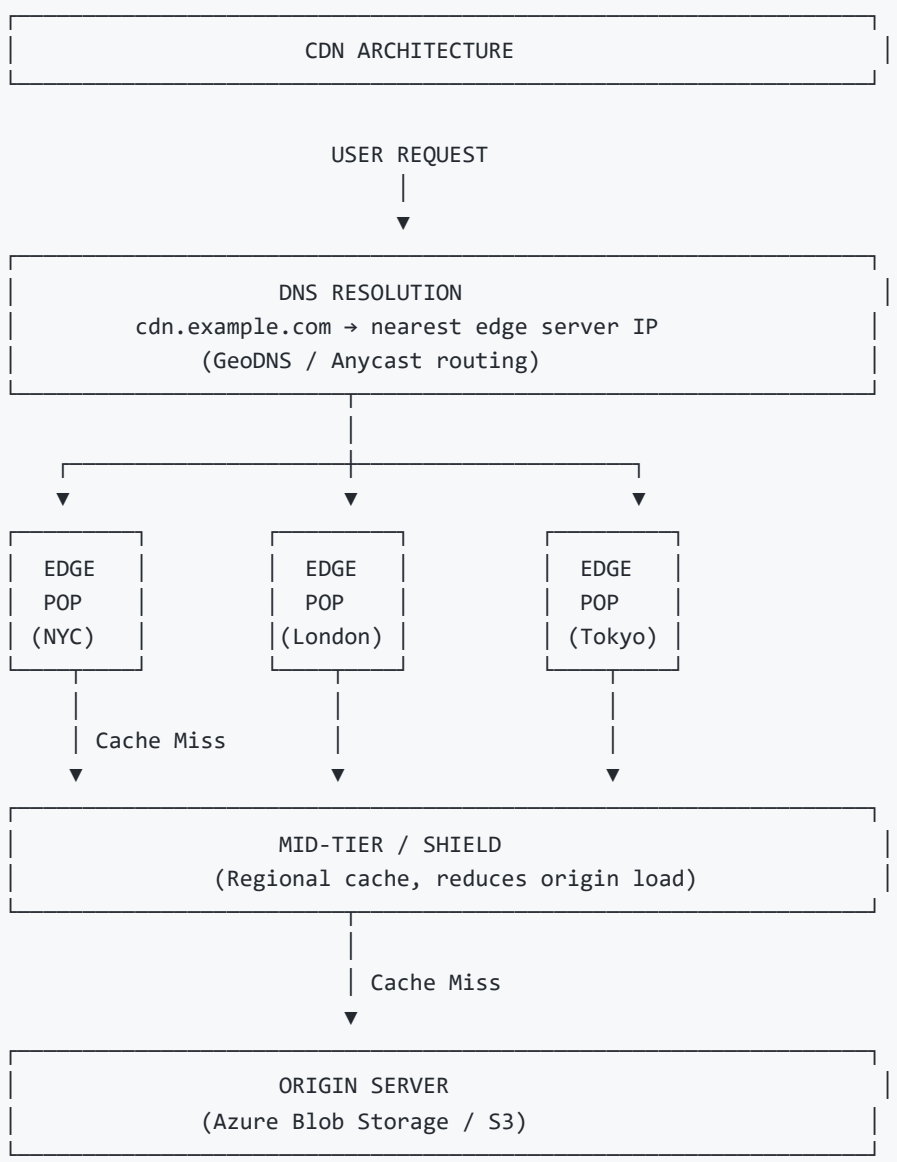
7. **Security**: DDoS protection, token auth needed?

Design Reasoning

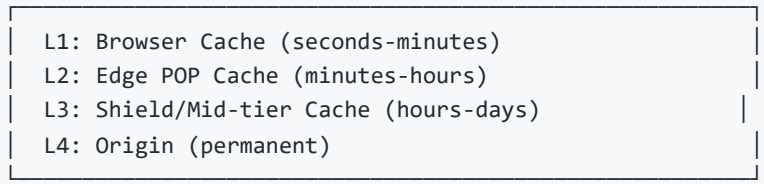
> **Core philosophy**: Minimize latency through a multi-tier caching hierarchy (edge, shield, origin) where content is served as close to users as possible, with Anycast routing for instant failover and versioned URLs for cache-friendly invalidation.

Decision	Choice	Why	Alternative Considered
Routing	Anycast	Same IP announced from all POPs; BGP routes to nearest automatically with instant failover	GeoDNS — slower failover (DNS TTL delay), more complex configuration per region
Cache architecture	3-tier (edge, shield, origin)	Shield layer absorbs cache misses from multiple edge POPs, protecting origin from thundering herd	2-tier (edge, origin) — origin overwhelmed when popular content expires across all edges
Edge server	Nginx/Varnish on SSD + RAM	Hot content in RAM for sub-ms response; warm content on SSD keeps cache hit ratio above 95%	RAM-only — too expensive for large content libraries; SSD-only — too slow for hot assets
Cache invalidation	Versioned URLs (primary)	Append version hash to filenames; instant updates without purge propagation delay	TTL-based purge — stale content served until expiry; purge API — complex tracking of what to invalidate
Content distribution	Consistent hashing	Even distribution across edge servers; minimal reshuffling when servers are added/removed	Random assignment — poor cache locality, same content duplicated across servers
Origin storage	Azure Blob Storage	Virtually unlimited capacity, high durability (11 nines), and low cost for static assets	Self-hosted origin servers — operational burden, capacity planning, lower durability

High-Level Design



CACHE HIERARCHY:



API Design

Content Management APIs

Method	Endpoint	Description

`POST`	`/api/v1/distributions`	Create a CDN distribution
`PUT`	`/api/v1/distributions/{id}`	Update distribution config
`POST`	`/api/v1/distributions/{id}/invalidate`	Invalidate cache
`GET`	`/api/v1/distributions/{id}/stats`	Get cache hit/miss stats

Cache Invalidation Request

```
POST /api/v1/distributions/{id}/invalidate
{
  "paths": ["/images/*", "/css/main.css"],
  "strategy": "immediate"   // or "soft"
}
```

Edge Request Flow

```
GET /assets/image.jpg
Headers:
  Host: cdn.example.com
  If-None-Match: "etag-abc123"

Response: 200 OK (cache miss) / 304 Not Modified (cache hit)
Headers:
  Cache-Control: public, max-age=86400
  ETag: "etag-abc123"
  X-Cache: HIT | MISS
  X-Edge-Location: us-east-1
```

Key Components

Component	Purpose	Technology
GeoDNS	Route to nearest POP	Route 53 / Azure Traffic Manager
Edge Server	Serve cached content	Nginx/Varnish
Shield	Reduce origin load	Regional cache tier
Origin	Source of truth	Blob storage
Purge Service	Invalidate cache	Message queue
Analytics	Track hits/misses	Log aggregation

Cache Key Design

```
Cache Key = Hash(URL + Vary headers + Query params)

Examples:
  /images/logo.png → key: "abc123"
  /images/logo.png?v=2 → key: "def456" (different version)
  /api/user (Accept: json) → key: "ghi789"
  /api/user (Accept: xml) → key: "jkl012" (different format)

Cache-Control Headers:
  public, max-age=86400              # Cache for 24h
  private, no-cache                   # Don't cache
  s-maxage=3600, max-age=60          # CDN: 1h, Browser: 1min
```

Cache Invalidation Strategies

```
1. TTL-based: Content expires after set time
   Pros: Simple, automatic
   Cons: Stale content until expiry

2. Purge API: Explicitly invalidate by URL/pattern
   POST /purge {"urls": ["/images/logo.png"]}
   Pros: Immediate
   Cons: Need to track what to purge

3. Versioned URLs: Append version to filename
   /images/logo.v123.png → /images/logo.v124.png
   Pros: Instant update, cache-friendly
   Cons: URL management

4. Soft Purge / Stale-While-Revalidate:
   Serve stale, fetch fresh in background
   Pros: Fast response, fresh content soon
   Cons: Brief staleness
```

Scalability Considerations

- **POPs**: Deploy edge servers in 50+ locations globally
- **Consistent hashing**: Distribute content across edge servers
- **Hot content**: Replicate popular content to all edges
- **Bandwidth**: Use tiered architecture to reduce origin bandwidth
- **DDoS protection**: Rate limiting, challenge pages at edge

Trade-offs

Decision	Option A	Option B	Recommendation
Routing	GeoDNS	Anycast	**Anycast** - faster failover
Cache storage	Memory	SSD	**Hybrid** - hot in RAM, warm on SSD
Invalidation	TTL	Purge	**Versioned URLs** - best of both
Architecture	2-tier	3-tier (shield)	**3-tier** - protects origin

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
CDN	Edge caching and content distribution	Tech Guide
Consistent Hashing	Even distribution across edge servers	Tech Guide
Load Balancer	Traffic distribution and health checks	Tech Guide

---

How to Explain It Well

> **Opening**: "A CDN reduces latency by serving content from servers close to users. I'll design for 95%+ cache hit ratio with sub-50ms latency."

> **Key talking points**:
>
> - "Anycast routing: same IP announced from all POPs, BGP routes to nearest"
> - "Shield tier absorbs cache misses, prevents origin overload"
> - "Versioned URLs for instant updates without purging"
> - "Cache key includes Vary headers to handle content negotiation"
> - "Consistent hashing distributes content across edge servers"

---

11. Design a Notification Service

Key Terms:

Term	Full Name	What it does

APNs	Apple Push Notification service	Apple's push delivery to iOS devices
FCM	Firebase Cloud Messaging	Google's push delivery to Android/iOS
SES	Simple Email Service	AWS email sending service
DLQ	Dead Letter Queue	Queue for messages that failed processing
2FA	Two-Factor Authentication	Security requiring two verification methods

Problem Statement

Design a scalable notification system that delivers email, SMS, and push notifications to millions of users with high reliability and low latency.

Clarifying Questions to Ask

1. **Channels**: Email, SMS, push, in-app, or all?
2. **Volume**: How many notifications per day? (1B?)
3. **Latency**: Real-time or batched acceptable?
4. **Priority**: Support urgent vs. normal notifications?
5. **Preferences**: User opt-in/out per channel?
6. **Templates**: Dynamic content or fixed templates?
7. **Tracking**: Track delivery, opens, clicks?

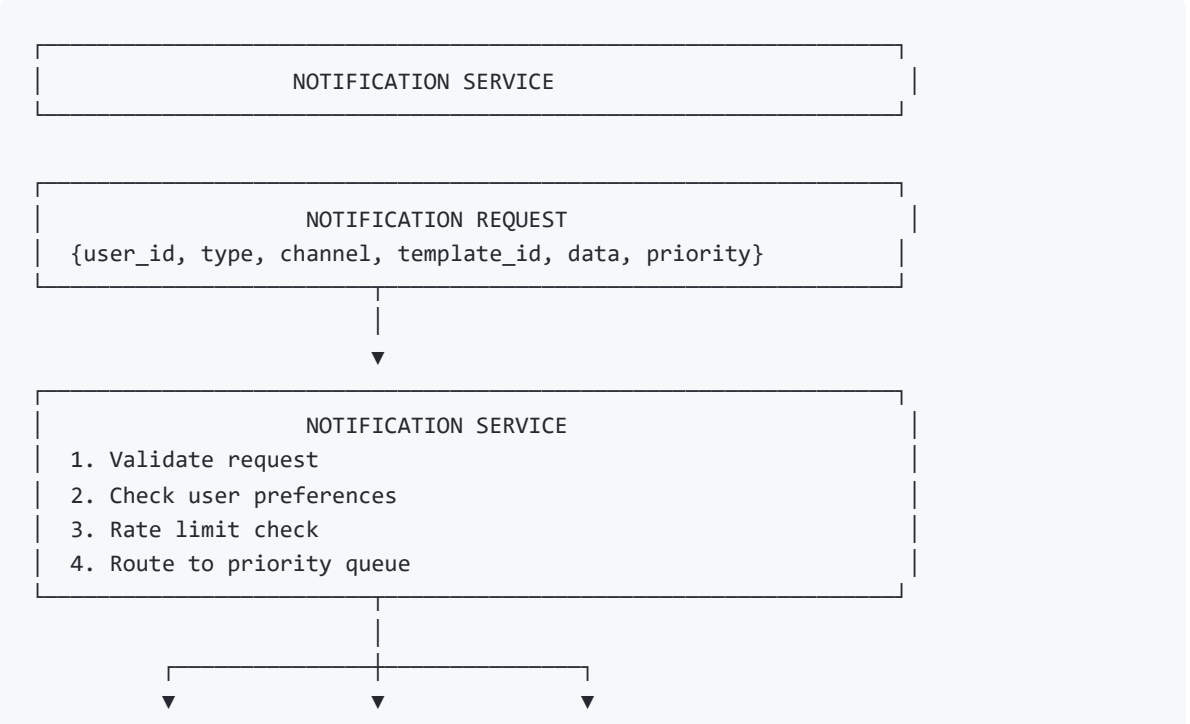
Design Reasoning

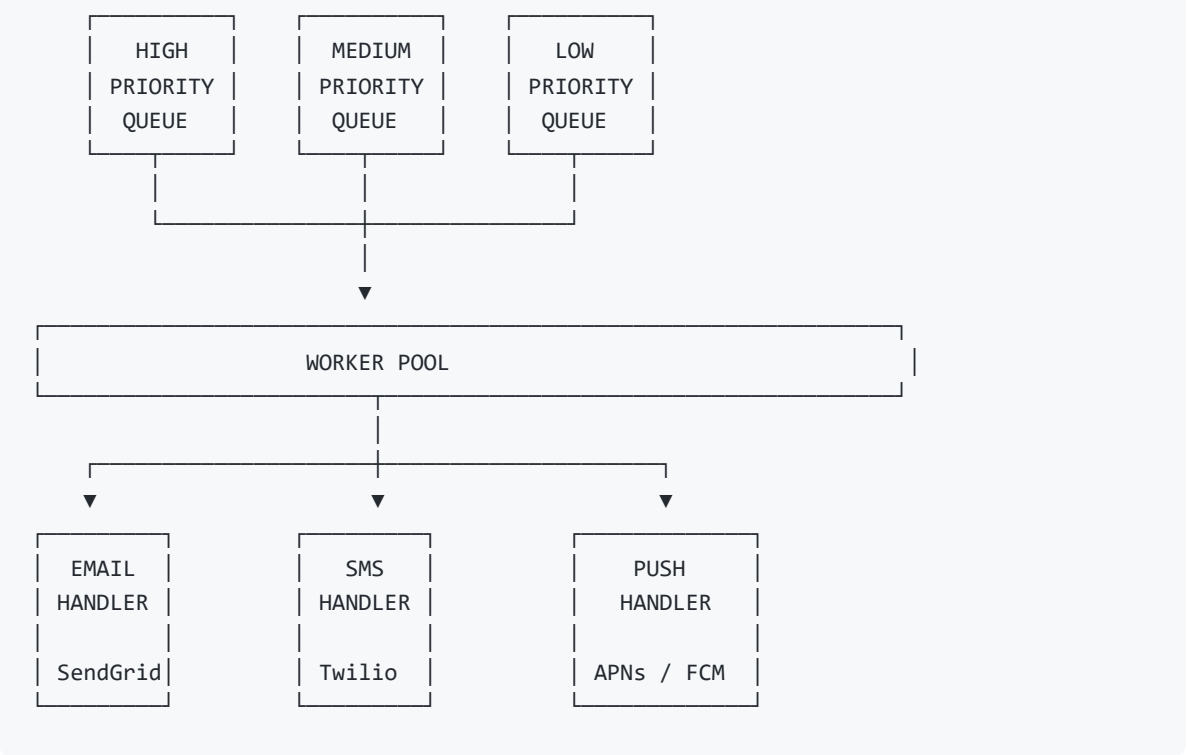
> **Core philosophy**: Reliable, prioritized delivery across multiple channels where no notification is silently lost — using async processing with retry guarantees to handle billions of daily messages.

Decision	Choice	Why	Alternative Considered
Message Queue	Kafka	High throughput for 1B daily notifications; durable log enables replay	RabbitMQ — lower throughput ceiling, harder to scale horizontally
Queue Architecture	Priority queues (high/med/low)	2FA codes and security alerts processed before marketing emails	Single FIFO queue — urgent notifications delayed behind bulk sends

Delivery Guarantee	At-least-once with idempotency	Simpler than exactly-once; duplicate detection handled at receiver	Exactly-once — requires distributed transactions, adds significant complexity
Failed Message Handling	Dead Letter Queue (DLQ)	Isolates poison messages for inspection and manual replay without blocking	Infinite retry — blocks queue progress, delays all other notifications
Template Rendering	Centralized service (Handlebars)	Reusable templates across channels; consistent branding, easy updates	Inline content per request — duplicated logic, hard to maintain at scale
Channel Providers	Multi-provider per channel (SendGrid + SES)	Automatic failover if primary provider has outage	Single provider — single point of failure for entire delivery channel
User Preferences	PostgreSQL + Redis cache	Structured preference data with fast cached lookup on every request	In-memory only — lost on restart, inconsistent across service instances

High-Level Design





API Design

Notification APIs

Method	Endpoint	Description
`POST`	`/api/v1/notifications/send`	Send a notification
`POST`	`/api/v1/notifications/batch`	Send bulk notifications
`GET`	`/api/v1/notifications?userId={id}&limit=20`	Get user's notification history
`PUT`	`/api/v1/notifications/{notifId}/read`	Mark notification as read
`PUT`	`/api/v1/notifications/read-all`	Mark all as read

Preference APIs

Method	Endpoint	Description
`GET`	`/api/v1/users/{userId}/preferences`	Get notification preferences
`PUT`	`/api/v1/users/{userId}/preferences`	Update notification preferences

`POST`	`/api/v1/users/{userId}/devices`	Register device for push notifications
`DELETE`	`/api/v1/users/{userId}/devices/{deviceId}`	Unregister device

Send Notification Request

```
POST /api/v1/notifications/send
{
  "user_id": "user_123",
  "type": "order_shipped",
  "channels": ["push", "email", "in_app"],
  "priority": "high",
  "template_id": "tmpl_ship_01",
  "data": {
    "order_id": "ORD-456",
    "tracking_url": "https://..."
  }
}
```

Key Components

Component	Purpose	Technology
API Gateway	Receive notification requests	REST API
Preference Service	User notification settings	PostgreSQL
Template Service	Render dynamic content	Handlebars/Mustache
Queue	Buffer and prioritize	Kafka / RabbitMQ
Workers	Process and send	Distributed workers
Email Provider	Send emails	SendGrid, SES
SMS Provider	Send SMS	Twilio, Nexmo
Push Provider	Mobile push	APNs, FCM

Notification Flow

```
1. SERVICE SENDS REQUEST:
   POST /notifications
   {
     "user_id": "123",
```

```
    "template_id": "order_shipped",
    "channels": ["email", "push"],
    "data": {"order_id": "456", "tracking": "ABC123"},
    "priority": "high"
  }

2. VALIDATION & ROUTING:
   - Check user exists and preferences
   - Check rate limits (max 10 emails/hour per user)
   - Enqueue to appropriate priority queue

3. WORKER PROCESSING:
   - Dequeue message
   - Render template with data
   - Call provider API
   - Handle response (success/failure)
   - Store delivery status

4. RETRY LOGIC:
   - Exponential backoff: 1s, 2s, 4s, 8s...
   - Max 5 retries
   - Dead letter queue for failures
```

Database Schema

```sql
-- Notifications
CREATE TABLE notifications (
    id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    template_id VARCHAR(50),
    channel VARCHAR(20),  -- email, sms, push
    status VARCHAR(20),   -- pending, sent, delivered, failed
    priority INT,
    data JSONB,
    created_at TIMESTAMP,
    sent_at TIMESTAMP,
    delivered_at TIMESTAMP,
    error_message TEXT
);

-- User Preferences
CREATE TABLE notification_preferences (
    user_id BIGINT,
    channel VARCHAR(20),
    enabled BOOLEAN DEFAULT true,
    quiet_hours_start TIME,
    quiet_hours_end TIME,
    PRIMARY KEY (user_id, channel)
);
```

Scalability Considerations

- **Priority queues**: Process high-priority (2FA codes) before newsletters
- **Rate limiting**: Per-user and per-channel limits
- **Batching**: Batch digest emails (daily summary instead of per-event)
- **Multi-provider**: Fallback to secondary provider if primary fails
- **Async processing**: Never block the caller

Trade-offs

Decision	Option A	Option B	Recommendation
Queue	Single queue	Priority queues	**Priority** - urgent first
Delivery	At-least-once	Exactly-once	**At-least-once** - simpler
Templates	Inline	Centralized	**Centralized** - reusable
Tracking	Basic (sent)	Full (opens)	**Full** - better analytics

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	Event streaming for notification queue	Tech Guide
Redis	Rate limiting and user preference caching	Tech Guide
WebSocket	Real-time in-app notifications	Tech Guide
SignalR	Managed push notification service	Tech Guide

How to Explain It Well

> **Opening**: "A notification service needs reliability and low latency for time-sensitive messages. I'll design for 1B daily notifications across email, SMS, and push."

> **Key talking points**:
>
> - "Priority queues ensure 2FA codes go out before marketing emails"
> - "User preferences checked before sending - respect quiet hours"

> - "Exponential backoff with dead letter queue for failures"
> - "Track delivery status and engagement (opens, clicks)"
> - "Multi-provider setup for failover (SendGrid → SES)"

---

13. Optimize Cost While Maintaining Performance

Key Terms:

Term	Full Name	What it does
RI	Reserved Instance	Pre-committed compute for 1-3 years at discount
Spot Instance	Spare Capacity VM	Unused cloud capacity at 60-90% discount (can be evicted)
vCPU	Virtual CPU	One thread of a physical CPU core
SLA	Service Level Agreement	Guaranteed uptime/performance commitment
FinOps	Financial Operations	Practice of managing cloud costs
Auto-scaling	Automatic scaling	Adjusting resources based on demand

Problem Statement

Design strategies to optimize cloud costs in Azure while maintaining system performance and reliability.

Clarifying Questions to Ask

1. **Current spend**: What's the monthly cloud bill?
2. **Workload type**: Steady or variable traffic?
3. **Performance SLAs**: What latency/availability is required?
4. **Environment**: Prod/dev/test breakdown?
5. **Data requirements**: Hot vs. cold data ratio?
6. **Commitments**: Willing to commit for discounts?

7. **Flexibility**: Can workloads tolerate interruption?

Design Reasoning

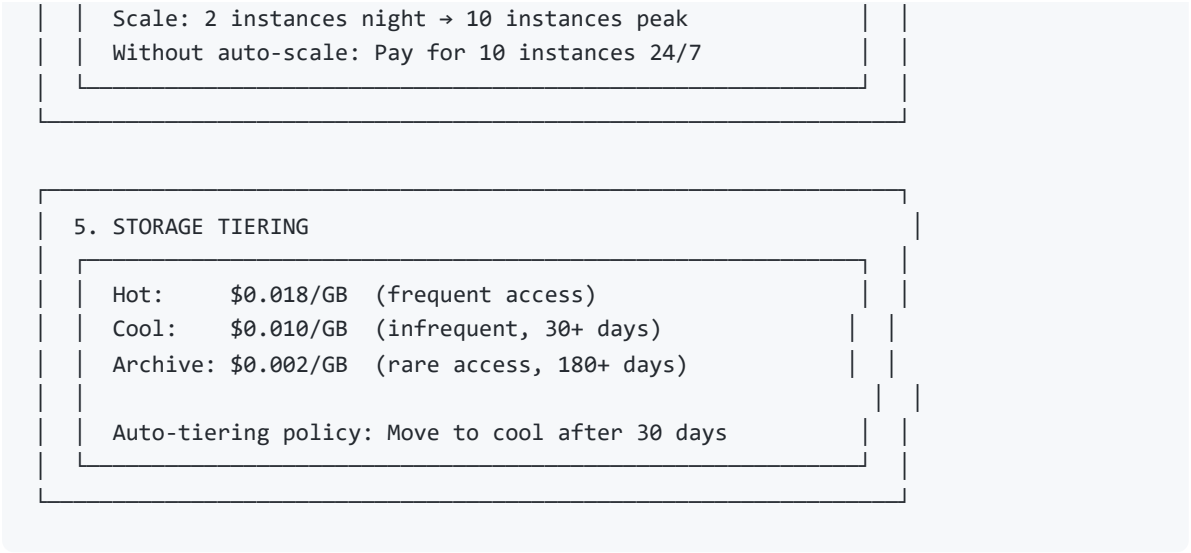
> **Core philosophy**: Layer multiple cost strategies — right-size first, commit to reservations for baseline, use spot and auto-scale for variable load, and tier storage by access frequency — while preserving SLA guarantees.

Decision	Choice	Why	Alternative Considered
Baseline Compute	Reserved Instances (1-3 year)	30-50% savings on always-on workloads with predictable capacity	On-demand only — pays full price for steady-state load that never changes
Variable Compute	Spot Instances + Auto-scaling	60-90% savings for interruptible batch work; auto-scale matches real demand	Over-provisioned on-demand — paying for peak capacity 24/7 when traffic is variable
Storage	Tiered lifecycle (Hot/Cool/Archive)	Auto-migrate data by age; up to 80% savings on infrequently accessed data	Single hot tier — wastes budget on compliance and historical data rarely read
VM Sizing	Right-size based on utilization metrics	Most VMs run at 15% CPU; downsizing saves 30-50% with no performance impact	Keeping over-provisioned VMs — paying for unused CPU and memory
Non-production Environments	Auto-shutdown schedules (nights/weekends)	Dev/test VMs idle 65% of the week; shutting down eliminates that waste	Running 24/7 — paying for idle dev resources no one uses off-hours
Cost Governance	Azure Cost Management + tagging + budgets	Visibility into spend per team/project; alerts before budget overruns	No governance — leads to uncontrolled cloud sprawl and surprise bills

Compute Generation	Latest-generation VM families	Better price-to-performance ratio than older generations	Previous-gen VMs — often more expensive per compute unit with lower performance

High-Level Design





API Design

Cost Management APIs

Method	Endpoint	Description
`GET`	`/api/v1/costs/summary`	Get cost breakdown by service
`GET`	`/api/v1/costs/forecast?period=30d`	Forecast future costs
`POST`	`/api/v1/budgets`	Create a budget alert
`GET`	`/api/v1/budgets/{budgetId}/status`	Get budget utilization

Performance Monitoring APIs

Method	Endpoint	Description
`GET`	`/api/v1/metrics/latency?service={name}&period=1h`	Get latency metrics
`GET`	`/api/v1/metrics/throughput`	Get throughput metrics
`GET`	`/api/v1/recommendations`	Get cost optimization recommendations
`POST`	`/api/v1/recommendations/{id}/apply`	Apply a recommendation

Scaling APIs

Method	Endpoint	Description

`PUT`	`/api/v1/autoscale/{service}/rules`	Configure autoscaling rules
`GET`	`/api/v1/autoscale/{service}/history`	Get scaling event history

Key Strategies

Strategy	Savings	Trade-off	Best For
Right-sizing	20-50%	May need scaling later	Overprovisioned VMs
Reserved Instances	30-50%	Upfront commitment	Steady workloads
Spot Instances	60-90%	Can be interrupted	Batch, CI/CD
Auto-scaling	30-70%	Complexity	Variable traffic
Storage tiering	50-80%	Access latency	Old data
Serverless	Variable	Cold start latency	Sporadic workloads

Implementation Checklist

```
IMMEDIATE WINS (Week 1):
□ Delete unused resources (orphaned disks, old snapshots)
□ Stop dev/test environments nights/weekends
□ Enable auto-shutdown for dev VMs
□ Review and remove unused IPs, load balancers

SHORT-TERM (Month 1):
□ Analyze utilization, right-size VMs
□ Move to reserved instances for baseline
□ Implement storage lifecycle policies
□ Use spot instances for batch jobs

LONG-TERM (Quarter):
□ Architect for serverless where appropriate
□ Implement auto-scaling policies
□ Multi-region optimization (use cheaper regions)
□ Establish FinOps practices and budgets
```

Monitoring and Governance

```
AZURE COST MANAGEMENT:
┌──────────────────────────────────────────────────────────────┐
│  1. Set budgets and alerts (80%, 90%, 100% thresholds)       │
│  2. Tag resources for cost allocation by team/project        │
│  3. Review Advisor recommendations weekly                     │
```

```
│  4. Use Cost Analysis for trend identification                  │
│  5. Implement Azure Policy to prevent expensive resources      │
└────────────────────────────────────────────────────────────────┘

COST ALLOCATION TAGS:
  {
    "team": "platform",
    "project": "search",
    "environment": "production",
    "cost-center": "CC-1234"
  }
```

Trade-offs

Decision	Option A	Option B	Recommendation
Compute	On-demand	Reserved	**Mix** - RI for base, on-demand for burst
Availability	Multi-region	Single region	**Based on SLA** - multi costs 2x
Data retention	Keep all	Lifecycle rules	**Lifecycle** - archive old data
Instance type	Latest gen	Previous gen	**Latest** - often better price/perf

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
AKS/Kubernetes	Auto-scaling pods and spot instance management	Tech Guide
Redis	Caching layer to reduce compute and database costs	Tech Guide
CDN	Edge caching to reduce origin server load and bandwidth costs	Tech Guide

How to Explain It Well

> **Opening**: "Cost optimization balances spending with performance. I focus on right-sizing, reservations, and auto-scaling."

> **Key talking points**:
>
> - "Right-size first - most VMs run at 15% CPU, can downsize 50%"
> - "Reserved instances for baseline load - 30-50% savings"
> - "Spot instances for batch jobs - 80% savings, handle interruption"
> - "Auto-scale to match traffic - don't pay for night capacity during day"
> - "Storage tiering: hot for active, archive for compliance data"

---

14. Design High-Availability Across Regions

Key Terms:

Term	Full Name	What it does
RTO	Recovery Time Objective	Max acceptable downtime after failure
RPO	Recovery Point Objective	Max acceptable data loss (time-based)
Active-Active	Multi-region deployment	Both regions serve traffic simultaneously
Active-Passive	Hot standby	Secondary region waits as backup
GRS	Geo-Redundant Storage	Azure storage replicated across regions
CAP	Consistency, Availability, Partition tolerance	Theorem: can only guarantee 2 of 3
WAF	Web Application Firewall	Protects against web exploits

Problem Statement

Design a system that remains available even when an entire Azure region fails, achieving 99.99%+ uptime.

Clarifying Questions to Ask

1. **RTO/RPO**: Recovery time and data loss tolerance?
2. **Traffic pattern**: Global users or region-specific?
3. **Data consistency**: Strong or eventual consistency acceptable?
4. **Cost constraints**: Budget for multi-region?
5. **Compliance**: Data residency requirements?
6. **Failover**: Automatic or manual failover?
7. **Active-active**: Both regions serve traffic or standby?

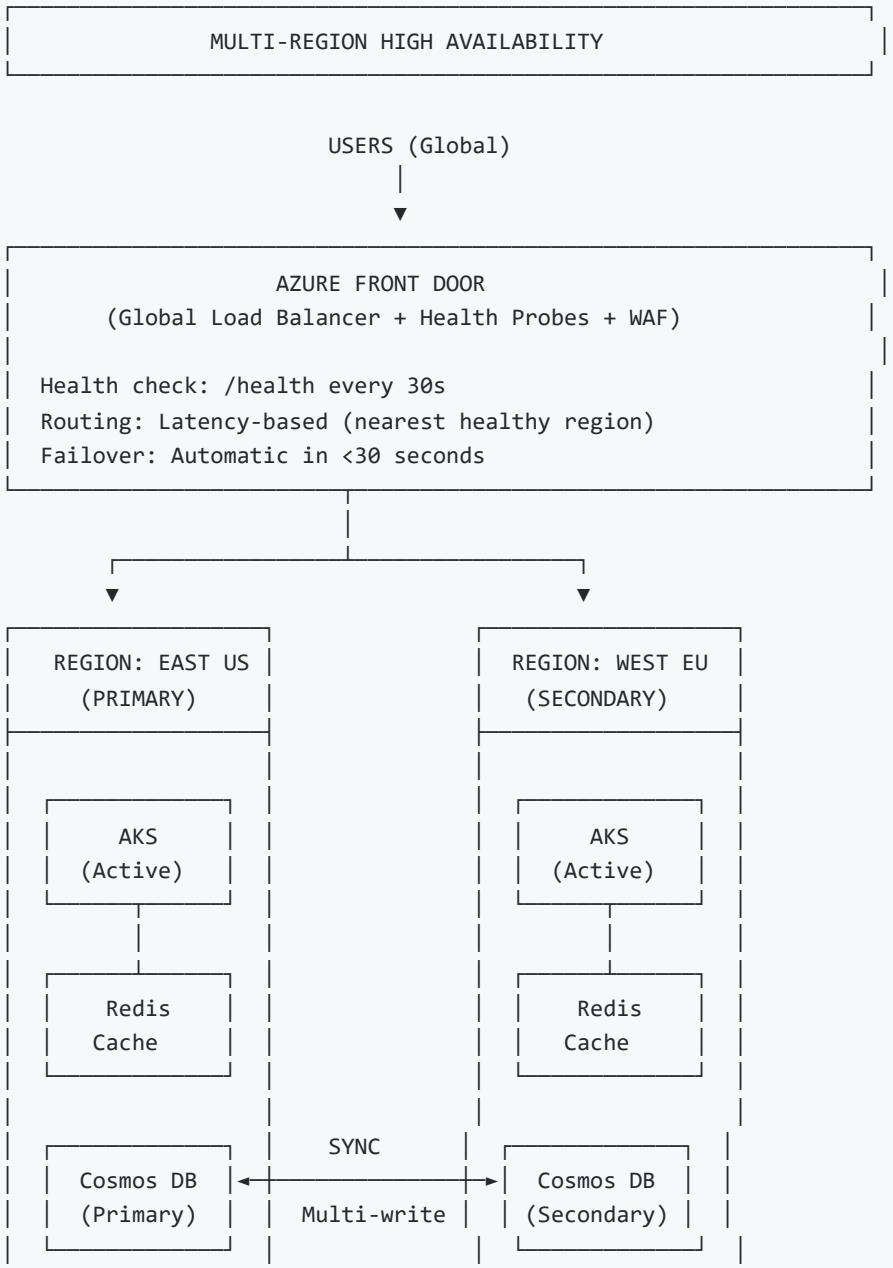
Design Reasoning

> **Core philosophy**: Eliminate every single point of failure through active-active multi-region deployment with automatic failover, accepting eventual consistency as the trade-off for continuous availability under the CAP theorem.

Decision	Choice	Why	Alternative Considered
Deployment Pattern	Active-Active (both regions serve traffic)	Instant failover, better latency for global users, full resource utilization	Active-Passive — wasted standby capacity, slower failover during outage
Global Load Balancer	Azure Front Door	Built-in health probes, latency-based routing, WAF, and <30s automatic failover	Traffic Manager — DNS-based only, slower failover due to TTL caching
Database	Cosmos DB with multi-region writes	Automatic geo-replication, configurable consistency, <10ms cross-region replication	PostgreSQL with read replicas — manual failover, no multi-write capability
Consistency Model	Eventual consistency	Maximizes availability across regions per CAP theorem; acceptable for most reads	Strong consistency — higher latency, reduced availability during network partitions
Regional Caching	Azure Cache for Redis (per region)	Low-latency reads without cross-region database round-trips	Shared central cache — cross-region latency defeats the purpose of caching

Data Replication	Asynchronous replication	Lower write latency and higher availability; RPO of seconds is acceptable	Synchronous — higher latency, risk of unavailability if secondary region is unreachable
Failover Mechanism	Automatic health-probe-driven	<60s recovery without human intervention; 24/7 protection	Manual failover — longer RTO, requires on-call engineer response

High-Level Design



```
└──────────────────┘              └──────────────────┘
```

API Design

Region Management APIs

Method	Endpoint	Description
`GET`	`/api/v1/regions`	List all regions and their status
`GET`	`/api/v1/regions/{region}/health`	Get region health details
`POST`	`/api/v1/failover`	Trigger manual failover
`GET`	`/api/v1/failover/history`	Get failover event history

Replication APIs

Method	Endpoint	Description
`GET`	`/api/v1/replication/status`	Get replication lag per region
`PUT`	`/api/v1/replication/config`	Update replication policy

Traffic Management APIs

Method	Endpoint	Description
`GET`	`/api/v1/traffic/distribution`	Get traffic routing weights
`PUT`	`/api/v1/traffic/distribution`	Update traffic weights between regions
`PUT`	`/api/v1/traffic/drain/{region}`	Drain traffic from a region

Deployment Patterns

```
ACTIVE-PASSIVE:
┌──────────────────────────────────────────────────────────────┐
│  Primary handles all traffic                                 │
│  Secondary is standby (data replicated)                      │
│  Failover: Switch DNS/traffic to secondary                   │
│                                                              │
│  Pros: Simpler, cheaper                                      │
│  Cons: Wasted secondary capacity, slower failover            │
│  Best for: Cost-sensitive, lower availability needs          │
```

```
└──────────────────────────────────────────────────────────────┘

ACTIVE-ACTIVE:
┌──────────────────────────────────────────────────────────────┐
│  Both regions serve traffic simultaneously                   │
│  Users routed to nearest region                              │
│  Failover: Automatic, near-instant                           │
│                                                              │
│  Pros: Better latency, instant failover, full utilization    │
│  Cons: Complex, data consistency challenges                  │
│  Best for: Global users, high availability requirements      │
└──────────────────────────────────────────────────────────────┘
```

Data Replication Strategies

```
SYNCHRONOUS REPLICATION:
  Write → Primary → Secondary → Ack to client
  RPO: 0 (no data loss)
  Cons: Higher latency, reduced availability

ASYNCHRONOUS REPLICATION:
  Write → Primary → Ack to client → Secondary (async)
  RPO: Seconds to minutes (some data loss possible)
  Pros: Lower latency, higher availability

COSMOS DB MULTI-REGION:
  - Configurable consistency levels
  - Automatic failover
  - Multi-write for active-active
  - <10ms replication latency
```

Failover Handling

```
AUTOMATIC FAILOVER SEQUENCE:
┌──────────────────────────────────────────────────────────────┐
│  T+0s:   Health probe fails in primary region                │
│  T+30s:  3 consecutive failures, region marked unhealthy     │
│  T+31s:  Traffic automatically routes to secondary           │
│  T+60s:  Alert sent to operations team                       │
│                                                              │
│  Total failover time: ~30-60 seconds                         │
└──────────────────────────────────────────────────────────────┘

HEALTH CHECK ENDPOINT:
  GET /health
  {
    "status": "healthy",
    "checks": {
      "database": "ok",
      "cache": "ok",
```

```
      "dependencies": "ok"
    }
  }
```

Key Components

Component	Purpose	Azure Service
Global LB	Route to healthy region	Azure Front Door
DNS failover	Automatic DNS updates	Traffic Manager
Database	Multi-region replication	Cosmos DB
Cache	Regional cache	Azure Cache for Redis
Storage	Geo-redundant storage	Azure Blob (GRS)
Messaging	Cross-region events	Event Grid / Service Bus

Trade-offs

Decision	Option A	Option B	Recommendation
Pattern	Active-Passive	Active-Active	**Active-Active** - better availability
Consistency	Strong	Eventual	**Eventual** - for availability
Failover	Manual	Automatic	**Automatic** - faster recovery
Replication	Sync	Async	**Async** - better latency

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Load Balancer	Global traffic distribution via Azure Front Door	Tech Guide
Raft Consensus	Leader election and state replication across regions	Tech Guide
Circuit Breaker	Prevent cascade failures during regional outages	Tech Guide

How to Explain It Well

> **Opening**: "High availability requires eliminating single points of failure. I'll design for 99.99% uptime with automatic failover under 60 seconds."

> **Key talking points**:
>
> - "Azure Front Door provides global load balancing with health probes"
> - "Active-active: both regions serve traffic, instant failover"
> - "Cosmos DB multi-write for consistent data across regions"
> - "Accept eventual consistency for availability (CAP theorem)"
> - "Health endpoints check all dependencies, not just the service"

---

16. Design a Rate Limiter

Key Terms:

Term	Full Name	What it does
Token Bucket	Rate limiting algorithm	Allows bursts up to bucket size, then rate-limits
Leaky Bucket	Rate limiting algorithm	Processes requests at constant rate, smooths bursts
Sliding Window	Rate limiting algorithm	Counts requests in rolling time window
429	HTTP Status Code	"Too Many Requests" - rate limit exceeded
Lua Script	Redis scripting	Atomic operations in Redis (check + update)
Fail Open	Failure mode	Allow traffic if rate limiter fails (availability > limiting)

Problem Statement

Design a rate limiting system that protects APIs from abuse while allowing legitimate traffic, supporting various limiting strategies.

Clarifying Questions to Ask

1. **Limit type**: Per user, per IP, per API key, or global?

2. **Rate**: What limits? (100 req/min per user?)
3. **Granularity**: Per endpoint or global for service?
4. **Response**: Block, queue, or degrade service?
5. **Distribution**: Single server or distributed system?
6. **Accuracy**: Strict enforcement or approximate OK?
7. **Burst**: Allow short bursts above limit?

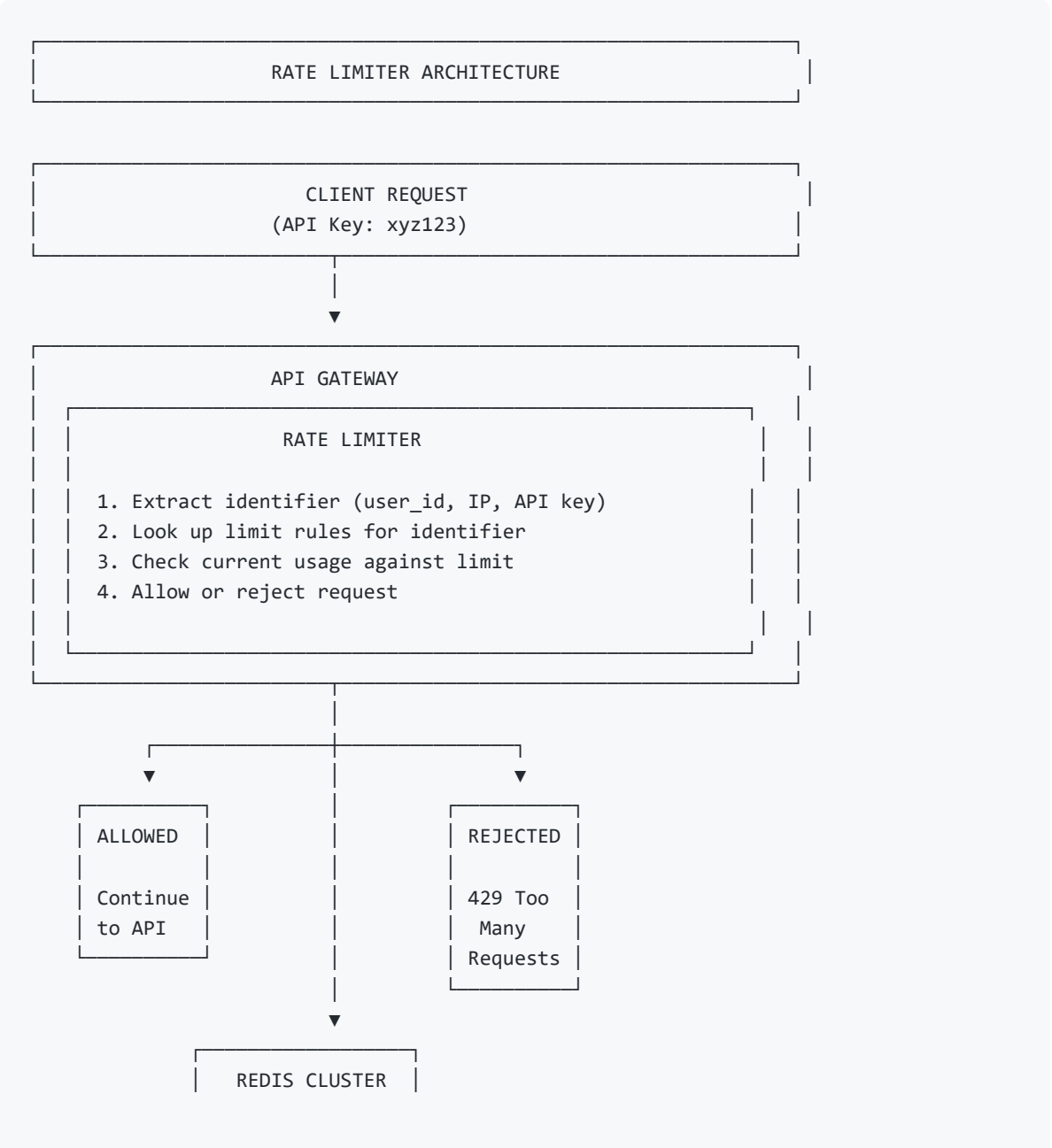
Design Reasoning

> **Core philosophy**: Enforce rate limits as close to the edge as possible using Redis for distributed shared state, with atomic Lua scripts to prevent race conditions and a fail-open policy to prioritize service availability over strict enforcement.

Decision	Choice	Why	Alternative Considered
Algorithm	Token Bucket	Allows controlled bursts while enforcing average rate; intuitive bucket-size and refill-rate params	Leaky Bucket (no burst tolerance) / Fixed Window (boundary spike problem)
State Store	Redis Cluster	Sub-millisecond lookups, native TTL for key expiration, shared across all API gateway instances	Local in-memory — inconsistent limits across servers in a distributed deployment
Atomicity	Redis Lua Scripts	Atomic check-and-decrement in one round-trip; prevents race conditions between concurrent requests	Separate GET + SET — race condition window allows over-limit requests through
Failure Mode	Fail Open (allow traffic)	Preserves service availability if Redis is down; brief over-limit is preferable to full outage	Fail Closed — blocks all traffic when limiter is unavailable, causing self-inflicted outage
Limit Granularity	Multi-tier (per-second + per-minute + per-hour)	Catches both short bursts and sustained abuse patterns simultaneously	Single time window — misses either burst abuse or slow sustained abuse

Client Identifier	API Key / Client ID	Consistent across requests regardless of IP; supports tiered limits per subscription plan	IP-based — fails behind NAT/proxies, unfairly penalizes users on shared networks
Response Communication	Standard headers (X-RateLimit-*, Retry-After)	Clients can self-throttle based on remaining quota, reducing rejected request volume	No headers — clients retry blindly, amplifying load during rate-limit events

High-Level Design



```
                │  (Rate Counters) │
                └──────────────────┘
```

API Design

Rate Limiter Configuration APIs

Method	Endpoint	Description
`POST`	`/api/v1/rules`	Create a rate limiting rule
`GET`	`/api/v1/rules`	List all rate limiting rules
`PUT`	`/api/v1/rules/{ruleId}`	Update a rule
`DELETE`	`/api/v1/rules/{ruleId}`	Delete a rule

Rate Check API (Internal)

Method	Endpoint	Description
`POST`	`/api/v1/check`	Check if request is allowed
`GET`	`/api/v1/usage?clientId={id}`	Get current usage for a client

Rate Limit Rule Example

```
POST /api/v1/rules
{
  "name": "api-standard-tier",
  "key": "client_id",
  "limits": [
    { "window": "1s", "max_requests": 10 },
    { "window": "1m", "max_requests": 200 },
    { "window": "1h", "max_requests": 5000 }
  ],
  "algorithm": "sliding_window",
  "action": "reject"  // or "throttle"
}
```

Response Headers (on every API response)

```
HTTP/1.1 200 OK                    // or 429 Too Many Requests
X-RateLimit-Limit: 200
X-RateLimit-Remaining: 150
```

```
X-RateLimit-Reset: 1672531260
Retry-After: 30                    // only on 429
```

Rate Limiting Algorithms

```
1. TOKEN BUCKET (Recommended for most cases)
┌─────────────────────────────────────────────────────────────────┐
│  Bucket holds tokens, refills at constant rate                  │
│  Each request consumes 1 token                                  │
│  If bucket empty → reject                                       │
│                                                                 │
│  Parameters:                                                    │
│    - Bucket size: 10 (max burst)                                │
│    - Refill rate: 1 token/second                                │
│                                                                 │
│  Time    Tokens    Request     Result                           │
│  0.0s    10        req1        Allow (9 tokens left)            │
│  0.1s    9         req2-10     Allow (0 tokens left)            │
│  0.2s    0         req11       REJECT                           │
│  1.0s    1         req12       Allow (0 tokens left)            │
└─────────────────────────────────────────────────────────────────┘

2. SLIDING WINDOW LOG
┌─────────────────────────────────────────────────────────────────┐
│  Store timestamp of each request                                │
│  Count requests in last N seconds                               │
│                                                                 │
│  Pros: Precise                                                  │
│  Cons: Memory-heavy (stores all timestamps)                     │
│                                                                 │
│  Redis: ZADD requests:<user> <timestamp> <request_id>           │
│         ZCOUNT requests:<user> <now - window> <now>             │
└─────────────────────────────────────────────────────────────────┘

3. SLIDING WINDOW COUNTER (Best balance)
┌─────────────────────────────────────────────────────────────────┐
│  Combine fixed window counts with weighted average              │
│                                                                 │
│  Current minute: 40 requests                                    │
│  Previous minute: 60 requests                                   │
│  Time into current minute: 30 seconds (50%)                     │
│                                                                 │
│  Weighted count = 40 + (60 * 0.5) = 70 requests                 │
│  Limit: 100/minute → Allow                                      │
└─────────────────────────────────────────────────────────────────┘

4. LEAKY BUCKET
┌─────────────────────────────────────────────────────────────────┐
│  Requests enter bucket, processed at constant rate              │
│  Smooths out bursts                                             │
│                                                                 │
│  Pros: Consistent output rate                                   │
```

```
│  Cons: Doesn't allow bursts                                    │
└────────────────────────────────────────────────────────────────┘
```

Redis Implementation (Token Bucket)

```lua
-- Lua script for atomic token bucket (runs in Redis)
local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2])
local now = tonumber(ARGV[3])
local requested = tonumber(ARGV[4])

local bucket = redis.call('HMGET', key, 'tokens', 'last_refill')
local tokens = tonumber(bucket[1]) or capacity
local last_refill = tonumber(bucket[2]) or now

-- Refill tokens based on time passed
local elapsed = now - last_refill
local refill = elapsed * refill_rate
tokens = math.min(capacity, tokens + refill)

-- Check if request can be allowed
if tokens >= requested then
    tokens = tokens - requested
    redis.call('HMSET', key, 'tokens', tokens, 'last_refill', now)
    redis.call('EXPIRE', key, 3600)
    return 1  -- Allowed
else
    redis.call('HMSET', key, 'tokens', tokens, 'last_refill', now)
    return 0  -- Rejected
end
```

Response Headers

```
HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1705312400
Retry-After: 30

{
  "error": "rate_limit_exceeded",
  "message": "Too many requests. Please retry after 30 seconds.",
  "retry_after": 30
}
```

Scalability Considerations

- **Distributed**: Use Redis Cluster for shared state
- **Local + global**: Local rate limit first, then global check
- **Race conditions**: Lua scripts for atomic operations
- **Failover**: Allow traffic if rate limiter unavailable (fail open)
- **Multi-tier**: Different limits per tier (free, pro, enterprise)

Trade-offs

Decision	Option A	Option B	Recommendation
Algorithm	Fixed window	Sliding window	**Sliding** - smoother
Storage	Local memory	Redis	**Redis** - for distributed
Fail mode	Fail open	Fail closed	**Fail open** - availability
Precision	Exact	Approximate	**Approximate** - simpler

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Rate Limiter	Token bucket and sliding window algorithms for traffic control	Tech Guide
Redis	Distributed counter storage with Lua scripts for atomic operations	Tech Guide

How to Explain It Well

> **Opening**: "Rate limiting protects services from abuse. I'll use token bucket for burst-friendly limiting with Redis for distributed state."

> **Key talking points**:
>
> - "Token bucket allows bursts up to bucket size, then rate-limited"
> - "Redis Lua scripts ensure atomic check-and-decrement"
> - "Sliding window counter balances precision and memory"
> - "Return standard headers: X-RateLimit-Remaining, Retry-After"

> - "Fail open if Redis unavailable - availability over rate limiting"

---

17. Implement Sharding and Partitioning

Key Terms:

Term	Full Name	What it does
Shard	Database partition	Horizontal slice of data on separate server
Shard Key	Partition key	Field used to determine which shard holds data
Consistent Hashing	Distribution algorithm	Minimizes data movement when adding/removing nodes
Hot Spot	Uneven load	One shard receiving disproportionate traffic
Scatter-Gather	Query pattern	Query all shards, merge results
Resharding	Rebalancing	Moving data when adding/removing shards

Problem Statement

Design a data sharding strategy for a large-scale database that distributes data across multiple servers while maintaining query efficiency.

Clarifying Questions to Ask

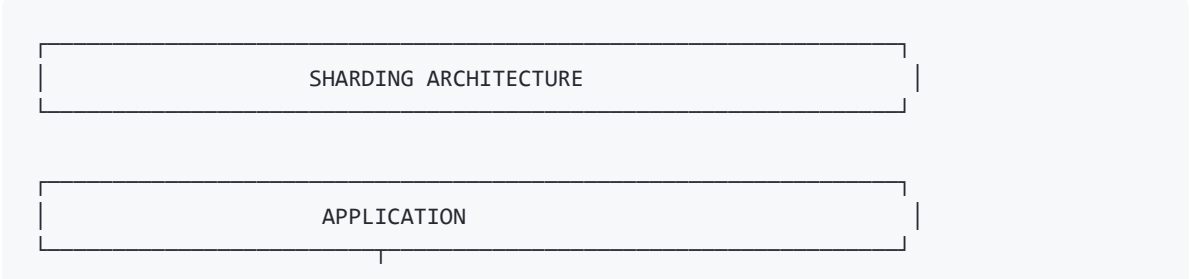
1. **Data size**: How much data? Growth rate?
2. **Query patterns**: Point lookups, range queries, aggregations?
3. **Access patterns**: Even distribution or hot spots?
4. **Consistency**: Strong or eventual consistency?
5. **Resharding**: How to add/remove nodes?
6. **Cross-shard**: Queries spanning multiple shards needed?
7. **Technology**: SQL or NoSQL database?

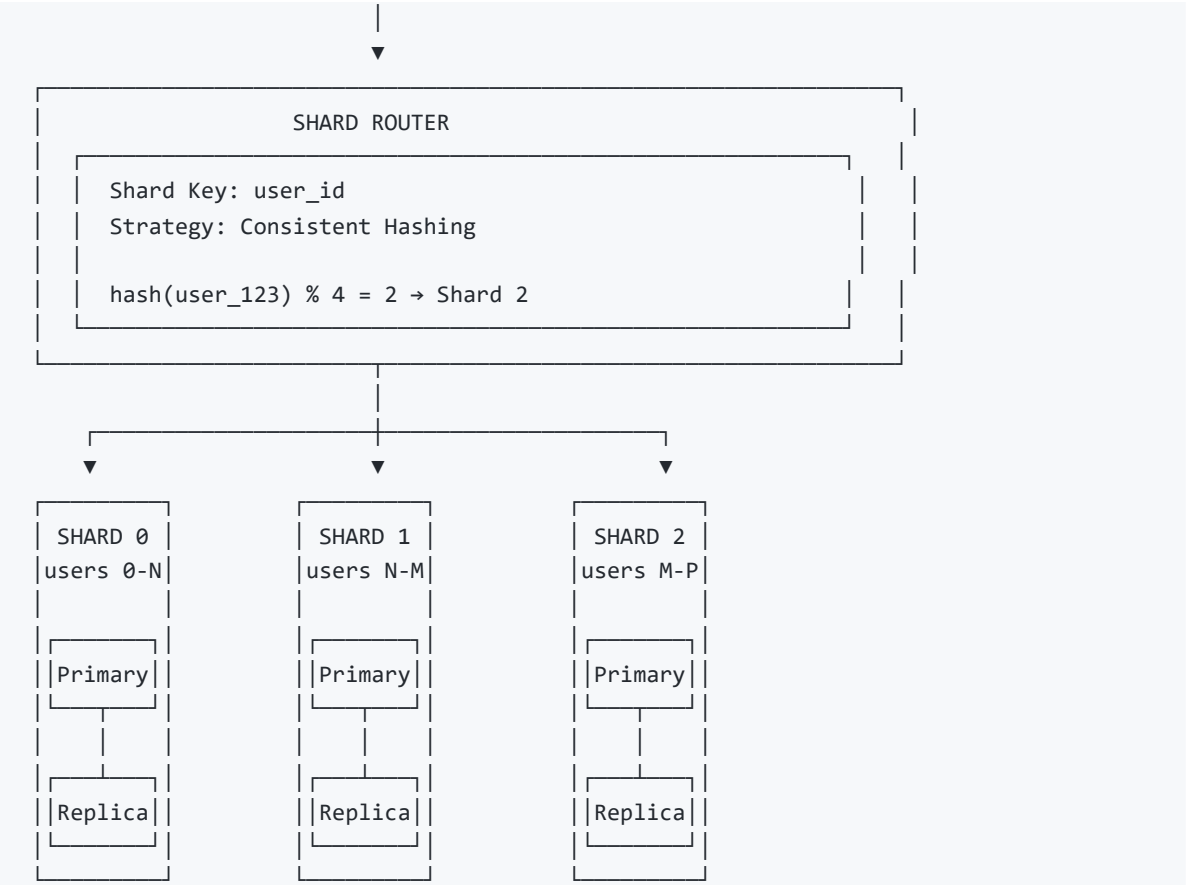
Design Reasoning

> **Core philosophy**: Distribute data using consistent hashing to minimize resharding disruption, choose shard keys aligned with query patterns to keep most operations single-shard, and design for zero-downtime shard management.

Decision	Choice	Why	Alternative Considered
Sharding Strategy	Consistent Hashing	Adding/removing nodes moves only 1/N of data; smooth horizontal scaling	Hash modulo — adding one shard reshuffles nearly all data across the cluster
Shard Key	Compound key (e.g., tenant_id + user_id)	Keeps related data co-located; avoids cross-shard queries for common access patterns	Single-field key — may split related data across shards, forcing scatter-gather
Reference Data	Global tables replicated to all shards	Enables local joins for small, rarely-changing data (countries, config, enums)	Cross-shard joins — slow scatter-gather for every lookup involving reference data
Resharding	Online dual-write migration	Zero downtime; both old and new shard mappings stay consistent during transition	Offline resharding — requires maintenance window, impacts availability
Cross-Shard Queries	Scatter-gather with result merging	Necessary fallback for analytics or non-shard-key queries	Banning all cross-shard queries — too restrictive, forces excessive denormalization
Replication	Primary-Replica per shard	Read scaling and fault tolerance within each shard; automatic promotion on failure	Single node per shard — no redundancy, shard failure causes data loss

High-Level Design





API Design

Shard Management APIs

Method	Endpoint	Description
`GET`	`/api/v1/shards`	List all shards and their status
`POST`	`/api/v1/shards/split`	Split a shard
`POST`	`/api/v1/shards/merge`	Merge two shards
`GET`	`/api/v1/shards/{shardId}/stats`	Get shard statistics (size, QPS)

Routing API (Internal)

Method	Endpoint	Description
`GET`	`/api/v1/route?key={shardKey}`	Resolve shard for a given key
`GET`	`/api/v1/mapping`	Get full shard mapping table

Resharding APIs

Method	Endpoint	Description
`POST`	`/api/v1/reshard`	Initiate resharding operation
`GET`	`/api/v1/reshard/{jobId}/status`	Get resharding progress
`POST`	`/api/v1/reshard/{jobId}/cancel`	Cancel resharding

Sharding Strategies

```
1. HASH-BASED SHARDING
┌─────────────────────────────────────────────────────────────────┐
│  shard_id = hash(shard_key) % num_shards                        │
│                                                                 │
│  Pros: Even distribution                                        │
│  Cons: Resharding requires data movement                        │
│                                                                 │
│  Example: hash("user_123") % 4 = 2 → Shard 2                    │
└─────────────────────────────────────────────────────────────────┘

2. RANGE-BASED SHARDING
┌─────────────────────────────────────────────────────────────────┐
│  Shard 0: A-F                                                   │
│  Shard 1: G-M                                                   │
│  Shard 2: N-S                                                   │
│  Shard 3: T-Z                                                   │
│                                                                 │
│  Pros: Range queries efficient                                  │
│  Cons: Hot spots (if 'S' names are popular)                     │
└─────────────────────────────────────────────────────────────────┘

3. CONSISTENT HASHING (Recommended)
┌─────────────────────────────────────────────────────────────────┐
│  Nodes placed on hash ring                                      │
│  Data placed on ring, assigned to next node clockwise           │
│                                                                 │
│         ┌──────────────────┐                                    │
│        /                    \                                   │
│       │    ●Node A          │                                   │
│       │         ○ data1     │                                   │
│       │              ●Node B │                                  │
│       │    ○ data2          │                                   │
│        \         ●Node C    /                                   │
│         └──────────────────┘                                    │
│                                                                 │
│  Add node: Only data between new node and predecessor moves     │
│  Remove node: Only that node's data redistributes               │
└─────────────────────────────────────────────────────────────────┘

4. DIRECTORY-BASED SHARDING
```

```
┌─────────────────────────────────────────────────────────────┐
│  Lookup table maps keys to shards                           │
│                                                             │
│  user_123 → Shard 2                                         │
│  user_456 → Shard 0                                         │
│  user_789 → Shard 1                                         │
│                                                             │
│  Pros: Flexible, can handle hot spots                       │
│  Cons: Directory is single point of failure                 │
└─────────────────────────────────────────────────────────────┘
```

Choosing a Shard Key

```
GOOD SHARD KEYS:
┌─────────────────────────────────────────────────────────────┐
│  ✓ user_id - Even distribution, most queries are per-user   │
│  ✓ tenant_id - Multi-tenant SaaS, queries within tenant     │
│  ✓ order_id - E-commerce, orders are independent            │
│  ✓ Compound: (tenant_id, user_id) - Keeps tenant data together │
└─────────────────────────────────────────────────────────────┘

BAD SHARD KEYS:
┌─────────────────────────────────────────────────────────────┐
│  ✗ timestamp - All recent data on one shard (hot spot)      │
│  ✗ country - Uneven (US shard overloaded)                   │
│  ✗ status - Few values, poor distribution                   │
│  ✗ auto-increment ID - Sequential writes to one shard       │
└─────────────────────────────────────────────────────────────┘
```

Cross-Shard Operations

```
SCATTER-GATHER PATTERN:
┌─────────────────────────────────────────────────────────────┐
│  Query: SELECT * FROM orders WHERE total > 1000             │
│  (No shard key in WHERE clause)                             │
│                                                             │
│  1. Router sends query to ALL shards (scatter)              │
│  2. Each shard executes query locally                       │
│  3. Router merges results (gather)                          │
│                                                             │
│  Performance: O(num_shards) - avoid if possible             │
└─────────────────────────────────────────────────────────────┘

GLOBAL TABLES:
┌─────────────────────────────────────────────────────────────┐
│  Small, rarely-changing tables replicated to all shards     │
│  Example: countries, currencies, config                     │
│                                                             │
```

```
│  Allows local joins without cross-shard queries                │
└────────────────────────────────────────────────────────────────┘
```

Resharding Strategy

```
ONLINE RESHARDING (Zero-downtime):
┌────────────────────────────────────────────────────────────────┐
│  1. Add new shards to cluster                                  │
│  2. Start dual-writes to old and new locations                 │
│  3. Background migration of existing data                      │
│  4. Verify data consistency                                    │
│  5. Switch reads to new shard mapping                          │
│  6. Stop writes to old locations                               │
│  7. Remove old shard assignments                               │
│                                                                │
│  Tools: Vitess (MySQL), Citus (PostgreSQL)                     │
└────────────────────────────────────────────────────────────────┘
```

Trade-offs

Decision	Option A	Option B	Recommendation
Strategy	Hash	Range	**Hash** - better distribution
Shard key	Single	Compound	**Compound** - more flexibility
Cross-shard	Allow	Avoid	**Design to avoid** - better perf
Resharding	Offline	Online	**Online** - zero downtime

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Consistent Hashing	Minimize data movement when adding/removing shards	Tech Guide

How to Explain It Well

> **Opening**: "Sharding distributes data across servers for scale. I'll use consistent hashing for even distribution and minimal resharding impact."

> **Key talking points**:
>
> - "Consistent hashing: adding a node moves only 1/N of data"
> - "Choose shard key based on query patterns - usually user_id"
> - "Avoid cross-shard queries - design schema to keep related data together"
> - "Global tables for small reference data to enable local joins"
> - "Online resharding with dual-writes for zero downtime"

---

22. Design Logging & Monitoring

Key Terms:

Term	Full Name	What it does
APM	Application Performance Monitoring	Tracks app performance, errors, traces
RED	Rate, Errors, Duration	Key metrics methodology for services
Trace	Distributed trace	Request flow across multiple services
Span	Trace segment	Single operation within a trace
P99	99th percentile	Latency that 99% of requests are faster than
Sidecar	Container pattern	Logging agent running alongside app container
OpenTelemetry	Observability framework	Vendor-neutral instrumentation standard

Problem Statement

Design a logging and monitoring infrastructure for microservices that enables debugging, alerting, and performance analysis.

Clarifying Questions to Ask

1. **Scale**: How many services? Log volume per day?
2. **Retention**: How long to keep logs? (30 days? 1 year?)

3. **Features**: Search, dashboards, alerting?
4. **Latency**: Real-time or near-real-time acceptable?
5. **Compliance**: Audit log requirements?
6. **Correlation**: Trace requests across services?
7. **Cost**: Budget for storage and compute?

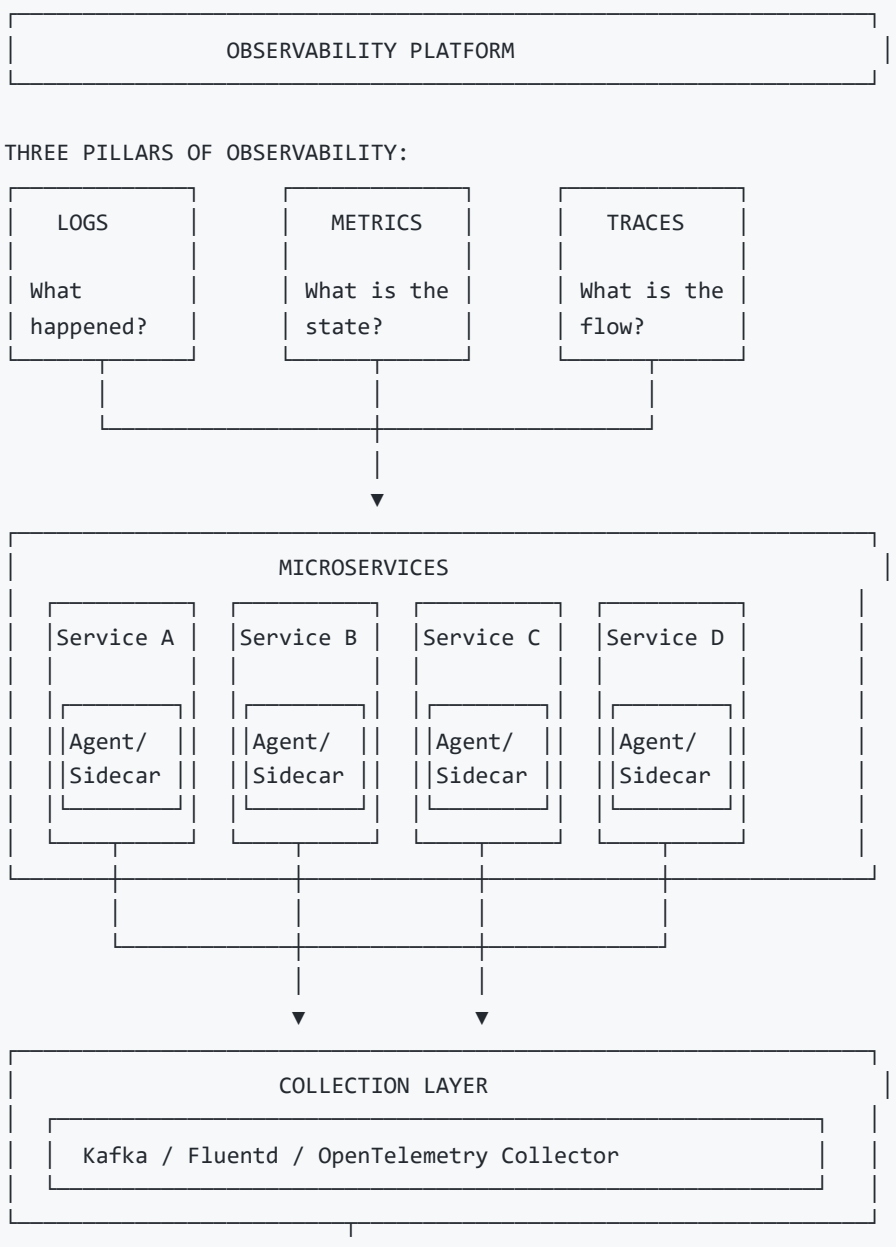
Design Reasoning

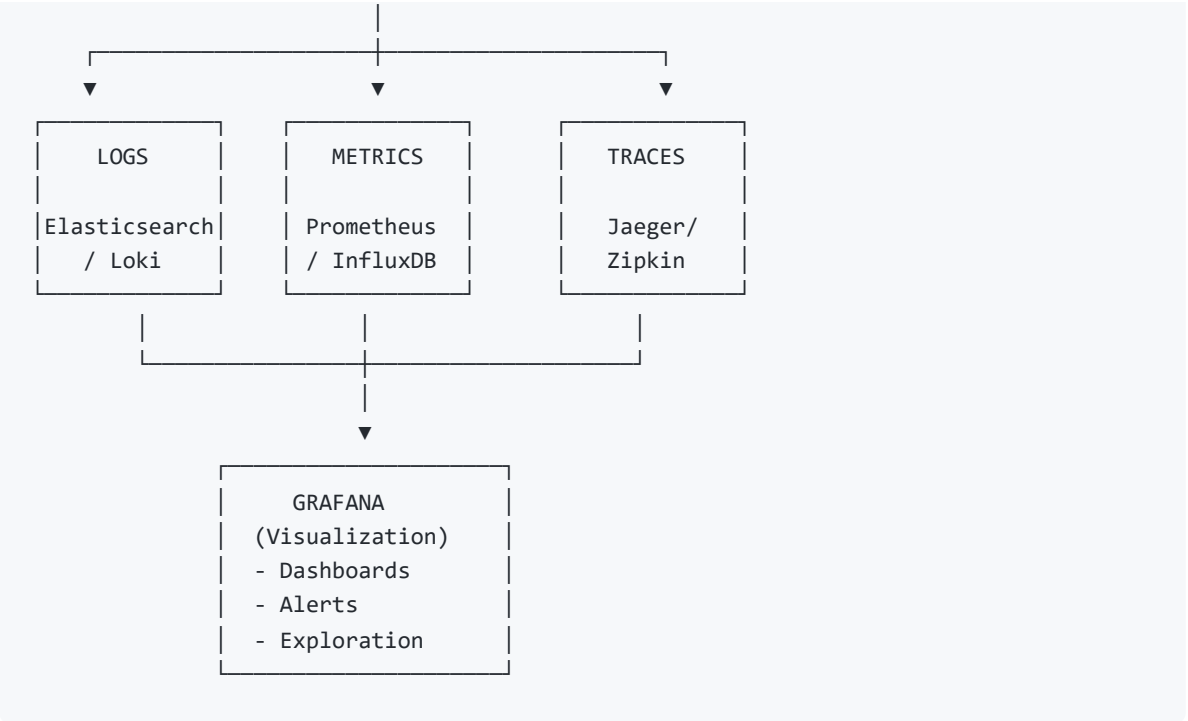
> **Core philosophy**: Unify logs, metrics, and traces as three pillars of observability, with a shared collection pipeline and correlated trace IDs across all telemetry.

Decision	Choice	Why	Alternative Considered
Collection pattern	Sidecar agent per pod	Kubernetes-native; isolates collection from application; survives app crashes	Per-node agent (shared resource contention, harder to configure per-service)
Instrumentation	OpenTelemetry	Vendor-neutral standard avoids lock-in; single SDK for logs, metrics, and traces	Vendor-specific SDKs (lock-in, separate instrumentation per pillar)
Log storage	Loki (label-based)	10x cheaper than Elasticsearch at scale; indexes only labels, not full text	Elasticsearch (powerful full-text search but expensive storage and indexing)
Metrics backend	Prometheus	Pull-based model with rich ecosystem; PromQL is the de facto standard for alerting rules	InfluxDB (push-based, smaller ecosystem, less Kubernetes integration)
Tracing backend	Jaeger	Distributed tracing with Kafka-backed ingestion for reliability; integrates with OpenTelemetry	Zipkin (simpler but fewer features for large-scale trace analysis)
Visualization	Grafana	Single pane of glass across all three pillars; supports Loki,	Kibana (Elasticsearch-only, cannot unify metrics and traces)

		Prometheus, and Jaeger as data sources	
Sampling strategy	Adaptive (100% errors, 1% success)	Captures all failures for debugging while controlling storage costs on healthy traffic	Fixed sampling rate (misses rare errors, or oversamples healthy requests)
Log format	Structured JSON with trace_id	Machine-parseable; trace_id enables correlation across logs, metrics, and spans	Unstructured text (requires regex parsing, no cross-pillar correlation)

High-Level Design





API Design

Log Ingestion APIs

Method	Endpoint	Description
`POST`	`/api/v1/logs`	Ingest log entries (batch)
`POST`	`/api/v1/logs/stream`	Stream logs via Server-Sent Events

Log Query APIs

Method	Endpoint	Description
`POST`	`/api/v1/logs/search`	Search logs with filters
`GET`	`/api/v1/logs/tail?service={name}`	Live tail logs (SSE)

Metrics APIs

Method	Endpoint	Description
`POST`	`/api/v1/metrics`	Push custom metrics
`GET`	`/api/v1/metrics/query?metric={name}&from={t1}&to={t2}`	Query time-series metrics

Alert APIs

Method	Endpoint	Description
`POST`	`/api/v1/alerts/rules`	Create alert rule
`GET`	`/api/v1/alerts/rules`	List alert rules
`GET`	`/api/v1/alerts/active`	Get active alerts
`POST`	`/api/v1/alerts/{alertId}/acknowledge`	Acknowledge an alert

Tracing APIs

Method	Endpoint	Description
`POST`	`/api/v1/traces`	Submit trace spans
`GET`	`/api/v1/traces/{traceId}`	Get full distributed trace
`GET`	`/api/v1/traces/search?service={name}&minDuration={ms}`	Search traces

Key Components

Component	Purpose	Technology
Log Agent	Collect and ship logs	Fluentd, Vector
Log Store	Index and search	Elasticsearch, Loki
Metrics Collector	Scrape metrics	Prometheus
Metrics Store	Time-series storage	Prometheus, InfluxDB
Trace Collector	Distributed tracing	Jaeger, Zipkin
Visualization	Dashboards and alerts	Grafana
APM	Application performance	Application Insights

Structured Logging

```
LOG ENTRY (Structured JSON):
{
```

```
    "timestamp": "2024-01-15T14:30:00.123Z",
    "level": "ERROR",
    "service": "order-service",
    "version": "1.2.3",
    "trace_id": "abc123",
    "span_id": "def456",
    "user_id": "user_789",
    "message": "Failed to process order",
    "error": {
      "type": "PaymentDeclinedException",
      "message": "Card declined",
      "stack_trace": "..."
    },
    "context": {
      "order_id": "order_123",
      "amount": 99.99,
      "currency": "USD"
    },
    "host": "pod-xyz-123",
    "environment": "production"
  }

  LOG LEVELS:
    DEBUG → Development debugging (not in prod)
    INFO  → Normal operations
    WARN  → Potential issues
    ERROR → Failures requiring attention
    FATAL → System unable to continue
```

Distributed Tracing

```
REQUEST FLOW ACROSS SERVICES:
┌──────────────────────────────────────────────────────────────────┐
│  User Request: POST /orders                                      │
│  Trace ID: abc123 (propagated through all services)              │
│                                                                  │
│  ┌──────────────────────────────────────────────────────┐        │
│  │ API Gateway    [span: abc123-001] ──────────────────┤        │
│  │   │                                          50ms   │        │
│  │   ▼                                                 │        │
│  │ Order Service  [span: abc123-002] ──────────────┐   │        │
│  │   │                                      30ms   │   │        │
│  │   ├──▶ Inventory Service [span: abc123-003] ──┐ │   │        │
│  │   │                                    15ms   │ │   │        │
│  │   │                                           │ │   │        │
│  │   ├──▶ Payment Service   [span: abc123-004] ──┐ │ │   │        │
│  │   │                                  20ms     │ │ │   │        │
│  │   │                                           │ │ │   │        │
│  │   └──▶ Notification Svc  [span: abc123-005] ┐ │ │ │   │        │
│  │                                      5ms    │ │ │ │   │        │
│  └─────────────────────────────────────────────┴─┴─┴─┴───┘        │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

```
│  Total: 50ms | Bottleneck: Payment Service (20ms)              │
└──────────────────────────────────────────────────────────────┘

CONTEXT PROPAGATION:
  HTTP Header: traceparent: 00-abc123-def456-01
  Passes trace_id and span_id to downstream services
```

Metrics Collection

```
KEY METRICS (RED Method):
┌──────────────────────────────────────────────────────────────┐
│  Rate:   Requests per second                                 │
│  Errors: Error rate (%)                                      │
│  Duration: Latency percentiles (p50, p95, p99)               │
└──────────────────────────────────────────────────────────────┘

PROMETHEUS METRICS EXAMPLE:
  # Request count
  http_requests_total{service="order", method="POST", status="200"}

  # Latency histogram
  http_request_duration_seconds_bucket{le="0.1"} 1000
  http_request_duration_seconds_bucket{le="0.5"} 1500
  http_request_duration_seconds_bucket{le="1.0"} 1520

GOLDEN SIGNALS:
  - Latency: How long requests take
  - Traffic: How many requests
  - Errors: Rate of failed requests
  - Saturation: How full is the service (CPU, memory)
```

Alerting Strategy

```
ALERT DEFINITIONS:
┌──────────────────────────────────────────────────────────────┐
│  CRITICAL (Page on-call):                                    │
│    - Error rate > 5% for 5 minutes                           │
│    - P99 latency > 2s for 5 minutes                          │
│    - Service completely down                                 │
│                                                              │
│  WARNING (Slack notification):                               │
│    - Error rate > 1% for 15 minutes                          │
│    - P95 latency > 1s for 10 minutes                         │
│    - Disk usage > 80%                                        │
│                                                              │
│  INFO (Dashboard only):                                      │
│    - Deployment completed                                    │
│    - Scaling event                                           │
└──────────────────────────────────────────────────────────────┘
```

```
ALERT FATIGUE PREVENTION:
  - Group related alerts
  - Suppress during maintenance windows
  - Escalation paths with timeouts
  - Runbook links in alert messages
```

Scalability Considerations

- **Log volume**: Sample debug logs in production (1% sampling)
- **Retention**: Hot storage (30 days) → Cold storage (1 year)
- **Indexing**: Only index searchable fields, store rest as JSON
- **Cardinality**: Limit label cardinality in metrics
- **Correlation**: Use consistent trace_id across all telemetry

Trade-offs

Decision	Option A	Option B	Recommendation
Log storage	Elasticsearch	Loki	**Loki** - cheaper, label-based
Metrics	Prometheus	InfluxDB	**Prometheus** - ecosystem
Collection	Agent per node	Sidecar	**Sidecar** - Kubernetes native
Sampling	All traces	Sample 1%	**Adaptive** - all errors, sample success

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	Log collection and distribution pipeline	Tech Guide
Elasticsearch	Log indexing and full-text search	Tech Guide

How to Explain It Well

> **Opening**: "Observability requires logs, metrics, and traces working together. I'll design for debugging, alerting, and performance analysis."

> **Key talking points**:
>
> - "Structured JSON logs with trace_id for correlation"

> - "OpenTelemetry for vendor-neutral instrumentation"
> - "RED metrics: Rate, Errors, Duration for every service"
> - "Distributed tracing to identify bottlenecks across services"
> - "Alert on symptoms (error rate) not causes (CPU) to reduce noise"

---

23. Design a Load Balancer

Key Terms:

Term	Full Name	What it does
L4	Layer 4 (Transport)	Routes based on IP/port, doesn't inspect content
L7	Layer 7 (Application)	Routes based on HTTP headers, URL, cookies
VIP	Virtual IP	Single IP address representing multiple servers
VRRP	Virtual Router Redundancy Protocol	Failover protocol for LB high availability
Anycast	Network routing	Same IP announced from multiple locations
SSL Termination	TLS decryption	LB decrypts HTTPS, forwards HTTP to backends
Health Check	Backend monitoring	Periodic probes to detect unhealthy servers

Problem Statement

Design a load balancer that distributes traffic across multiple backend servers with health checking, session persistence, and high availability.

Clarifying Questions to Ask

1. **Layer**: L4 (TCP) or L7 (HTTP)?
2. **Scale**: How many requests per second? Backend servers?

3. **Features**: SSL termination, session stickiness, rate limiting?
4. **Health checks**: Active probing or passive monitoring?
5. **Algorithm**: Round-robin, least connections, weighted?
6. **Availability**: Active-passive or active-active LB?
7. **Protocol**: HTTP, gRPC, WebSocket, raw TCP?

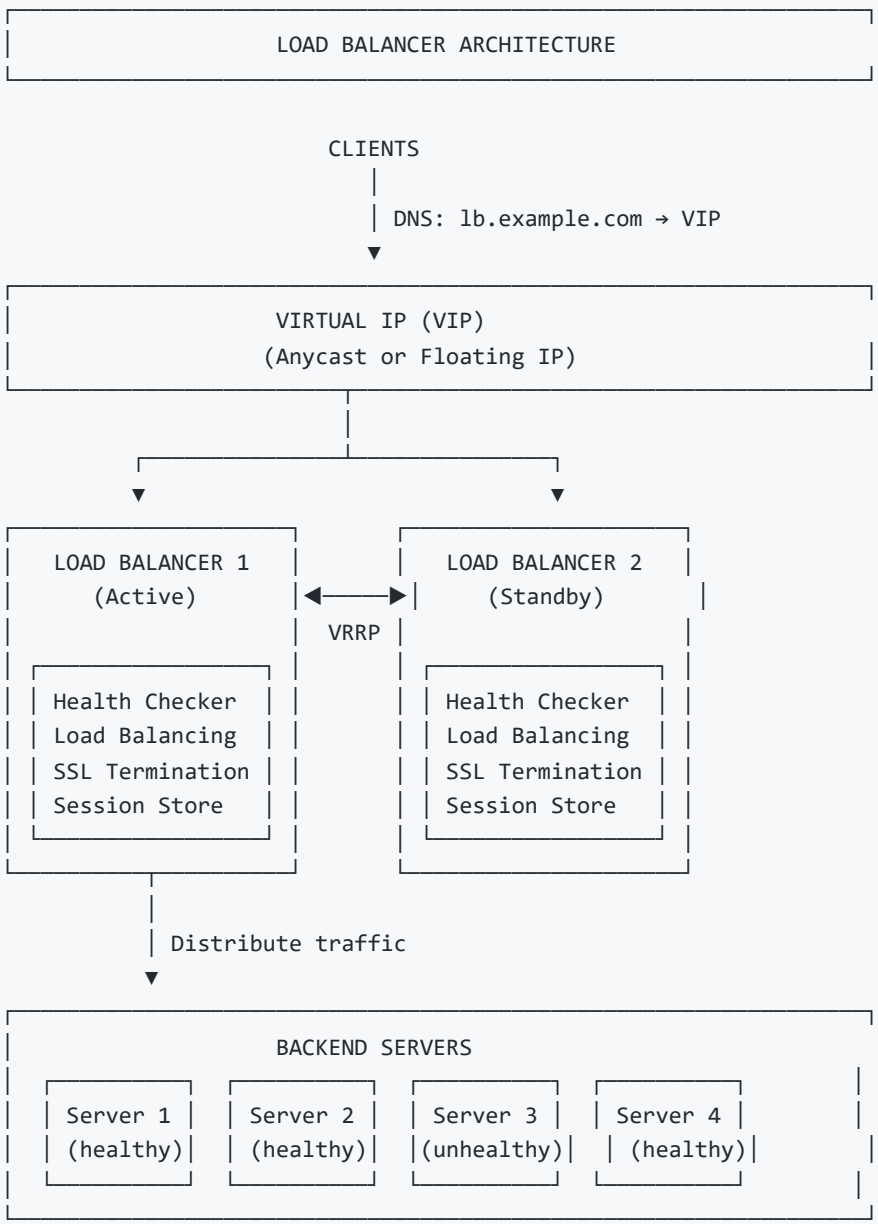
Design Reasoning

> **Core philosophy**: An L7 load balancer provides content-aware routing and SSL termination, with active-passive HA ensuring the balancer itself is never a single point of failure.

Decision	Choice	Why	Alternative Considered
Layer	L7 (Application)	Enables content-based routing (/api vs /static), SSL termination, and header inspection for microservices	L4 (faster but cannot route by URL path, headers, or cookies)
Algorithm	Least connections	Handles varying request durations gracefully; avoids overloading slow backends	Round robin (ignores current server load, poor for mixed workloads)
High availability	Active-passive with VRRP	Floating VIP failover in 1-3 seconds; simple and battle-tested for LB redundancy	Active-active (more complex state synchronization, higher operational cost)
SSL handling	SSL termination at LB	Offloads TLS from backends, enables L7 inspection; re-encrypt to backends if required	End-to-end SSL (higher backend CPU, LB cannot inspect traffic for routing)
Session persistence	External session store (Redis)	Stateless backends enable true horizontal scaling; sessions survive backend failures	Sticky sessions (creates hotspots, sessions lost when backend dies)

Health checking	Active + passive combined	Active probes catch crashed servers; passive monitoring catches degradation that probes miss	Active only (misses issues visible only under real traffic load)

High-Level Design



API Design

Load Balancer Configuration APIs

Method	Endpoint	Description
`POST`	`/api/v1/pools`	Create a backend pool
`PUT`	`/api/v1/pools/{poolId}`	Update pool configuration
`POST`	`/api/v1/pools/{poolId}/targets`	Add backend target
`DELETE`	`/api/v1/pools/{poolId}/targets/{targetId}`	Remove backend target

Health Check APIs

Method	Endpoint	Description
`GET`	`/api/v1/pools/{poolId}/health`	Get health status of all backends
`PUT`	`/api/v1/pools/{poolId}/healthcheck`	Configure health check rules

Routing Rules APIs

Method	Endpoint	Description
`POST`	`/api/v1/rules`	Create routing rule (path/header-based)
`GET`	`/api/v1/rules`	List routing rules
`PUT`	`/api/v1/rules/{ruleId}`	Update routing rule

Monitoring APIs

Method	Endpoint	Description
`GET`	`/api/v1/stats`	Get request distribution stats
`GET`	`/api/v1/stats/latency?backend={id}`	Get per-backend latency
`GET`	`/api/v1/stats/connections`	Get active connection count

Load Balancing Algorithms

```
1. ROUND ROBIN
┌──────────────────────────────────────────────────────────────────┐
│  Request 1 → Server A                                            │
│  Request 2 → Server B                                            │
│  Request 3 → Server C                                            │
│  Request 4 → Server A (cycle repeats)                           |
|                                                                   |
```

```
│  Pros: Simple, even distribution                                  │
│  Cons: Ignores server load/capacity                               │
└──────────────────────────────────────────────────────────────────┘

2. WEIGHTED ROUND ROBIN
┌──────────────────────────────────────────────────────────────────┐
│  Server A: weight=3, Server B: weight=1                           │
│                                                                   │
│  Request 1 → A                                                    │
│  Request 2 → A                                                    │
│  Request 3 → A                                                    │
│  Request 4 → B                                                    │
│                                                                   │
│  Use case: Heterogeneous servers (different capacities)           │
└──────────────────────────────────────────────────────────────────┘

3. LEAST CONNECTIONS
┌──────────────────────────────────────────────────────────────────┐
│  Server A: 10 active connections                                  │
│  Server B: 5 active connections                                   │
│  Server C: 8 active connections                                   │
│                                                                   │
│  New request → Server B (least connections)                       │
│                                                                   │
│  Use case: Long-lived connections, varying request duration       │
└──────────────────────────────────────────────────────────────────┘

4. IP HASH (Consistent Hashing)
┌──────────────────────────────────────────────────────────────────┐
│  hash(client_ip) % num_servers = server_index                     │
│                                                                   │
│  Client 1.2.3.4 → always Server A                                 │
│  Client 5.6.7.8 → always Server B                                 │
│                                                                   │
│  Use case: Session stickiness without cookies                     │
└──────────────────────────────────────────────────────────────────┘

5. LEAST RESPONSE TIME
┌──────────────────────────────────────────────────────────────────┐
│  Track average response time per server                           │
│  Route to server with lowest response time                        │
│                                                                   │
│  Use case: Optimize for latency                                   │
└──────────────────────────────────────────────────────────────────┘
```

Health Checking

```
ACTIVE HEALTH CHECKS:
┌──────────────────────────────────────────────────────────────────┐
│  LB periodically probes each backend:                             │
│                                                                   │
│  GET /health HTTP/1.1                                             │
│  Host: backend-server                                             │
```

```
│                                                                  │
│  Healthy response:                                               │
│    HTTP 200 OK                                                   │
│    {"status": "healthy", "db": "ok", "cache": "ok"}              │
│                                                                  │
│  Unhealthy thresholds:                                           │
│    - 3 consecutive failures → mark unhealthy                     │
│    - 2 consecutive successes → mark healthy                      │
│                                                                  │
│  Interval: 5-10 seconds                                          │
└──────────────────────────────────────────────────────────────────┘

PASSIVE HEALTH CHECKS:
┌──────────────────────────────────────────────────────────────────┐
│  Monitor real traffic for failures:                              │
│    - 5xx errors                                                  │
│    - Connection timeouts                                         │
│    - Connection resets                                           │
│                                                                  │
│  If error rate > threshold → mark unhealthy                      │
│  Pros: No extra traffic, catches issues active checks miss       │
└──────────────────────────────────────────────────────────────────┘
```

Session Persistence (Sticky Sessions)

```
METHODS:
┌──────────────────────────────────────────────────────────────────┐
│  1. Cookie-based:                                                │
│     Set-Cookie: SERVERID=server-a; Path=/                        │
│     LB reads cookie, routes to same server                       │
│                                                                  │
│  2. Source IP:                                                   │
│     hash(client_ip) → consistent server                          │
│     Problem: NAT, proxies share IP                               │
│                                                                  │
│  3. Application-generated session ID:                            │
│     LB inspects session_id header/cookie                         │
│     Maintains session→server mapping                             │
│                                                                  │
│  Best practice: Use external session store (Redis)               │
│  instead of sticky sessions for stateless backends               │
└──────────────────────────────────────────────────────────────────┘
```

L4 vs L7 Load Balancing

```
L4 (Transport Layer):
┌──────────────────────────────────────────────────────────────────┐
│  Routes based on: IP address, TCP port                           │
│  Cannot inspect: HTTP headers, cookies, URL path                 │
│                                                                  │
```

```
│  Pros: Fast, low overhead, protocol-agnostic                    │
│  Cons: Limited routing options                                  │
│                                                                 │
│  Use case: Database connections, raw TCP, high throughput       │
└─────────────────────────────────────────────────────────────────┘

L7 (Application Layer):
┌─────────────────────────────────────────────────────────────────┐
│  Routes based on: URL path, headers, cookies, body              │
│                                                                 │
│  /api/* → API servers                                           │
│  /static/* → CDN or static servers                              │
│  Header: X-Feature: beta → Beta servers                         │
│                                                                 │
│  Pros: Flexible routing, SSL termination, caching               │
│  Cons: Higher latency, more resource intensive                  │
│                                                                 │
│  Use case: HTTP APIs, microservices, content-based routing      │
└─────────────────────────────────────────────────────────────────┘
```

High Availability

```
ACTIVE-PASSIVE WITH VRRP:
┌─────────────────────────────────────────────────────────────────┐
│                    Virtual IP: 10.0.0.100                       │
│                           │                                     │
│              ┌────────────┴────────────┐                        │
│              ▼                         ▼                        │
│      ┌──────────────┐           ┌──────────────┐                │
│      │    LB 1      │           │    LB 2      │                │
│      │  (Active)    │◄─────────►│  (Standby)   │                │
│      │ 10.0.0.101   │ Heartbeat │ 10.0.0.102   │                │
│      └──────────────┘           └──────────────┘                │
│                                                                 │
│  If LB1 fails:                                                  │
│    1. Heartbeat timeout detected                                │
│    2. LB2 claims Virtual IP                                     │
│    3. Failover in 1-3 seconds                                   │
└─────────────────────────────────────────────────────────────────┘
```

Trade-offs

Decision	Option A	Option B	Recommendation
Layer	L4	L7	**L7** - more routing options
Algorithm	Round-robin	Least connections	**Least conn** - handles variance
Stickiness	Sticky sessions	External session	**External** - better scaling

Health check	Active	Active + Passive	**Both** - comprehensive

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Load Balancer	L4/L7 traffic distribution with health checking	Tech Guide
Consistent Hashing	IP hash algorithm for session stickiness	Tech Guide

How to Explain It Well

> **Opening**: "A load balancer distributes traffic for scalability and reliability. I'll design an L7 LB with health checking and HA."

> **Key talking points**:
>
> - "L7 enables content-based routing: /api to API servers, /static to CDN"
> - "Least connections algorithm handles varying request durations"
> - "Active health checks every 5 seconds, passive monitors real traffic"
> - "VRRP for HA: standby takes over VIP in 1-3 seconds"
> - "Avoid sticky sessions - use external session store (Redis)"

---

24. Handle Eventual Consistency

Key Terms:

Term	Full Name	What it does
CAP Theorem	Consistency, Availability, Partition tolerance	Trade-off: can only guarantee 2 of 3
LWW	Last Write Wins	Conflict resolution using timestamps
CRDT	Conflict-free Replicated Data Type	Data structures that auto-merge

Vector Clock	Distributed timestamp	Tracks causality across nodes
Idempotency	Same result on retry	Operation can be safely repeated
Optimistic UI	Immediate feedback	Show change before server confirms

Problem Statement

Design strategies to handle eventual consistency in distributed databases while providing good user experience and data integrity.

Clarifying Questions to Ask

1. **Data type**: What data can tolerate eventual consistency?
2. **Delay tolerance**: How much delay is acceptable? (seconds? minutes?)
3. **Conflict resolution**: How to handle concurrent updates?
4. **Read patterns**: Do users read their own writes frequently?
5. **Cross-region**: Geographic distribution involved?
6. **Compensation**: Can incorrect operations be undone?
7. **Visibility**: Should users know about consistency delays?

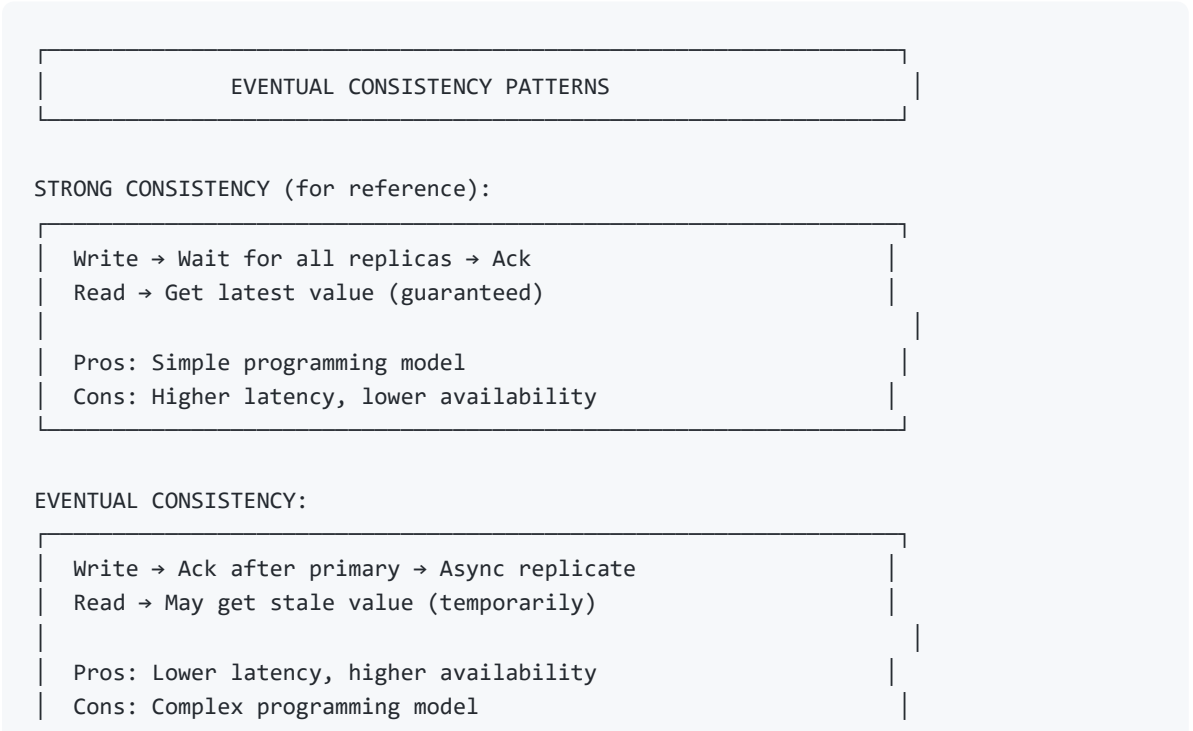
Design Reasoning

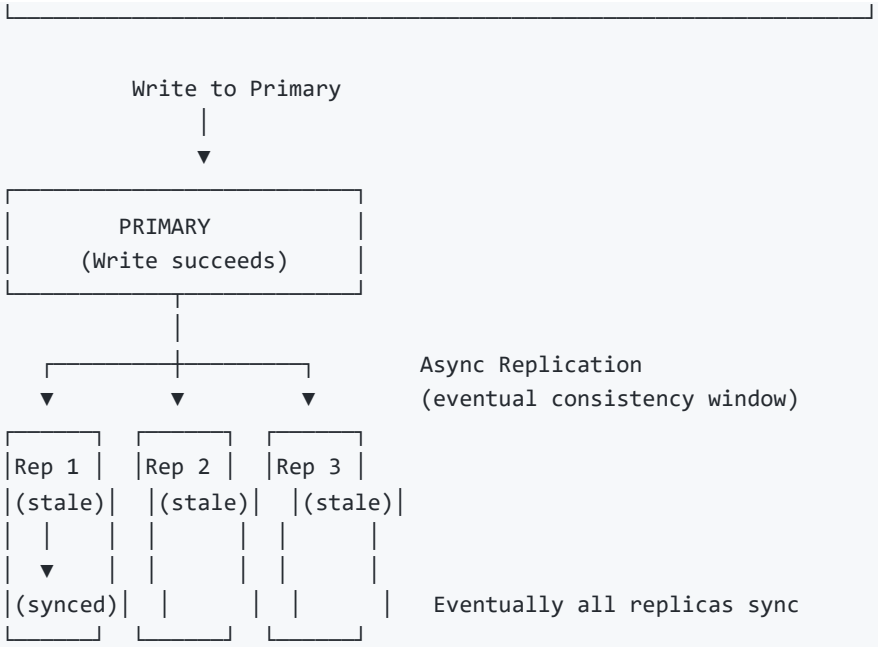
> **Core philosophy**: Accept eventual consistency for availability and latency, then layer patterns (read-your-own-writes, optimistic UI, idempotency) to make it invisible to users.

Decision	Choice	Why	Alternative Considered
Default consistency	Eventual with per-query overrides	Keeps reads fast; critical operations can opt into strong consistency when needed	Strong consistency everywhere (high latency, reduced availability)
User's own writes	Sticky session to primary (time-bounded)	Users always see their own updates; routes to primary for 5 seconds after write, then back to replicas	Always read from primary (defeats the purpose of replicas, creates bottleneck)

Conflict resolution	Domain-specific merge functions	Shopping carts use union merge (no data loss); profiles use LWW (latest edit wins)	Single global strategy (LWW loses data for additive operations like carts)
Concurrent updates	CRDTs where applicable	Conflict-free data types auto-merge without coordination; ideal for counters, sets, and flags	Locking / pessimistic concurrency (kills availability in distributed setting)
Client experience	Optimistic UI with background reconciliation	Users see instant feedback; background sync confirms or corrects within seconds	Synchronous confirmation (perceived latency increases, worse UX)
Retry safety	Idempotency keys per operation	Client-generated keys allow safe retries without duplicate side effects (double-charges, duplicate posts)	Server-side dedup window (memory overhead, time-limited, not fully reliable)
Monotonic reads	Session-versioned reads	Track highest version seen per user session; never serve older data than previously observed	No guarantee (users see version go backward on refresh, confusing UX)

High-Level Design





API Design

Consistency-Aware Read APIs

Method	Endpoint	Description
`GET`	`/api/v1/resources/{id}`	Read with eventual consistency (default)
`GET`	`/api/v1/resources/{id}?consistency=strong`	Read with strong consistency
`GET`	`/api/v1/resources/{id}?consistency=session`	Read-your-own-writes consistency
`GET`	`/api/v1/resources/{id}?consistency=bounded&staleness=5s`	Bounded staleness read

Write APIs (with consistency feedback)

```
POST /api/v1/resources
{
  "data": { ... },
  "consistency_level": "quorum"
}

Response: 201 Created
{
```

```
    "id": "res_123",
    "version": 5,
    "write_timestamp": 1672531200000,
    "replicated_to": ["region-1", "region-2"],
    "pending_regions": ["region-3"]
  }
```

Conflict Resolution APIs

Method	Endpoint	Description
`GET`	`/api/v1/resources/{id}/versions`	List conflicting versions
`POST`	`/api/v1/resources/{id}/resolve`	Resolve conflict manually

Patterns for Handling Eventual Consistency

```
1. READ YOUR OWN WRITES
┌─────────────────────────────────────────────────────────────┐
│  Problem: User updates profile, refreshes, sees old data    │
│                                                             │
│  Solution A: Sticky sessions to primary                     │
│    - Route user's reads to primary after their writes       │
│    - Track: user_123 wrote at T, read from primary until T+5s │
│                                                             │
│  Solution B: Version token                                  │
│    - Write returns version: v=42                            │
│    - Read includes: If-None-Match: v=42                     │
│    - If replica is behind, route to primary                 │
│                                                             │
│  Solution C: Local cache + background refresh               │
│    - Show optimistic update immediately                     │
│    - Background sync confirms or corrects                   │
└─────────────────────────────────────────────────────────────┘

2. MONOTONIC READS
┌─────────────────────────────────────────────────────────────┐
│  Problem: User sees v=5, refreshes, sees v=4 (went backward)  │
│                                                             │
│  Solution: Session-consistent reads                         │
│    - Track highest version seen per user                    │
│    - Only serve reads from replicas at or above that version  │
│    - If no replica is current enough, wait or route to primary │
└─────────────────────────────────────────────────────────────┘

3. CAUSAL CONSISTENCY
┌─────────────────────────────────────────────────────────────┐
│  Problem: User A posts, User B comments, User C sees comment  │
│           but not the original post (causality violated)    │
│                                                             │
```

```
│  Solution: Causal ordering with vector clocks                    │
│    - Each write includes dependency list                         │
│    - Reads wait for dependencies to be visible                   │
│                                                                  │
│  Post: {id: 1, deps: []}                                         │
│  Comment: {id: 2, deps: [1]}                                     │
│  Read comment → ensure post 1 is visible first                   │
└──────────────────────────────────────────────────────────────────┘
```

Conflict Resolution Strategies

```
1. LAST WRITE WINS (LWW)
┌──────────────────────────────────────────────────────────────────┐
│  Each update has timestamp                                       │
│  Latest timestamp wins during merge                              │
│                                                                  │
│  Node A: {value: "A", ts: 100}                                   │
│  Node B: {value: "B", ts: 105}                                   │
│  Merge result: "B" wins                                          │
│                                                                  │
│  Pros: Simple, deterministic                                     │
│  Cons: Data loss (A's update silently lost)                      │
└──────────────────────────────────────────────────────────────────┘

2. MERGE FUNCTION (Application-specific)
┌──────────────────────────────────────────────────────────────────┐
│  Shopping cart example:                                          │
│  Node A: {items: ["apple", "banana"]}                            │
│  Node B: {items: ["apple", "cherry"]}                            │
│  Merge: {items: ["apple", "banana", "cherry"]} (union)           │
│                                                                  │
│  Counter example:                                                │
│  Node A: {count: +5}                                             │
│  Node B: {count: +3}                                             │
│  Merge: {count: +8} (sum of deltas)                              │
└──────────────────────────────────────────────────────────────────┘

3. OPERATIONAL TRANSFORMATION / CRDT
┌──────────────────────────────────────────────────────────────────┐
│  Design data structures that merge automatically                 │
│                                                                  │
│  G-Counter (grow-only counter):                                  │
│  Node A: {A: 5, B: 0}                                            │
│  Node B: {A: 0, B: 3}                                            │
│  Merge: {A: 5, B: 3} → total = 8                                 │
│                                                                  │
│  Pros: Always converges, no conflicts                            │
│  Cons: Limited data structure types                              │
└──────────────────────────────────────────────────────────────────┘
```

User Experience Patterns

```
OPTIMISTIC UI:
┌─────────────────────────────────────────────────────────────┐
│  1. User clicks "Like"                                      │
│  2. Immediately show liked state (optimistic)               │
│  3. Send request to server in background                    │
│  4. If fails → revert UI + show error                       │
│                                                             │
│  User perceives instant response                            │
└─────────────────────────────────────────────────────────────┘

LOADING STATES + CONFIRMATION:
┌─────────────────────────────────────────────────────────────┐
│  1. User submits order                                      │
│  2. Show "Processing..." state                              │
│  3. Poll or WebSocket for confirmation                      │
│  4. Show "Order confirmed" when synced                      │
└─────────────────────────────────────────────────────────────┘

IDEMPOTENCY KEYS:
┌─────────────────────────────────────────────────────────────┐
│  Client generates unique key per operation:                 │
│    POST /orders                                             │
│    Idempotency-Key: order-abc123                            │
│                                                             │
│  Server deduplicates:                                       │
│    If seen → return cached response                         │
│    If new → process and store response                      │
│                                                             │
│  Safe to retry on timeout/failure                           │
└─────────────────────────────────────────────────────────────┘
```

Trade-offs

Decision	Option A	Option B	Recommendation
Conflict resolution	LWW	Custom merge	**Domain-specific**
User writes	Read-after-write	Optimistic UI	**Optimistic** - better UX
Retries	Auto-retry	Idempotency	**Idempotency keys**
Visibility	Hide delays	Show status	**Show** - transparency

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference

Raft Consensus	Leader-based replication for strong consistency when needed	Tech Guide

How to Explain It Well

> **Opening**: "Eventual consistency trades immediate consistency for availability. I'll design patterns for good UX despite replication delays."

> **Key talking points**:
>
> - "Read-your-own-writes: route reads to primary after user's writes"
> - "Optimistic UI: show changes immediately, reconcile in background"
> - "Idempotency keys: safe retries without duplicate operations"
> - "CRDTs: design data structures that merge automatically"
> - "Conflict resolution depends on domain: LWW for profiles, merge for carts"

---

25. Design Real-Time Log Processing

Key Terms:

Term	Full Name	What it does
EPS	Events Per Second	Throughput measure for log systems
CEP	Complex Event Processing	Detecting patterns across multiple events
Lambda Architecture	Batch + Speed layers	Combines real-time and batch processing
Kappa Architecture	Stream-only	Simplified Lambda using only streaming
Backpressure	Flow control	Slowing producers when consumers are overwhelmed
Tumbling Window	Fixed time window	Non-overlapping aggregation periods
Sliding Window	Rolling time window	Continuously updated aggregation

Problem Statement

Design a system that processes billions of log events daily in real-time for alerting, analytics, and debugging.

Clarifying Questions to Ask

1. **Volume**: How many events per second? (100K+ EPS?)
2. **Latency**: Real-time (<1s) or near-real-time (<1min)?
3. **Processing**: Aggregations, pattern matching, anomaly detection?
4. **Output**: Alerts, dashboards, data warehouse?
5. **Replay**: Need to reprocess historical data?
6. **Retention**: How long to keep raw vs. aggregated?
7. **Schema**: Structured, semi-structured, or unstructured logs?

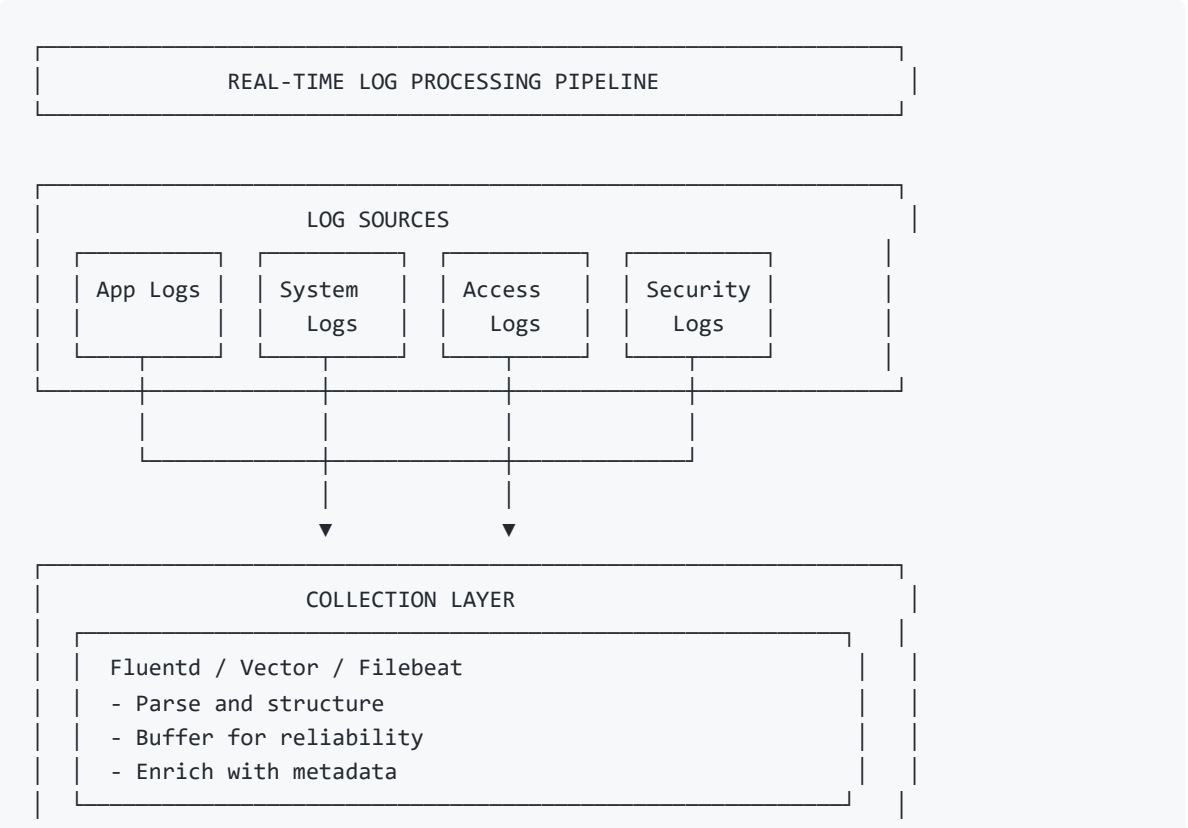
Design Reasoning

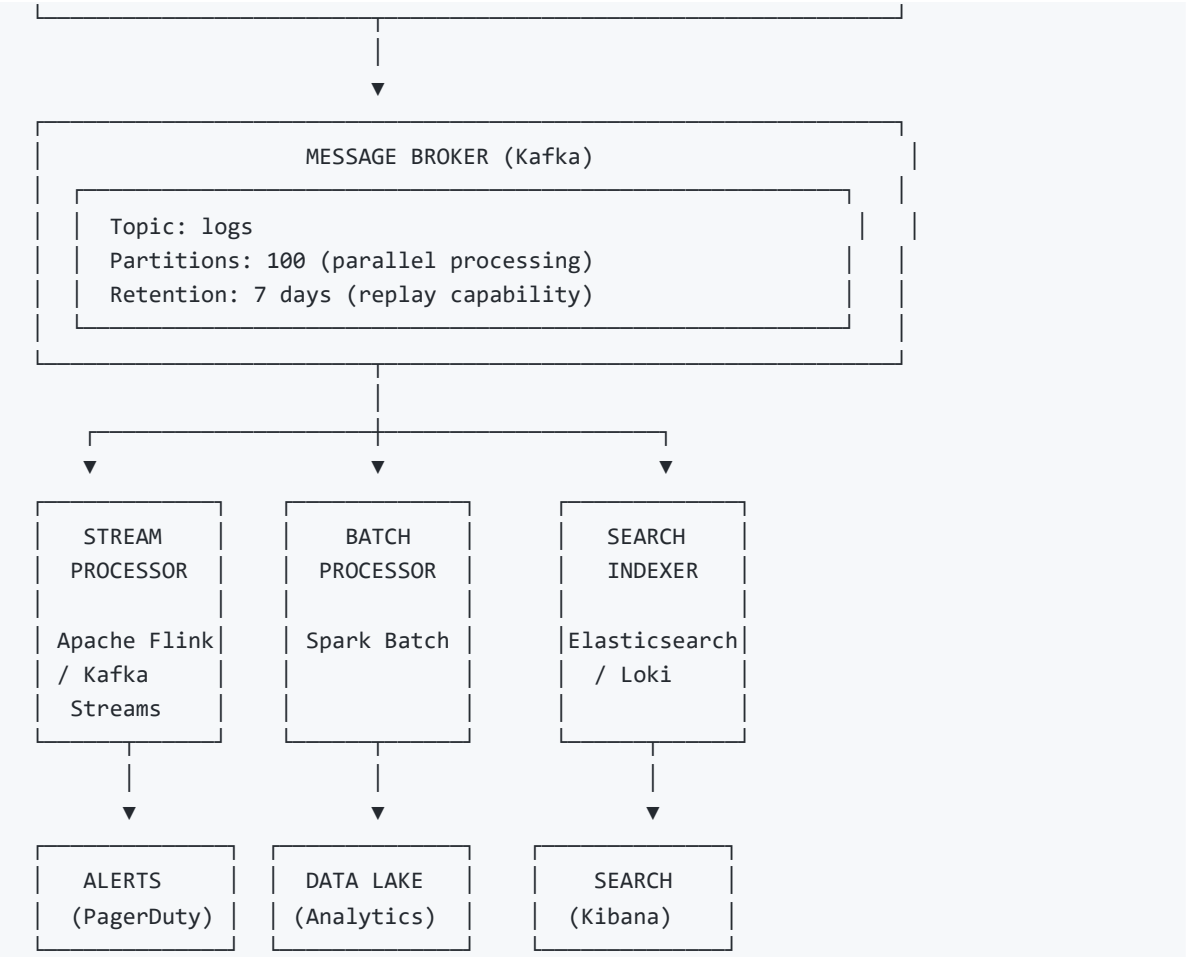
> **Core philosophy**: Kafka as the durable backbone provides replay capability and decouples producers from consumers, enabling both real-time alerting and batch analytics on the same data.

Decision	Choice	Why	Alternative Considered
Message broker	Kafka (100 partitions, 7-day retention)	Durable, replayable, and horizontally scalable; 7-day retention enables reprocessing after bug fixes	RabbitMQ (no built-in replay, not designed for high-throughput log volume)
Stream processing	Apache Flink	True per-event streaming with exactly-once semantics; CEP library for pattern detection (e.g., brute-force)	Kafka Streams (simpler but lacks CEP and advanced windowing)
Architecture	Kappa (stream-only) as default	Simpler than Lambda — single processing path reduces code duplication and operational overhead	Lambda (batch + speed layers — needed only if batch accuracy corrections are required)
Search engine	Loki (label-based indexing)	10x cheaper at scale than Elasticsearch; sufficient	Elasticsearch (powerful full-text but expensive

		for label-filtered log queries	storage for billions of daily events)
Long-term storage	Tiered: SSD → HDD → S3	Recent logs on fast storage for debugging; aggregated older data on cheap storage for compliance	Keep all on SSD (cost-prohibitive at billions of events per day)
Collection agents	Fluentd / Vector	Lightweight, buffer locally for reliability, enrich with metadata before shipping to Kafka	Direct application logging to Kafka (no buffering, app blocked if Kafka slow)
Flow control	Backpressure with buffering	Slow consumers buffer in Kafka; never drop logs even during traffic spikes	Drop on overflow (loses critical security and error logs during incidents)
Alerting pipeline	Dedicated consumer group on Kafka	Independent alert processing — alert lag doesn't affect search indexing or analytics	Shared consumer (alert latency increases when search indexer falls behind)

High-Level Design





API Design

Log Ingestion APIs

Method	Endpoint	Description
`POST`	`/api/v1/logs/ingest`	Ingest log batch (JSON array)
`POST`	`/api/v1/logs/stream`	Stream logs via chunked transfer

Pipeline Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/pipelines`	Create a processing pipeline
`GET`	`/api/v1/pipelines/{pipelineId}`	Get pipeline status
`PUT`	`/api/v1/pipelines/{pipelineId}`	Update pipeline config
`GET`	`/api/v1/pipelines/{pipelineId}/metrics`	Get pipeline throughput/lag

Query APIs

Method	Endpoint	Description
`POST`	`/api/v1/query`	Run ad-hoc query on processed logs
`GET`	`/api/v1/query/{queryId}/results`	Get async query results

Alert APIs

Method	Endpoint	Description
`POST`	`/api/v1/alerts`	Create real-time alert rule
`GET`	`/api/v1/alerts/active`	Get triggered alerts

Key Components

Component	Purpose	Technology
Collection	Gather and ship logs	Fluentd, Vector
Buffering	Decouple and buffer	Kafka
Stream Processing	Real-time analytics	Flink, Kafka Streams
Batch Processing	Historical analysis	Spark
Search	Ad-hoc queries	Elasticsearch
Storage	Long-term retention	S3 / Data Lake
Alerting	Notify on patterns	PagerDuty

Stream Processing Patterns

```
1. WINDOWED AGGREGATIONS
┌─────────────────────────────────────────────────────────────┐
│  Tumbling Window: Count errors per minute                   │
│                                                             │
│  |----min 1----|----min 2----|----min 3----|                │
│  [  50 errors  ][  120 errors][  45 errors ]                │
│                                                             │
│  If errors > 100 in any window → Alert                      │
└─────────────────────────────────────────────────────────────┘

│  Sliding Window: Rolling 5-minute error rate                │
│                                                             │
```

```
  |--window 1--|                                                    |
     |--window 2--|                                                 |
        |--window 3--|                                              |
                                                                     |
  Continuously updated as events arrive                             |
└──────────────────────────────────────────────────────────────────┘

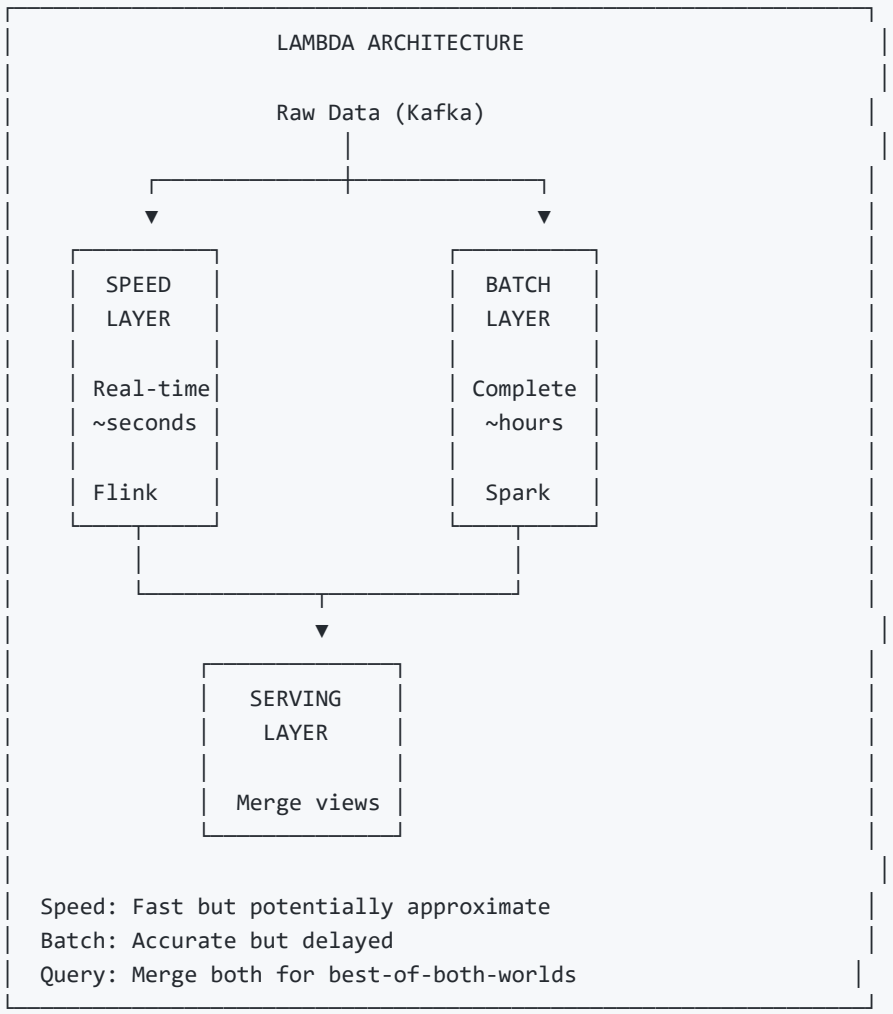
2. PATTERN DETECTION
┌──────────────────────────────────────────────────────────────────┐
  Complex Event Processing (CEP):                                   |
                                                                    |
  Pattern: Login failure × 5 from same IP within 1 minute           |
           → Trigger "brute force" alert                            |
                                                                    |
  Flink CEP:                                                        |
  Pattern.begin("start")                                            |
    .where(event.type == "login_failure")                           |
    .times(5)                                                       |
    .within(Time.minutes(1))                                        |
└──────────────────────────────────────────────────────────────────┘

3. ANOMALY DETECTION
┌──────────────────────────────────────────────────────────────────┐
  Statistical anomaly:                                              |
    - Compute rolling mean and std deviation                        |
    - Alert if value > mean + 3*stddev                              |
                                                                    |
  Example: Request latency                                          |
    Normal: 50ms ± 10ms                                             |
    Current: 150ms → Anomaly!                                       |
└──────────────────────────────────────────────────────────────────┘
```

Kafka Configuration for Log Processing

```
TOPIC CONFIGURATION:
┌──────────────────────────────────────────────────────────────────┐
  Topic: application-logs                                           |
  Partitions: 100 (parallelism)                                     |
  Replication Factor: 3 (durability)                                |
  Retention: 7 days (replay capability)                             |
                                                                    |
  Partition Key: service_name (group logs by service)               |
                                                                    |
  Consumer Groups:                                                  |
    - alerts-processor (real-time alerting)                         |
    - elasticsearch-indexer (search)                                |
    - analytics-processor (aggregations)                            |
    - archive-writer (S3 backup)                                    |
└──────────────────────────────────────────────────────────────────┘
```

Lambda Architecture



Scalability Considerations

- **Partitioning**: Kafka partitions = processing parallelism
- **Backpressure**: Handle spikes with buffering, don't drop logs
- **Sampling**: Sample debug logs in high-volume scenarios
- **Compaction**: Aggregate old data, keep recent detailed
- **Tiered storage**: Hot (SSD) → Warm (HDD) → Cold (S3)

Trade-offs

Decision	Option A	Option B	Recommendation
Processing	Micro-batch	True streaming	**True streaming** - lower latency
Search	Elasticsearch	Loki	**Loki** - cheaper at scale
Architecture	Lambda	Kappa	**Kappa** - simpler if streaming suffices

Retention	Keep all	Sample	**Tiered** - detailed recent, sampled old

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	High-throughput log buffering with replay capability	Tech Guide
Elasticsearch	Log indexing and ad-hoc search queries	Tech Guide
Flink/Spark	Stream and batch processing for aggregations and alerting	Tech Guide

How to Explain It Well

> **Opening**: "Real-time log processing requires handling millions of events per second with sub-second alerting. I'll use Kafka + Flink for stream processing."

> **Key talking points**:
>
> - "Kafka provides replay capability - reprocess if processing bug found"
> - "Flink for windowed aggregations: error rate per minute per service"
> - "CEP patterns detect multi-event sequences like brute force attacks"
> - "Lambda architecture: fast approximate + slow accurate views"
> - "Tiered retention: 7 days detailed, 90 days aggregated, years archived"

---

26. Handle CAP Theorem Trade-offs

Key Terms:

Term	Full Name	What it does
CAP	Consistency, Availability, Partition tolerance	Theorem: distributed systems can only guarantee 2 of 3
CP	Consistency + Partition tolerance	Sacrifice availability for data accuracy
AP	Availability + Partition tolerance	Sacrifice consistency for uptime

PACELC	Partition-Availability-Consistency Else Latency-Consistency	Extension of CAP for normal operation
Split-brain	Network partition scenario	System divides into isolated groups
Bounded Staleness	Consistency level	Data within N seconds/versions of latest

Problem Statement

Understand and apply CAP theorem when designing distributed systems, making conscious trade-offs between consistency, availability, and partition tolerance.

Clarifying Questions to Ask

1. **Data type**: What kind of data? (Financial, social, inventory?)
2. **Users**: How do users interact? (Read-heavy, write-heavy?)
3. **Network**: Single datacenter or multi-region?
4. **Failure tolerance**: Which failures must the system survive?
5. **Business impact**: Cost of inconsistency vs. unavailability?
6. **Recovery**: How to recover from split-brain scenarios?
7. **SLAs**: What availability/consistency SLAs are required?

Design Reasoning

> **Core philosophy**: No single consistency model fits all data — choose CP or AP per data type based on the business cost of inconsistency versus unavailability.

Decision	Choice	Why	Alternative Considered
Financial data	CP (strong consistency)	Incorrect account balances or double-charges are unacceptable; reject operations during partition	AP (risks overdraft or duplicate transactions)
Social media feeds	AP (eventual consistency)	A stale feed is better than no feed; users	CP (users see errors during network

		tolerate seconds-old data for availability	issues, poor experience)
Inventory/stock	CP with reservation pattern	Overselling is costly; reserve stock atomically, release on timeout	AP (risks overselling, requires costly compensation)
Session data	AP (high availability)	Users must stay logged in; session replication lag is invisible to users	CP (users randomly logged out during partitions)
Consistency tuning	Per-operation via Cosmos DB consistency levels	Strong for transfers, session for profiles, eventual for counters — one database, tunable per query	Separate databases per consistency need (operational overhead)
Partition recovery	Application-level merge with conflict resolution	Domain-specific merges (union for carts, LWW for profiles) preserve intent better than generic strategies	Automatic LWW everywhere (silently loses data for additive operations)

API Design

Consistency Configuration APIs

Method	Endpoint	Description
`GET`	`/api/v1/config/consistency`	Get current consistency settings
`PUT`	`/api/v1/config/consistency`	Update consistency level
`GET`	`/api/v1/partitions/status`	Check partition health

Data Access APIs (consistency-aware)

```
// AP System (Available + Partition-tolerant)
GET /api/v1/data/{key}
Headers:
  X-Consistency: eventual
  X-Max-Staleness: 5s

// CP System (Consistent + Partition-tolerant)
GET /api/v1/data/{key}
Headers:
```

```
    X-Consistency: strong
    X-Read-Quorum: majority
```

Response Headers (consistency metadata)

```
X-Data-Version: 42
X-Last-Modified: 2025-03-15T10:00:00Z
X-Source-Region: us-east
X-Replication-Lag-Ms: 150
```

CAP Theorem Explained

```
┌─────────────────────────────────────────────────────────────────┐
│                      CAP THEOREM                                │
│                                                                 │
│  In a distributed system, you can only guarantee 2 of 3:       │
│                                                                 │
│                      CONSISTENCY                                │
│                          /\                                     │
│                         /  \                                    │
│                        /    \                                   │
│                       / CP   \                                  │
│                      /        \                                 │
│                     /    CA    \                                │
│    AVAILABILITY ──────────────── PARTITION                      │
│                     \    AP    /    TOLERANCE                   │
│                      \        /                                 │
│                       \      /                                  │
│                        \    /                                   │
│                         \  /                                    │
│                          \/                                     │
│                                                                 │
│  CA: Not practical (network partitions happen)                  │
│  CP: Sacrifice availability for consistency                     │
│  AP: Sacrifice consistency for availability                     │
└─────────────────────────────────────────────────────────────────┘
```

Trade-off Scenarios

```
SCENARIO 1: BANKING SYSTEM (Choose CP)

┌─────────────────────────────────────────────────────────────────┐
│  Requirement: Account balance must be accurate                  │
│  Trade-off: Reject transactions during partition                │
│                                                                 │
│  During network partition:                                      │
│    - Transfer request arrives                                   │
│    - Cannot verify balance with other nodes                     │
```

```
│    - Return error: "Service temporarily unavailable"           │
│    - Better to fail than allow overdraft                       │
│                                                                 │
│  Technologies: PostgreSQL with synchronous replication         │
│                ZooKeeper, etcd                                  │
└─────────────────────────────────────────────────────────────────┘

SCENARIO 2: SOCIAL MEDIA FEED (Choose AP)
┌─────────────────────────────────────────────────────────────────┐
│  Requirement: Users can always view and post                   │
│  Trade-off: May see slightly stale feed                        │
│                                                                 │
│  During network partition:                                      │
│    - User posts update                                          │
│    - Stored locally, queued for replication                    │
│    - Other users may not see it immediately                    │
│    - Eventually consistent when partition heals                │
│                                                                 │
│  Technologies: Cassandra, DynamoDB, CouchDB                    │
└─────────────────────────────────────────────────────────────────┘

SCENARIO 3: SHOPPING CART (Choose AP with reconciliation)
┌─────────────────────────────────────────────────────────────────┐
│  Requirement: Users can always add items                       │
│  Trade-off: Merge carts when partition heals                   │
│                                                                 │
│  During partition:                                              │
│    - User adds "apple" on device A                             │
│    - User adds "banana" on device B                            │
│    - Both writes succeed locally                               │
│                                                                 │
│  After partition heals:                                         │
│    - Merge: cart = ["apple", "banana"]                         │
│    - No data lost, both actions preserved                      │
│                                                                 │
│  Conflict resolution: Union (for add), LWW (for remove)        │
└─────────────────────────────────────────────────────────────────┘
```

PACELC Extension

```
┌─────────────────────────────────────────────────────────────────┐
│  PACELC: Extends CAP for normal operation                      │
│                                                                 │
│  If Partition:                                                  │
│    Choose Availability or Consistency (CAP)                    │
│  Else (normal operation):                                       │
│    Choose Latency or Consistency                               │
│                                                                 │
│  Examples:                                                      │
│    PA/EL (DynamoDB): Available during partition, low latency   │
│    PC/EC (Traditional SQL): Consistent always, higher latency  │
```

```
│    PA/EC (Cosmos DB): Available in partition, consistent else  │
└────────────────────────────────────────────────────────────────┘
```

Consistency Levels (Cosmos DB Example)

```
┌──────────────────────────────────────────────────────────────┐
│  STRONG: Read sees latest write (highest latency)            │
│    ├── Use for: Financial transactions                       │
│    └── Cost: Higher latency, lower throughput                │
│                                                               │
│  BOUNDED STALENESS: Read within N seconds/versions of latest │
│    ├── Use for: Leaderboards, activity feeds                 │
│    └── Cost: Moderate latency                                 │
│                                                               │
│  SESSION: Read-your-own-writes within session                 │
│    ├── Use for: User profiles, shopping carts                │
│    └── Cost: Low latency                                      │
│                                                               │
│  EVENTUAL: No ordering guarantee (lowest latency)            │
│    ├── Use for: Like counts, view counts                     │
│    └── Cost: May see stale data                              │
└──────────────────────────────────────────────────────────────┘
```

Trade-offs

Decision	Choose CP	Choose AP
Banking	✓ (accuracy critical)	
Social media		✓ (availability > consistency)
Inventory	✓ (overselling bad)	Or AP with reservation
DNS		✓ (must resolve)
Configuration	✓ (all nodes same config)	
Session data		✓ (user always logged in)

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Cosmos DB	Tunable consistency with PACELC support	Tech Guide

PostgreSQL	CP system with synchronous replication	Tech Guide
Cassandra	AP system for high availability	Tech Guide

How to Explain It Well

> **Opening**: "CAP theorem states we can't have all three: consistency, availability, and partition tolerance. I choose based on business requirements."

> **Key talking points**:
>
> - "Network partitions are inevitable - CA isn't practical"
> - "Banking chooses CP: reject transactions vs. risk inconsistency"
> - "Social media chooses AP: stale feed is better than no feed"
> - "PACELC extends CAP: even without partition, latency vs. consistency trade-off exists"
> - "Cosmos DB lets you choose per-operation: strong for critical, eventual for counters"

---

27. Microservices vs Monolith

Key Terms:

Term	Full Name	What it does
Monolith	Single deployment unit	All code in one deployable application
Microservice	Independent service	Small, focused service with own database
Bounded Context	DDD term	Logical boundary around a domain area
Service Mesh	Infrastructure layer	Handles service-to-service communication
Saga Pattern	Distributed transactions	Sequence of local transactions with compensations
Strangler Fig	Migration pattern	Gradually replace monolith with services
Conway's Law	Organization principle	System design mirrors team structure

Problem Statement

Decide when to use microservices architecture versus a monolith, and how to evolve between them.

Clarifying Questions to Ask

1. **Team size**: How many developers? How organized?
2. **Domain complexity**: How many bounded contexts?
3. **Scale requirements**: Different components scale differently?
4. **Deployment frequency**: How often do you deploy?
5. **Technology diversity**: Need different languages/frameworks?
6. **Operational maturity**: DevOps capabilities?
7. **Current state**: Greenfield or existing system?

Design Reasoning

> **Core philosophy**: Start monolithic for speed, extract microservices only when specific pain points (team conflicts, divergent scaling needs) justify the operational complexity.

Decision	Choice	Why	Alternative Considered
Starting architecture	Monolith-first	Faster iteration when domain boundaries are unclear; avoids premature distributed complexity	Microservices from day one (high upfront cost, wrong boundaries are expensive to fix)
Migration strategy	Strangler Fig pattern	Incremental extraction reduces risk — route traffic to new service, keep old code as fallback	Big-bang rewrite (high risk, long timeline, often fails)
Service communication	Event-driven (Kafka) as primary, gRPC for sync	Async events decouple services and prevent cascading failures; gRPC for low-latency internal calls	REST everywhere (synchronous coupling, higher latency, cascading failures)
Data ownership	Database per service	Enforces bounded contexts; services can	Shared database (tight coupling,

		evolve schema independently without cross-team coordination	schema changes break multiple services)
Service orchestration	API Gateway + Service Mesh	Gateway handles external routing and auth; mesh handles internal traffic, retries, and circuit breaking	Direct service-to-service calls (no observability, no fault tolerance)
Transaction management	Saga pattern with compensations	Distributed transactions across services without 2PC; compensating actions undo partial failures	Two-phase commit (blocks resources, doesn't scale, single coordinator failure)
Deployment	Kubernetes (AKS) with independent pipelines	Per-service CI/CD enables team autonomy; container orchestration handles scaling and self-healing	Single deployment pipeline (blocks all teams, monolith deployment pattern)

API Design

API Gateway Endpoints (Microservices)

Method	Endpoint	Description
`ANY`	`/api/v1/{service}/**`	Route to appropriate microservice
`GET`	`/api/v1/registry/services`	List registered services
`GET`	`/api/v1/registry/services/{name}/health`	Service health check

Inter-Service Communication

Pattern	Protocol	Use Case
REST	HTTP/JSON	Synchronous request-response
gRPC	HTTP/2 + Protobuf	High-performance internal calls

Events	Kafka/Event Hubs	Async event-driven communication

Service Mesh APIs

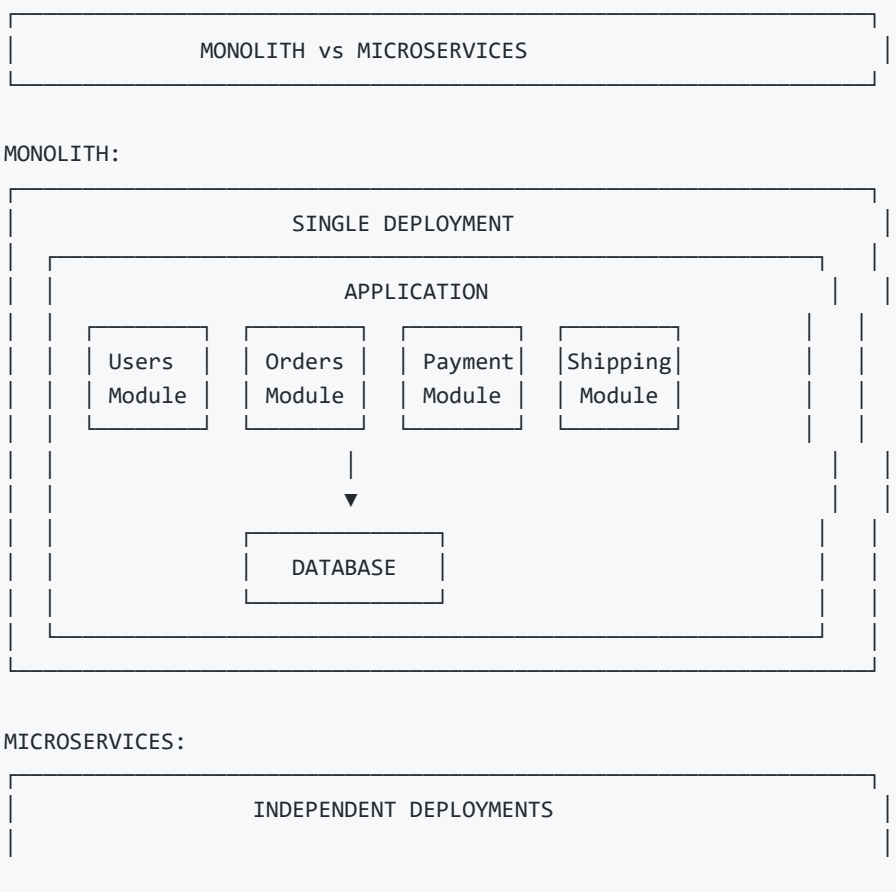
Method	Endpoint	Description
`GET`	`/api/v1/mesh/topology`	Get service dependency graph
`GET`	`/api/v1/mesh/traffic`	Get inter-service traffic metrics
`PUT`	`/api/v1/mesh/routing/{service}`	Configure traffic splitting (canary)

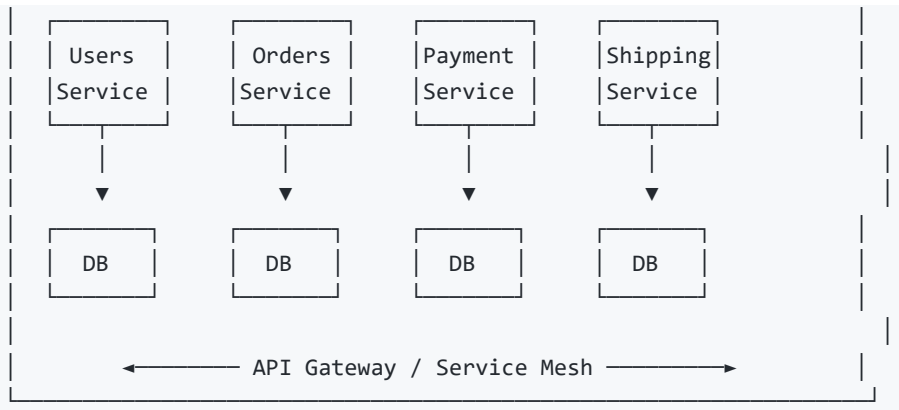
API Versioning Strategy

```
# URL versioning
/api/v1/users
/api/v2/users

# Header versioning
Accept: application/vnd.myapp.v2+json
```

Comparison





When to Choose What

```
CHOOSE MONOLITH WHEN:
┌─────────────────────────────────────────────────────────────┐
│  ✓ Small team (< 10 developers)                             │
│  ✓ New product with unclear requirements                    │
│  ✓ Simple domain                                            │
│  ✓ Limited DevOps experience                                │
│  ✓ Need to move fast and iterate                            │
│  ✓ Tight budget (no infrastructure overhead)                │
│                                                             │
│  "Start with a monolith, extract services when needed"      │
│                          - Martin Fowler                    │
└─────────────────────────────────────────────────────────────┘

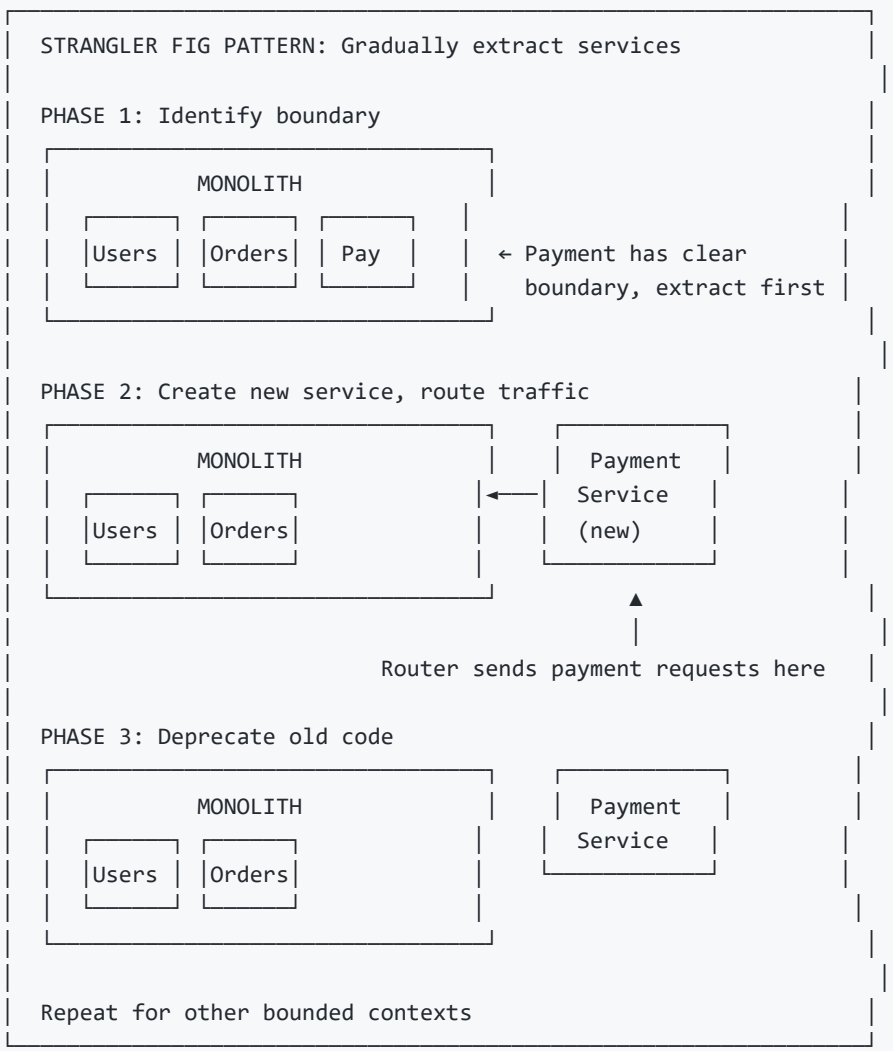
CHOOSE MICROSERVICES WHEN:
┌─────────────────────────────────────────────────────────────┐
│  ✓ Large team (multiple teams, 50+ developers)              │
│  ✓ Clear domain boundaries (bounded contexts)               │
│  ✓ Different scaling requirements per component             │
│  ✓ Need independent deployments                             │
│  ✓ Different technology requirements                        │
│  ✓ Strong DevOps/platform team                              │
│                                                             │
│  "You must be this tall to ride" - organizational maturity  │
└─────────────────────────────────────────────────────────────┘
```

Comparison Table

Aspect	Monolith	Microservices
Deployment	All-or-nothing	Independent
Scaling	Vertical or full horizontal	Per-service
Technology	Single stack	Polyglot

Data	Shared database	Database per service
Debugging	Simple (single process)	Complex (distributed tracing)
Latency	In-process calls	Network calls
Consistency	ACID transactions	Eventual consistency
Team autonomy	Low	High
Operational complexity	Low	High
Initial velocity	Fast	Slow (infrastructure setup)

Migration Strategy: Strangler Fig Pattern



Trade-offs

Decision	Monolith	Microservices	Recommendation
Startup	✓		**Monolith** - speed
Scale-up		✓	**Micro** - team autonomy
Complexity	Low	High	Consider team maturity
Data consistency	✓		**Monolith** if ACID critical

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
AKS/Kubernetes	Container orchestration for microservices	Tech Guide
Kafka	Event-driven communication between services	Tech Guide
Circuit Breaker	Fault tolerance in service communication	Tech Guide
Saga Pattern	Distributed transaction management	Tech Guide

How to Explain It Well

> **Opening**: "Microservices add complexity. I recommend starting monolithic and extracting services when you hit specific pain points."

> **Key talking points**:
>
> - "Monolith-first: faster to build, easier to refactor when domain unclear"
> - "Extract services when: team conflicts on deployment, different scaling needs"
> - "Strangler fig: incrementally extract, never big-bang rewrite"
> - "Microservices require: service discovery, distributed tracing, saga pattern"
> - "Conway's Law: architecture mirrors team structure"

---

28. Database Schema Evolution

Key Terms:

Term	Full Name	What it does
Migration	Schema change script	Versioned script that modifies database structure
Expand-Contract	Migration pattern	Add new → migrate data → switch → remove old
Backward Compatible	Safe change	Old app versions still work after change
gh-ost	GitHub Online Schema Transformation	Tool for zero-downtime MySQL migrations
Binlog	Binary log	MySQL log of all changes (used for replication)

Problem Statement

Design strategies for evolving database schemas over time without downtime or data loss.

Clarifying Questions to Ask

1. **Downtime**: Can you have maintenance windows?
2. **Scale**: How much data? How many tables?
3. **Frequency**: How often do schemas change?
4. **Rollback**: Need to roll back migrations?
5. **Testing**: How to test migrations safely?
6. **Multi-version**: Multiple app versions running?
7. **Replication**: Cross-region replication involved?

Design Reasoning

> **Core philosophy**: Zero-downtime schema evolution through backward-compatible, phased migrations that never break running application versions.

Decision	Choice	Why	Alternative Considered
Migration pattern	Expand-Contract	Safe rollback at every phase; old and new app versions coexist	Big-bang ALTER — risks downtime and table locks

Data backfill	Batched background jobs	Avoids long-held table locks on million-row tables	Synchronous UPDATE — locks table for entire duration
Online schema tool	gh-ost (binlog-based)	Zero-downtime for large MySQL tables via shadow-copy + binlog replay	pt-online-schema-change — trigger-based, more intrusive on writes
Schema tracking	Versioned migration table	Deterministic, auditable, exactly-once execution	Manual SQL scripts — error-prone, no rollback tracking
Change safety default	Additive-only changes	Backward compatible with all running app versions	Direct renames/drops — breaks old app versions mid-deploy
Primary database	PostgreSQL	Robust migration tooling (CREATE INDEX CONCURRENTLY, pg_repack)	MySQL — requires external tools like gh-ost for safe DDL

API Design

Migration Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/migrations`	Create and queue a migration
`GET`	`/api/v1/migrations`	List all migrations
`GET`	`/api/v1/migrations/{migrationId}/status`	Get migration status
`POST`	`/api/v1/migrations/{migrationId}/execute`	Execute a pending migration
`POST`	`/api/v1/migrations/{migrationId}/rollback`	Rollback a migration

Schema APIs

Method	Endpoint	Description
`GET`	`/api/v1/schemas/{table}/current`	Get current schema
`GET`	`/api/v1/schemas/{table}/history`	Get schema change history

`POST`	`/api/v1/schemas/{table}/validate`	Validate a proposed schema change

API Versioning for Schema Changes

```
// Support both old and new schema during migration
GET /api/v2/users    → { "full_name": "John Doe" }
GET /api/v1/users    → { "first_name": "John", "last_name": "Doe" }
```

Migration Strategies

```
1. EXPAND-CONTRACT PATTERN (Zero downtime)
┌─────────────────────────────────────────────────────────────┐
│  Rename column: username → user_name                        │
│                                                             │
│  PHASE 1: EXPAND (add new column)                           │
│  ┌─────────────────────────────────────────────────────┐    │
│  │  ALTER TABLE users ADD COLUMN user_name VARCHAR(255);   │    │
│  │  -- App writes to BOTH columns                      │    │
│  │  -- App reads from username (old)                   │    │
│  └─────────────────────────────────────────────────────┘    │
│                                                             │
│  PHASE 2: MIGRATE (backfill data)                           │
│  ┌─────────────────────────────────────────────────────┐    │
│  │  UPDATE users SET user_name = username              │    │
│  │  WHERE user_name IS NULL;                           │    │
│  │  -- Batch to avoid locking                          │    │
│  └─────────────────────────────────────────────────────┘    │
│                                                             │
│  PHASE 3: TRANSITION (switch reads)                         │
│  ┌─────────────────────────────────────────────────────┐    │
│  │  -- Deploy new app version                          │    │
│  │  -- App reads from user_name (new)                  │    │
│  │  -- App still writes to BOTH (for rollback)         │    │
│  └─────────────────────────────────────────────────────┘    │
│                                                             │
│  PHASE 4: CONTRACT (remove old column)                      │
│  ┌─────────────────────────────────────────────────────┐    │
│  │  ALTER TABLE users DROP COLUMN username;            │    │
│  │  -- Only after all app versions updated             │    │
│  └─────────────────────────────────────────────────────┘    │
└─────────────────────────────────────────────────────────────┘

2. ADDITIVE CHANGES ONLY
┌─────────────────────────────────────────────────────────────┐
│  Safe (backward compatible):                                │
│    ✓ Add nullable column                                    │
│    ✓ Add table                                              │
│    ✓ Add index (CREATE INDEX CONCURRENTLY)                  │
│    ✓ Widen column (VARCHAR(50) → VARCHAR(100))              │
```

```
│                                                                 │
│  Unsafe (requires expand-contract):                             │
│    ✗ Remove column                                              │
│    ✗ Rename column                                              │
│    ✗ Change column type                                         │
│    ✗ Add NOT NULL constraint                                    │
└─────────────────────────────────────────────────────────────────┘
```

Migration Tools

```
MIGRATION FILE STRUCTURE:

┌─────────────────────────────────────────────────────────────────┐
│  migrations/                                                    │
│  ├── 001_create_users_table.sql                                 │
│  ├── 002_add_email_to_users.sql                                 │
│  ├── 003_create_orders_table.sql                                │
│  └── 004_add_user_name_column.sql                               │
│                                                                 │
│  Each migration:                                                │
│    - Has unique version number                                  │
│    - Runs exactly once                                          │
│    - Tracked in migrations table                                │
└─────────────────────────────────────────────────────────────────┘

MIGRATION TRACKING TABLE:

┌─────────────────────────────────────────────────────────────────┐
│  CREATE TABLE schema_migrations (                               │
│    version VARCHAR(255) PRIMARY KEY,                            │
│    applied_at TIMESTAMP DEFAULT NOW()                           │
│  );                                                             │
│                                                                 │
│  -- Check what's applied                                        │
│  SELECT * FROM schema_migrations;                               │
│  | version | applied_at          |                             │
│  |---------|---------------------|                             │
│  | 001     | 2024-01-01 10:00:00 |                             │
│  | 002     | 2024-01-05 14:30:00 |                             │
│  | 003     | 2024-01-10 09:15:00 |                             │
└─────────────────────────────────────────────────────────────────┘
```

Large Data Migrations

```
BATCHED MIGRATION (Avoid locking):

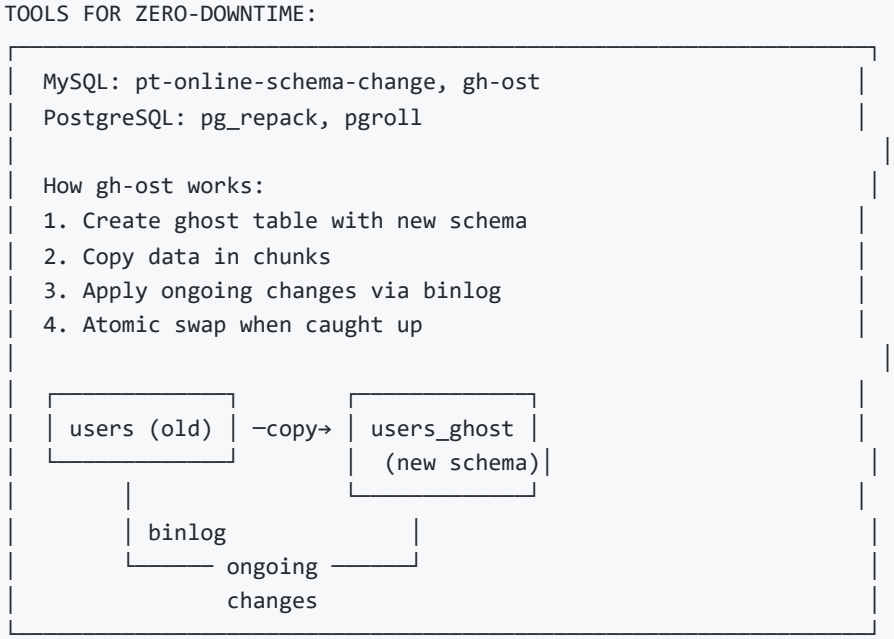
┌─────────────────────────────────────────────────────────────────┐
│  -- DON'T: Update all at once (locks table)                     │
│  UPDATE users SET new_col = old_col;                            │
│                                                                 │
│  -- DO: Batch with limits                                       │
│  DO $$                                                          │
│  DECLARE                                                        │
```

```
    batch_size INT := 1000;
    affected INT;
  BEGIN
    LOOP
      UPDATE users
      SET new_col = old_col
      WHERE new_col IS NULL
      AND id IN (
        SELECT id FROM users
        WHERE new_col IS NULL
        LIMIT batch_size
        FOR UPDATE SKIP LOCKED
      );

      GET DIAGNOSTICS affected = ROW_COUNT;
      EXIT WHEN affected = 0;

      PERFORM pg_sleep(0.1);  -- Breathing room
    END LOOP;
  END $$;
```

Online Schema Changes (Large Tables)



Trade-offs

Decision	Option A	Option B	Recommendation

Downtime	Maintenance window	Zero-downtime	**Zero** for production
Approach	Direct ALTER	Expand-contract	**Expand-contract** - safer
Data migration	Synchronous	Background job	**Background** for large tables
Rollback	Support	No support	**Support** - always have undo

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
PostgreSQL	Primary database for schema migrations	Tech Guide

How to Explain It Well

> **Opening**: "Schema evolution must be backward-compatible during deployment. I use expand-contract for breaking changes."

> **Key talking points**:
>
> - "Expand-contract: add new, migrate, switch, remove old"
> - "Only additive changes are safe: new columns, tables, indexes"
> - "Batch large data migrations to avoid table locks"
> - "Tools like gh-ost for online schema changes on large tables"
> - "Track migrations in database, never rely on manual execution"

---

37. Design Distributed ID Generator

Key Terms:

Term	Full Name	What it does

Snowflake ID	Twitter's ID format	64-bit: timestamp + node ID + sequence
UUID	Universally Unique Identifier	128-bit random/time-based ID
UUID v7	Time-sortable UUID	New standard with timestamp prefix
Epoch	Reference timestamp	Starting point for timestamp bits
NTP	Network Time Protocol	Clock synchronization across servers
Clock Drift	Time desynchronization	Clocks getting out of sync

Problem Statement

Design a system that generates globally unique IDs at scale with properties like sortability, no coordination, and high availability.

Clarifying Questions to Ask

1. **Scale**: How many IDs per second? (100K+ per second?)
2. **Properties**: Unique, sortable, compact?
3. **Length**: Max ID length acceptable? (64-bit?)
4. **Coordination**: Can nodes coordinate, or must be independent?
5. **Ordering**: Strictly ordered or roughly time-ordered?
6. **Format**: Numeric, alphanumeric, or UUID?
7. **Failure modes**: What if clock drifts or node fails?

Design Reasoning

> **Core philosophy**: Generate globally unique, time-sortable IDs at massive scale without inter-node coordination, using bit-packing to encode timestamp, origin, and sequence into a compact 64-bit integer.

Decision	Choice	Why	Alternative Considered
ID format	Snowflake (64-bit)	Compact, sortable, embeds timestamp for free ordering	UUID v4 — 128-bit, not sortable, poor index locality
Sortability	Timestamp as high-order bits	IDs naturally sort by creation time, enabling range queries on time	UUID v7 — sortable but 128-bit doubles storage and index size

Coordination	None (embedded node ID)	Each node generates independently — zero network hops, zero contention	Central ticket server — single point of failure, network bottleneck
Node ID assignment	ZooKeeper / etcd / static config	10-bit node ID supports 1024 nodes; static config is simplest for fixed fleets	Auto-discovery — adds complexity and risk of duplicate node IDs
Clock drift handling	Detect and wait	Waiting preserves monotonicity; throwing errors risks availability	Ignore drift — could produce duplicate or out-of-order IDs
Sequence overflow	Spin-wait for next millisecond	Guarantees uniqueness; 4096 IDs/ms/node rarely overflows in practice	Drop request — unacceptable for callers expecting an ID
Generation location	Client-side (in-process)	Eliminates network round-trip; sub-microsecond latency	Server-side service — adds latency and a dependency

High-Level Design



```
│  Pros: No coordination, truly unique                                  │
│  Cons: Not sortable, 128 bits (large), poor index locality    │
└──────────────────────────────────────────────────────────────────────┘

APPROACH 3: UUID v7 (Time-based, new standard)
┌──────────────────────────────────────────────────────────────────────┐
│  128-bit with timestamp prefix: sortable + random                    │
│                                                                       │
│  48 bits: Unix timestamp (milliseconds)                              │
│  4 bits:  Version (7)                                                │
│  12 bits: Random                                                     │
│  62 bits: Random                                                     │
│                                                                       │
│  Pros: Sortable, no coordination, standard format                    │
│  Cons: 128 bits                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

API Design

ID Generation APIs

Method	Endpoint	Description
`POST`	`/api/v1/ids/generate`	Generate a single unique ID
`POST`	`/api/v1/ids/batch?count={n}`	Generate a batch of IDs
`GET`	`/api/v1/ids/{id}/decode`	Decode ID to extract timestamp, worker, sequence

Generate ID Response

```
POST /api/v1/ids/generate

Response: 200 OK
{
  "id": "175942649856712704",
  "id_hex": "0x02710000000001",
  "decoded": {
    "timestamp": 1672531200000,
    "datacenter_id": 1,
    "worker_id": 5,
    "sequence": 0
  }
}
```

Batch Generation

```
POST /api/v1/ids/batch?count=100

Response: 200 OK
{
  "ids": ["175942649856712704", "175942649856712705", ...],
  "count": 100,
  "generated_in_ms": 1
}
```

Worker Management APIs (Internal)

Method	Endpoint	Description
`POST`	`/api/v1/workers/register`	Register a worker node
`GET`	`/api/v1/workers`	List active workers and their IDs
`DELETE`	`/api/v1/workers/{workerId}`	Deregister a worker

Snowflake Implementation

```
public class SnowflakeIdGenerator {
    private static final long EPOCH = 1609459200000L; // 2021-01-01
    private static final long NODE_BITS = 10;
    private static final long SEQUENCE_BITS = 12;

    private static final long MAX_NODE_ID = (1L << NODE_BITS) - 1;
    private static final long MAX_SEQUENCE = (1L << SEQUENCE_BITS) - 1;

    private final long nodeId;
    private long lastTimestamp = -1L;
    private long sequence = 0L;

    public SnowflakeIdGenerator(long nodeId) {
        if (nodeId > MAX_NODE_ID) throw new IllegalArgumentException();
        this.nodeId = nodeId;
    }

    public synchronized long nextId() {
        long timestamp = System.currentTimeMillis();

        if (timestamp < lastTimestamp) {
            // Clock went backwards - wait or throw
            throw new RuntimeException("Clock moved backwards");
        }
```

```
            if (timestamp == lastTimestamp) {
                sequence = (sequence + 1) & MAX_SEQUENCE;
                if (sequence == 0) {
                    // Sequence exhausted, wait for next millisecond
                    timestamp = waitNextMillis(lastTimestamp);
                }
            } else {
                sequence = 0;
            }

            lastTimestamp = timestamp;

            return ((timestamp - EPOCH) << (NODE_BITS + SEQUENCE_BITS))
                 | (nodeId << SEQUENCE_BITS)
                 | sequence;
        }
    }
```

Key Components

Component	Purpose	Technology
ID Service	Generate IDs	Stateless service
Node Registry	Assign node IDs	ZooKeeper / etcd
Clock Sync	Keep clocks aligned	NTP
Monitoring	Detect clock drift	Alerting

Scalability Considerations

- **No coordination**: Each node generates independently
- **Horizontal scaling**: Add nodes, assign unique node IDs
- **Clock sync**: Use NTP, monitor for drift > 1 second
- **Client-side generation**: Generate in application, no network hop
- **Batch pre-generation**: Pre-generate IDs for burst handling

Trade-offs

Decision	Option A	Option B	Recommendation
Format	UUID	Snowflake	**Snowflake** - smaller, sortable
Coordination	ZooKeeper	Config	**Config** - simpler for fixed nodes

Clock handling	Wait	Error	**Wait** - more available
Generation	Server	Client	**Client** - lower latency

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Raft Consensus	Node ID assignment via ZooKeeper/etcd	Tech Guide

---

How to Explain It Well

> **Opening**: "Distributed IDs need uniqueness without coordination. I'll use Snowflake: 64-bit IDs with timestamp, node ID, and sequence."

> **Key talking points**:
>
> - "Snowflake: 41 bits time + 10 bits node + 12 bits sequence"
> - "4096 IDs per millisecond per node, 1024 nodes = billions/sec"
> - "Time-ordered: IDs roughly sort by creation time"
> - "No coordination: each node generates independently"
> - "Clock drift: detect and wait, never go backwards"

---

38. Design Distributed Cache

Key Terms:

Term	Full Name	What it does
LRU	Least Recently Used	Eviction policy removing oldest accessed item
LFU	Least Frequently Used	Eviction policy removing least accessed item
TTL	Time To Live	Auto-expiration after set duration

Consistent Hashing	Distribution algorithm	Minimizes data movement when scaling
Virtual Node	Hash ring technique	Multiple ring positions per physical node
QPS	Queries Per Second	Throughput measure for cache
AOF	Append-Only File	Redis persistence by logging writes
RDB	Redis Database	Redis point-in-time snapshot persistence

Problem Statement

Design a distributed caching system like Redis or Memcached that provides low-latency data access across multiple nodes.

Clarifying Questions to Ask

1. **Scale**: Data size? QPS? (1TB, 1M QPS?)
2. **Consistency**: Strong or eventual?
3. **Eviction**: LRU, LFU, TTL?
4. **Persistence**: In-memory only or persist to disk?
5. **Data types**: Key-value only or complex types (lists, sets)?
6. **Replication**: Primary-replica or peer-to-peer?
7. **Partition tolerance**: How to handle network splits?

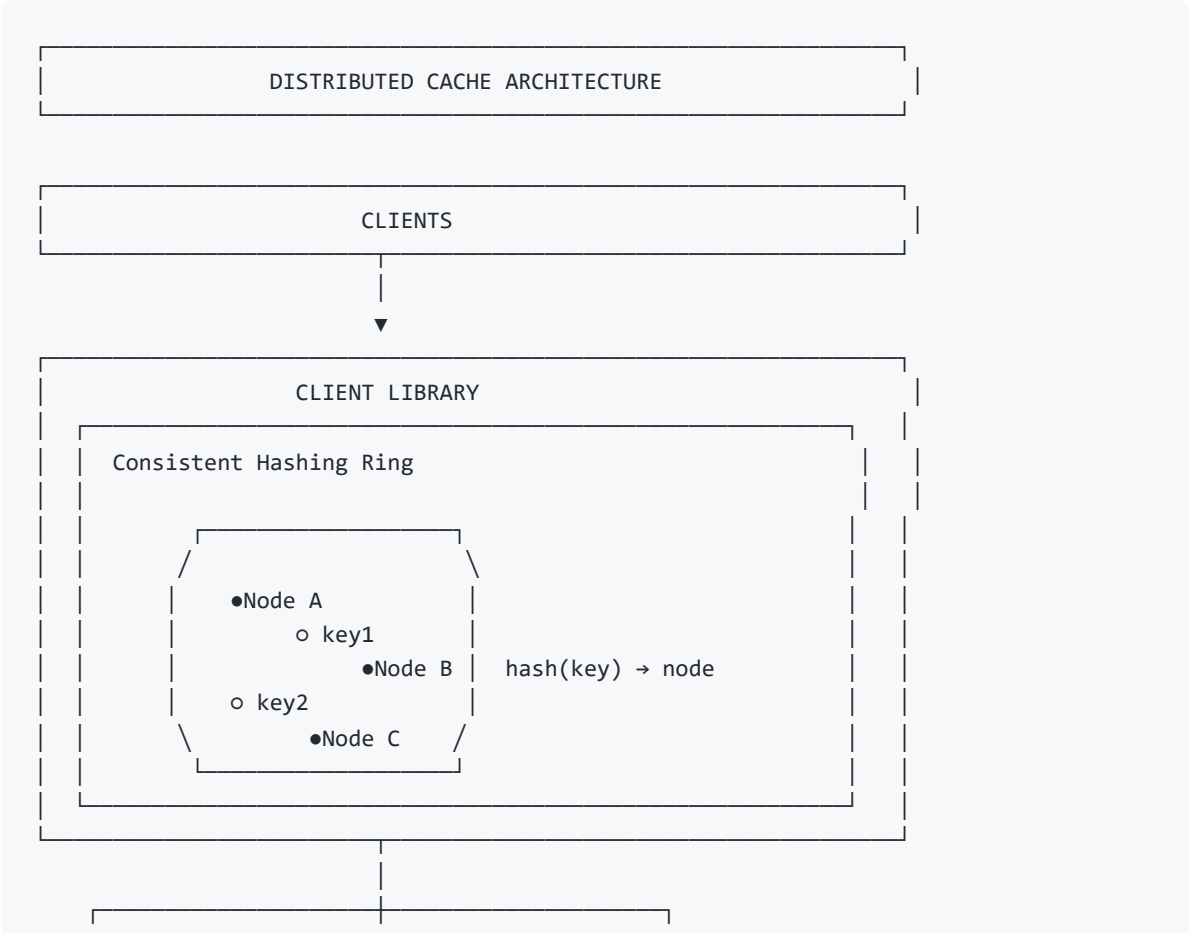
Design Reasoning

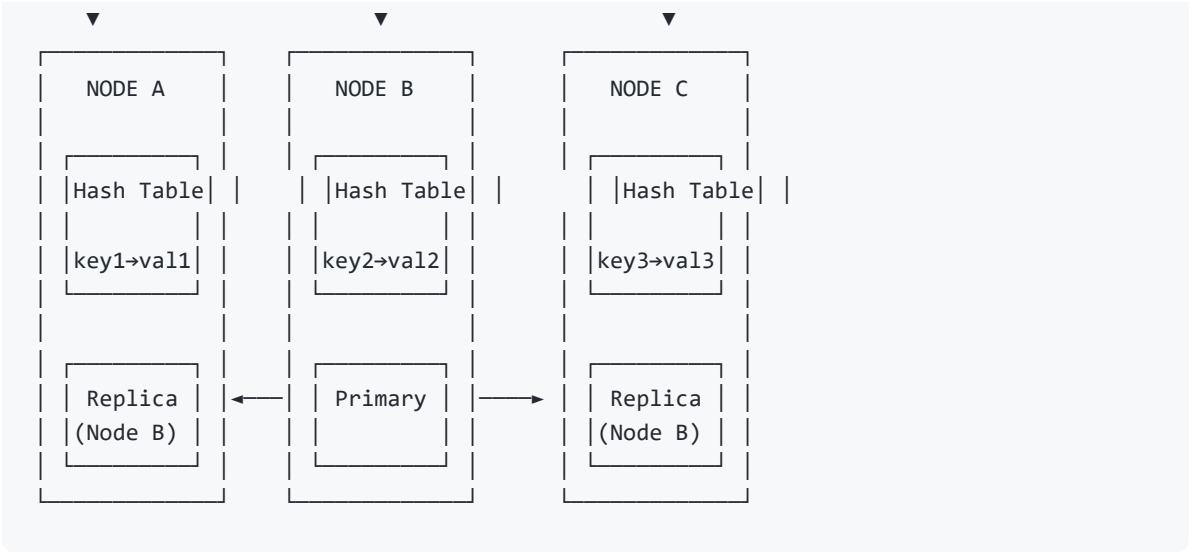
> **Core philosophy**: Minimize data movement during scaling and maximize throughput by distributing keys via consistent hashing, keeping routing logic in a smart client library to avoid proxy overhead.

Decision	Choice	Why	Alternative Considered
Key distribution	Consistent hashing with virtual nodes	Adding/removing a node moves only 1/N of keys; virtual nodes ensure even load	Modulo hashing — adding a node remaps nearly all keys, causing cache stampede
Routing	Client library (no proxy)	Eliminates an extra network hop; client directly connects to correct node	Proxy-based routing — adds latency and a single point of failure

Eviction policy	LRU	O(1) access and eviction with doubly-linked list + hash map; matches most workloads	LFU — better for stable hot-sets but more complex and scan-resistant workloads are rare
Consistency	Eventual	Caches tolerate staleness; strong consistency would add write latency that defeats the purpose	Strong consistency — requires synchronous replication, negating cache speed benefits
Replication	Async primary-replica	Low write latency; replica handles failover without blocking the write path	Synchronous replication — higher durability but doubles write latency
Persistence	Optional (AOF/RDB)	Pure cache needs no persistence; warm restart benefits from RDB snapshots	Always persist — unnecessary write overhead for ephemeral cache data

High-Level Design





API Design

Cache Operations APIs

Method	Endpoint	Description
`GET`	`/api/v1/cache/{key}`	Get cached value
`PUT`	`/api/v1/cache/{key}`	Set cache entry with TTL
`DELETE`	`/api/v1/cache/{key}`	Delete cache entry
`POST`	`/api/v1/cache/batch/get`	Batch get multiple keys
`POST`	`/api/v1/cache/batch/set`	Batch set multiple keys

Cache Set Request

```
PUT /api/v1/cache/user:123:profile
{
  "value": { "name": "John", "email": "john@example.com" },
  "ttl_seconds": 3600,
  "policy": "write-through"   // write-through, write-behind, write-around
}
```

Cache Management APIs

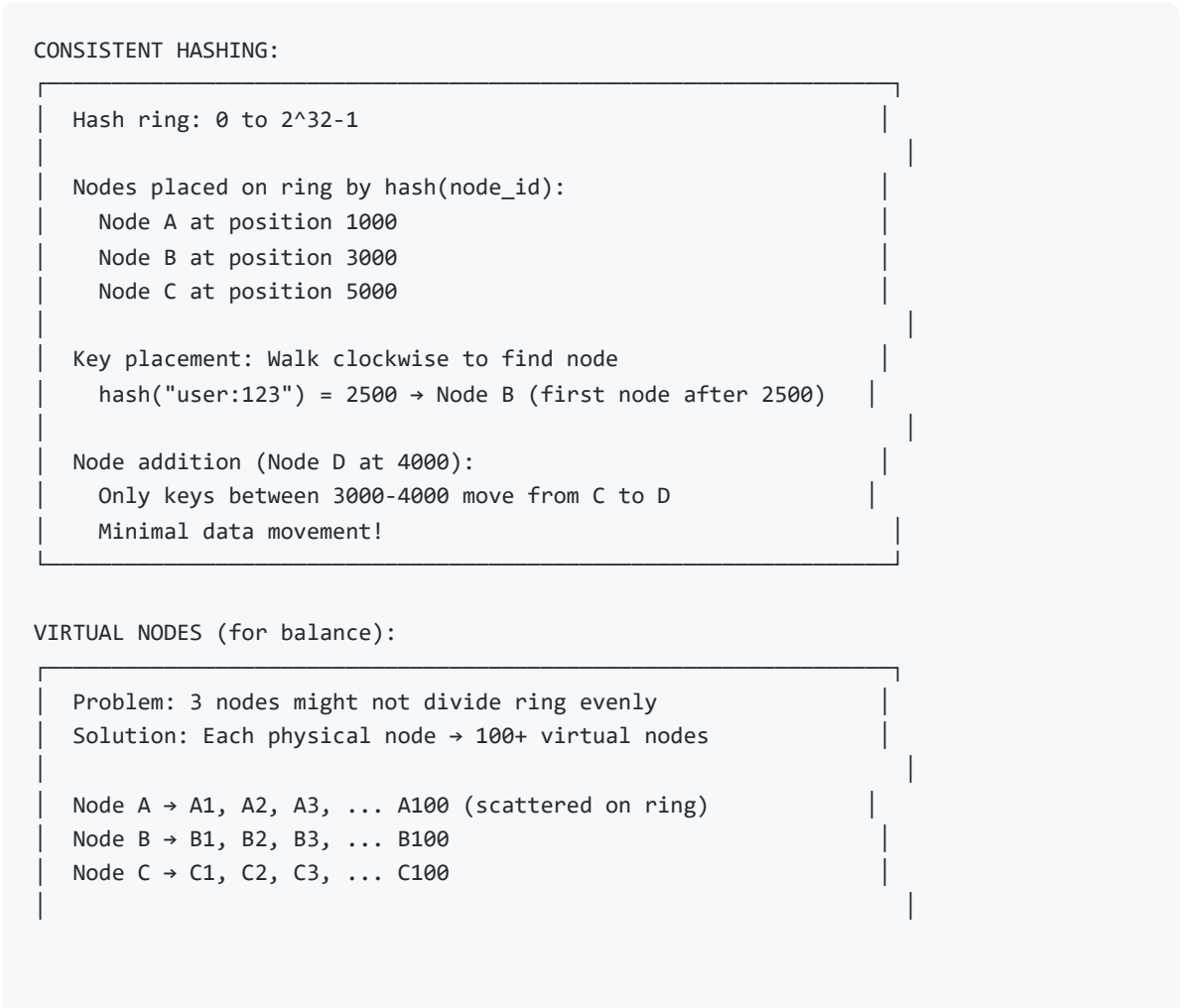
Method	Endpoint	Description
`POST`	`/api/v1/cache/invalidate`	Invalidate by pattern (e.g., `user:123:*`)

`GET`	`/api/v1/cache/stats`	Get hit rate, memory usage, eviction count
`GET`	`/api/v1/cache/nodes`	List cache nodes and distribution
`POST`	`/api/v1/cache/flush`	Flush entire cache (admin only)

Key Components

Component	Purpose	Technology
Client Library	Routing, connection pooling	Custom SDK
Cache Node	Store data in memory	Custom or Redis
Consistent Hashing	Distribute keys	Client-side
Replication	Data redundancy	Async replication
Cluster Manager	Node membership	Gossip protocol

Consistent Hashing



```
│  Result: Even distribution regardless of node count            │
└─────────────────────────────────────────────────────────────────┘
```

Eviction Policies

```
LRU (Least Recently Used):
┌─────────────────────────────────────────────────────────────────┐
│  Doubly-linked list + hash map                                  │
│                                                                 │
│  Access: Move to head (O(1))                                    │
│  Evict: Remove from tail (O(1))                                 │
│                                                                 │
│  HEAD ↔ [recent] ↔ [older] ↔ [oldest] ↔ TAIL                    │
└─────────────────────────────────────────────────────────────────┘

LFU (Least Frequently Used):
┌─────────────────────────────────────────────────────────────────┐
│  Track access count per key                                     │
│  Evict key with lowest count                                    │
│                                                                 │
│  Better for: Workloads with stable hot set                      │
│  Worse for: Scan patterns (one-time access pollutes cache)      │
└─────────────────────────────────────────────────────────────────┘

TTL (Time To Live):
┌─────────────────────────────────────────────────────────────────┐
│  Each key has expiration timestamp                              │
│  Background thread cleans expired keys                          │
│  Lazy expiration: Check on access                               │
└─────────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Horizontal scaling**: Add nodes, consistent hashing redistributes keys
- **Hot keys**: Replicate hot keys to multiple nodes
- **Memory limits**: Set max memory, configure eviction policy
- **Connection pooling**: Reuse connections in client library
- **Pipelining**: Batch multiple commands in single round trip

Trade-offs

Decision	Option A	Option B	Recommendation
Consistency	Strong	Eventual	**Eventual** - for cache
Eviction	LRU	LFU	**LRU** - simpler, good enough

Replication	Sync	Async	**Async** - lower latency
Persistence	None	AOF/RDB	**Optional** - depends on use case

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	In-memory cache with persistence options	Tech Guide
Cosmos DB	Session store with global distribution	Tech Guide
Consistent Hashing	Key distribution across cache nodes	Tech Guide

---

How to Explain It Well

> **Opening**: "Distributed cache needs consistent hashing for even distribution and minimal data movement when scaling."

> **Key talking points**:
>
> - "Consistent hashing: add node = move only 1/N of keys"
> - "Virtual nodes ensure even distribution across physical nodes"
> - "LRU eviction with doubly-linked list for O(1) operations"
> - "Async replication to replicas for fault tolerance"
> - "Client library handles routing - no proxy needed"

---

39. Design Distributed Job Scheduler

Key Terms:

Term	Full Name	What it does
Cron	Time-based scheduler	Unix schedule format (e.g., `0 0 * * *` = daily midnight)
DLQ	Dead Letter Queue	Queue for jobs that failed all retries

Idempotency	Same result on retry	Operation safe to run multiple times
Leader Election	Single active instance	Only one scheduler triggers jobs
Optimistic Locking	Concurrency control	UPDATE WHERE version=N prevents conflicts
Backoff	Retry delay	Increasing wait between retry attempts

Problem Statement

Design a system that schedules and executes jobs across a cluster of workers with support for cron schedules, one-time jobs, and retries.

Clarifying Questions to Ask

1. **Scale**: How many jobs? Execution frequency?
2. **Job types**: Scheduled, one-time, recurring?
3. **Duration**: Short tasks or long-running jobs?
4. **Reliability**: At-least-once or exactly-once execution?
5. **Priority**: Support different priority levels?
6. **Dependencies**: Jobs depend on other jobs?
7. **Visibility**: Track job status and history?

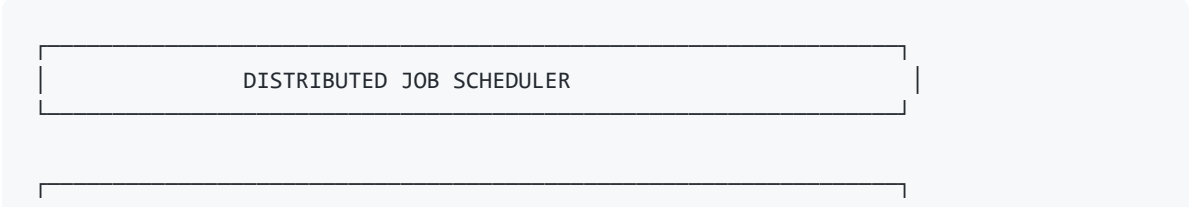
Design Reasoning

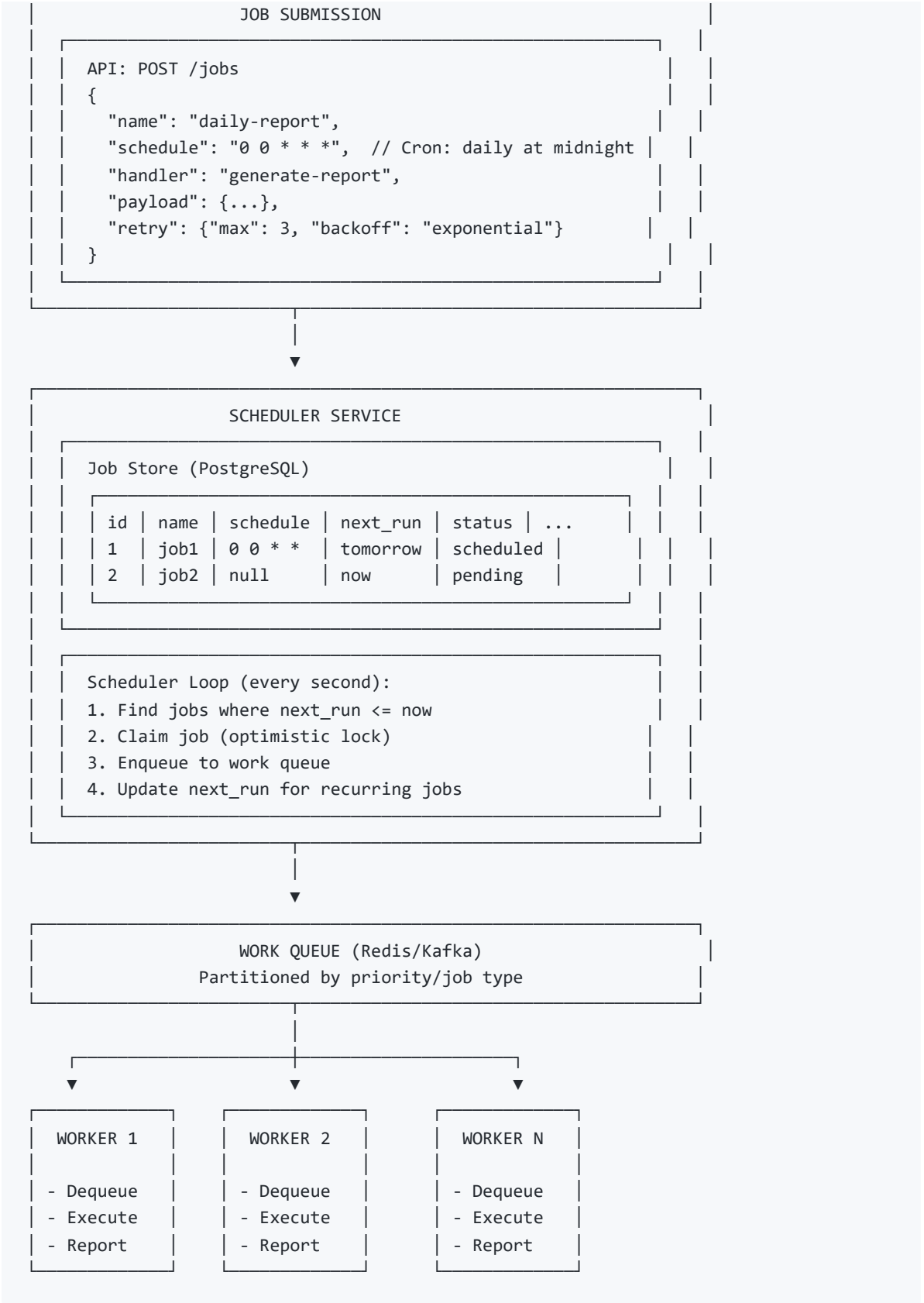
> **Core philosophy**: Decouple job triggering from job execution using a durable job store with optimistic locking for exactly-once triggering, and a work queue that lets stateless workers scale independently.

Decision	Choice	Why	Alternative Considered
Job store	PostgreSQL	ACID transactions for reliable scheduling; optimistic locking with `UPDATE WHERE version=N` prevents double-trigger	Redis — fast but lacks durable transactions for critical schedule state

Trigger mechanism	Polling with optimistic lock	Simple to implement; `UPDATE WHERE status='pending'` guarantees only one scheduler claims a job	Event-driven (cron daemon) — harder to make highly available across nodes
Work queue	Redis (Kafka at scale)	Redis lists are simple and low-latency for most workloads; Kafka adds durability for millions of jobs	Direct execution — tightly couples scheduler to workers, no back-pressure
Execution guarantee	At-least-once + idempotency keys	At-least-once is achievable without coordination; idempotency keys (`job_id + scheduled_time`) prevent duplicate side effects	Exactly-once — requires two-phase commit, too expensive for job scheduling
Scheduler HA	Leader election (ZooKeeper/etcd)	Single active scheduler avoids duplicate triggers; standby takes over on failure	Sharded schedulers — better for >1M jobs but adds partition-management complexity
Failure handling	Exponential backoff + DLQ	Backoff avoids overwhelming failing downstream services; DLQ preserves failed jobs for manual review	Immediate retry — can cause cascading failures; discard — loses work
Worker scaling	Auto-scale on queue depth	Stateless workers scale horizontally; queue depth is a direct measure of demand	Fixed pool — wastes resources during idle periods, under-provisions during spikes

High-Level Design





API Design

Job APIs

Method	Endpoint	Description
`POST`	`/api/v1/jobs`	Create/schedule a job
`GET`	`/api/v1/jobs/{jobId}`	Get job details
`PUT`	`/api/v1/jobs/{jobId}`	Update job configuration
`DELETE`	`/api/v1/jobs/{jobId}`	Delete a job
`POST`	`/api/v1/jobs/{jobId}/trigger`	Manually trigger a job
`POST`	`/api/v1/jobs/{jobId}/pause`	Pause a scheduled job
`POST`	`/api/v1/jobs/{jobId}/resume`	Resume a paused job

Create Job Request

```
POST /api/v1/jobs
{
  "name": "daily-report-generator",
  "type": "recurring",
  "schedule": "0 2 * * *",        // cron expression
  "handler": "report-service",
  "payload": { "report_type": "daily_summary" },
  "retry_policy": {
    "max_retries": 3,
    "backoff": "exponential"
  },
  "timeout_seconds": 300,
  "priority": "high"
}
```

Execution APIs

Method	Endpoint	Description
`GET`	`/api/v1/jobs/{jobId}/executions`	List past executions
`GET`	`/api/v1/executions/{execId}`	Get execution details
`GET`	`/api/v1/executions/{execId}/logs`	Get execution logs
`POST`	`/api/v1/executions/{execId}/cancel`	Cancel running execution

Key Components

Component	Purpose	Technology
API Service	Job CRUD operations	REST API
Job Store	Persist job definitions	PostgreSQL
Scheduler	Trigger due jobs	Singleton or leader election
Work Queue	Buffer jobs for workers	Redis or Kafka
Workers	Execute jobs	Stateless pods
Dead Letter Queue	Failed jobs for review	Separate queue

Exactly-Once Execution

```
CHALLENGE: Multiple schedulers might trigger same job

SOLUTION 1: Leader Election
┌─────────────────────────────────────────────────────────────┐
│  Only one scheduler instance is active (leader)             │
│  Others are standby, take over if leader fails              │
│  Use: ZooKeeper, etcd, or Redis SETNX                       │
└─────────────────────────────────────────────────────────────┘

SOLUTION 2: Optimistic Locking
┌─────────────────────────────────────────────────────────────┐
│  UPDATE jobs                                                │
│  SET status = 'claimed', claimed_by = 'scheduler-1'         │
│  WHERE id = 123                                             │
│    AND status = 'pending'                                   │
│    AND next_run <= NOW();                                   │
│                                                             │
│  Only one scheduler wins the UPDATE                         │
└─────────────────────────────────────────────────────────────┘

SOLUTION 3: Idempotency Keys
┌─────────────────────────────────────────────────────────────┐
│  Each job execution has unique key: job_id + scheduled_time │
│  Workers check: "Have I already processed this execution?"  │
│  Dedup store (Redis): SETNX execution:job1:2024-01-15T00:00 │
└─────────────────────────────────────────────────────────────┘
```

Retry and Failure Handling

```
RETRY STRATEGY:
┌─────────────────────────────────────────────────────────────┐
│  {                                                          │
│    "max_retries": 3,                                        │
```

```
│    "backoff": "exponential",                                    │
│    "initial_delay": "1s",                                       │
│    "max_delay": "1h"                                            │
│  }                                                               │
│                                                                  │
│  Attempt 1: Immediate                                           │
│  Attempt 2: 1 second later                                      │
│  Attempt 3: 2 seconds later                                     │
│  Attempt 4: 4 seconds later (if more retries)                   │
│                                                                  │
│  After max retries: Move to Dead Letter Queue                   │
└──────────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Scheduler HA**: Leader election or sharded by job ranges
- **Worker scaling**: Auto-scale based on queue depth
- **Partitioning**: Separate queues for different job types
- **Time zones**: Store all times in UTC
- **Clock sync**: NTP on all nodes

Trade-offs

Decision	Option A	Option B	Recommendation
Trigger	Polling	Event-driven	**Polling** - simpler
Execution	At-least-once	Exactly-once	**At-least-once + idempotency**
Scheduler	Single leader	Sharded	**Leader** - simpler for <1M jobs
Queue	Redis	Kafka	**Redis** - simpler; Kafka for huge scale

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	Work queue for job distribution at scale	Tech Guide
Redis	Work queue and idempotency key storage	Tech Guide
PostgreSQL	Job store and scheduling metadata	Tech Guide

How to Explain It Well

> **Opening**: "Job scheduler needs reliable triggering and at-least-once execution. I'll use optimistic locking for exactly-once triggering."

> **Key talking points**:
>
> - "Optimistic lock: UPDATE WHERE status=pending ensures one scheduler wins"
> - "Work queue decouples scheduling from execution"
> - "Exponential backoff with max retries for transient failures"
> - "Dead letter queue for manual review of persistent failures"
> - "Workers are stateless, can scale horizontally"

---

40. Design Key-Value Store

Key Terms:

Term	Full Name	What it does
LSM Tree	Log-Structured Merge Tree	Write-optimized storage with sequential writes
MemTable	In-memory table	Sorted buffer for recent writes
SSTable	Sorted String Table	Immutable on-disk sorted file
WAL	Write-Ahead Log	Durability log before memory write
Compaction	Merge process	Combines SSTables, removes tombstones
Tombstone	Delete marker	Marks deleted keys (removed during compaction)
Quorum	Majority agreement	W+R>N ensures consistency

Problem Statement

Design a distributed key-value store like Cosmos DB or DynamoDB that provides high availability, scalability, and tunable consistency.

Clarifying Questions to Ask

1. **Scale**: Data size? QPS? (Petabytes, millions QPS?)
2. **Consistency**: Tunable or fixed?
3. **Availability**: Multi-region active-active?
4. **Data model**: Just KV or support secondary indexes?
5. **Transactions**: Single-key or multi-key transactions?
6. **Latency**: Single-digit millisecond required?
7. **Durability**: What's acceptable data loss?

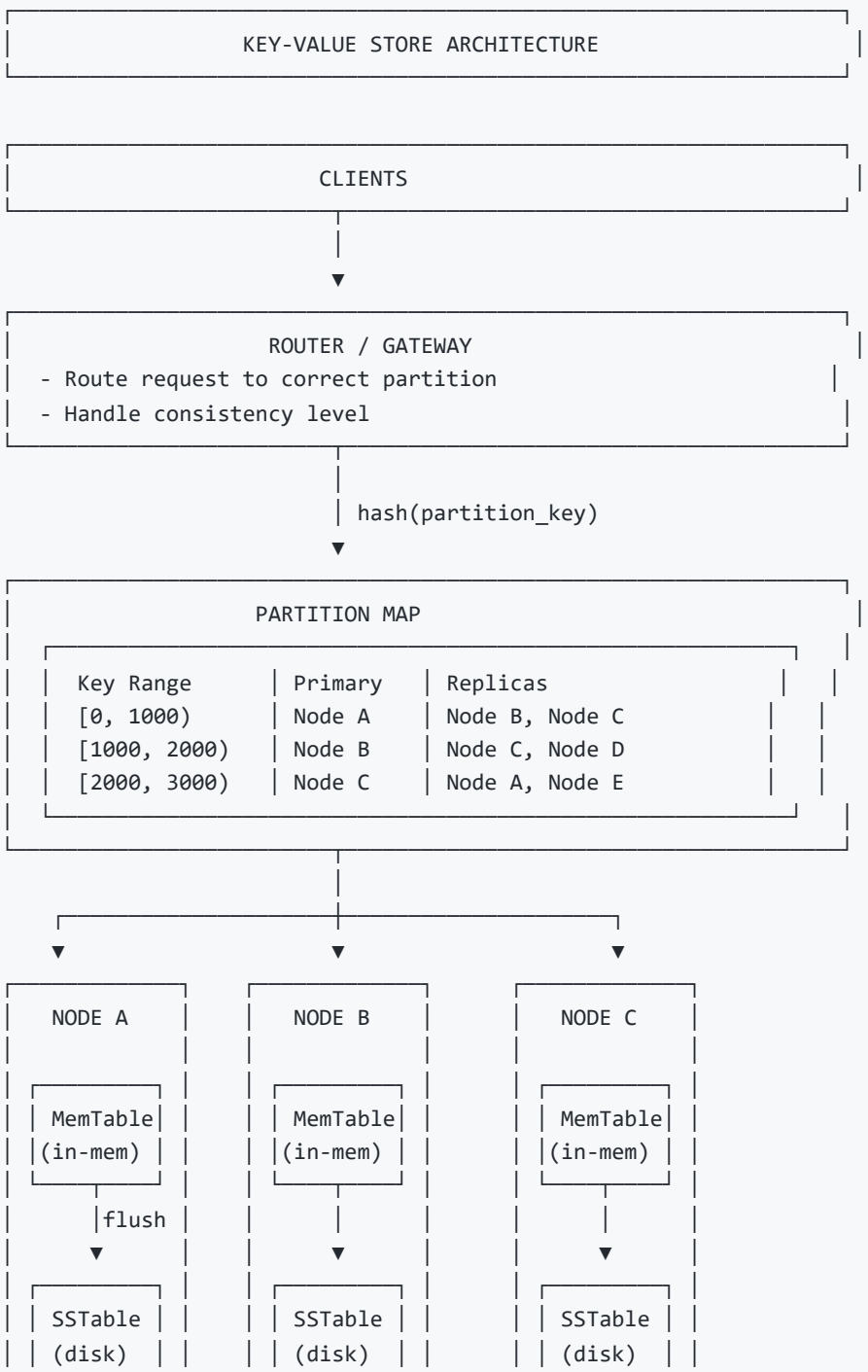
Design Reasoning

> **Core philosophy**: Optimize for write throughput using LSM trees with sequential I/O, partition data by hash for even distribution, and offer tunable consistency so callers can trade latency for correctness per request.

Decision	Choice	Why	Alternative Considered
Storage engine	LSM tree (RocksDB)	Sequential writes yield 10x+ write throughput over random I/O; compaction reclaims space in background	B-tree — better read performance but random writes become a bottleneck at scale
Partitioning	Hash-based (partition key)	Even distribution across nodes regardless of key patterns; avoids hot partitions	Range partitioning — enables range scans but skewed keys create hot spots
Consistency model	Tunable (strong → eventual)	Different use cases on the same cluster: strong for payments, eventual for counters	Fixed consistency — forces a one-size-fits-all trade-off on all callers
Replication	Quorum (W+R>N)	Tunable consistency at the replication level; 3 replicas with W=2, R=2 gives strong reads	Chain replication — simpler but less flexible; all-replica writes add latency
Read optimization	Bloom filters per SSTable	Eliminates disk reads for non-existent keys; reduces worst-case read amplification	No filter — every read must scan multiple SSTables, degrading P99 latency
Durability	Write-Ahead Log (WAL)	Survives crash before MemTable flush; sequential	No WAL — risks losing in-memory writes on

		append is fast	crash
Partition management	Raft consensus	Partition map is critical metadata; Raft ensures it survives leader failure	Gossip protocol — eventually consistent, risks split-brain partition ownership

High-Level Design





API Design

Core Key-Value APIs

Method	Endpoint	Description
`GET`	`/api/v1/kv/{key}`	Get value by key
`PUT`	`/api/v1/kv/{key}`	Put key-value pair
`DELETE`	`/api/v1/kv/{key}`	Delete a key
`GET`	`/api/v1/kv?prefix={prefix}&limit=100`	Range scan by prefix

Put Request with Options

```
PUT /api/v1/kv/user:123
{
  "value": "eyJuYW1lIjoiSm9obiJ9",  // base64 encoded
  "ttl_seconds": 86400,
  "consistency": "quorum",
  "if_version": 5                     // optimistic concurrency
}

Response: 200 OK
{
  "key": "user:123",
  "version": 6,
  "timestamp": 1672531200000
}
```

Batch APIs

Method	Endpoint	Description
`POST`	`/api/v1/kv/batch/get`	Get multiple keys
`POST`	`/api/v1/kv/batch/put`	Put multiple key-value pairs

Admin APIs

Method	Endpoint	Description

`GET`	`/api/v1/cluster/status`	Get cluster health and node info
`GET`	`/api/v1/cluster/partitions`	Get partition map
`POST`	`/api/v1/cluster/rebalance`	Trigger partition rebalancing

Key Components

Component	Purpose	Technology
Router	Direct requests to partitions	Stateless service
Partition Manager	Track partition ownership	Consensus (Paxos/Raft)
Storage Engine	Persist data	LSM tree (RocksDB)
Replication	Data redundancy	Chain or quorum
Compaction	Merge SSTables	Background process

LSM Tree Storage Engine

```
WRITE PATH:
┌──────────────────────────────────────────────────────────────┐
│  1. Write to Write-Ahead Log (WAL) - durability              │
│  2. Write to MemTable (in-memory sorted structure)           │
│  3. When MemTable full → flush to SSTable (disk)             │
│  4. Background compaction merges SSTables                     │
│                                                                │
│  Write: O(1) amortized - always sequential                   │
└──────────────────────────────────────────────────────────────┘

READ PATH:
┌──────────────────────────────────────────────────────────────┐
│  1. Check MemTable                                            │
│  2. Check Bloom filters of SSTables                          │
│  3. Search SSTables (newest to oldest)                       │
│  4. Return first match                                        │
│                                                                │
│  Read: O(log N) with Bloom filter optimization               │
└──────────────────────────────────────────────────────────────┘

COMPACTION:
┌──────────────────────────────────────────────────────────────┐
│  Level 0: Flushed MemTables (may overlap)                    │
│  Level 1+: Merged, non-overlapping ranges                    │
│                                                                │
│  Compaction merges levels, removes tombstones                │
```

```
│  Trade-off: Write amplification vs read performance            │
└─────────────────────────────────────────────────────────────────┘
```

Consistency Levels

```
TUNABLE CONSISTENCY (like Cosmos DB):
┌─────────────────────────────────────────────────────────────────┐
│                                                                 │
│  STRONG: Wait for all replicas (highest latency)               │
│    W: Write to all replicas before ACK                         │
│    R: Read from all, return latest                             │
│                                                                 │
│  BOUNDED STALENESS: Within K versions or T seconds             │
│    Guarantees: No reads older than bound                       │
│                                                                 │
│  SESSION: Monotonic reads within session                       │
│    Guarantees: Read-your-writes, no going backwards            │
│                                                                 │
│  EVENTUAL: Fastest, may return stale                           │
│    W: Write to any replica, async propagate                    │
│    R: Read from any replica                                    │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘

QUORUM FORMULA:
  N = total replicas
  W = write quorum
  R = read quorum

  Strong consistency if: W + R > N

  Example (N=3): W=2, R=2 → Always see latest write
```

Scalability Considerations

- **Partitioning**: Hash partition key for even distribution
- **Replication**: 3+ replicas across failure domains
- **Auto-scaling**: Split hot partitions automatically
- **Global distribution**: Multi-region with conflict resolution
- **Indexing**: Local secondary indexes per partition

Trade-offs

Decision	Option A	Option B	Recommendation
Storage	B-tree	LSM tree	**LSM** - better write throughput

Consistency	Fixed	Tunable	**Tunable** - flexibility
Partitioning	Range	Hash	**Hash** - even distribution
Replication	Chain	Quorum	**Quorum** - tunable consistency

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
LSM Tree	Write-optimized storage engine structure	Tech Guide
Cosmos DB	Tunable consistency and global distribution	Tech Guide
Bloom Filter	Efficient SSTable lookup optimization	Tech Guide
Raft Consensus	Partition ownership and replication	Tech Guide

---

How to Explain It Well

> **Opening**: "Key-value stores need high write throughput and tunable consistency. I'll use LSM trees for storage and quorum replication."

> **Key talking points**:
>
> - "LSM tree: writes are always sequential, great throughput"
> - "Bloom filters eliminate unnecessary SSTable reads"
> - "Quorum replication: W+R>N for strong consistency"
> - "Partition by hash for even distribution"
> - "Tunable consistency: strong for critical, eventual for counters"

---

41. Design Distributed Locking Service

Key Terms:

Term	Full Name	What it does
Mutex	Mutual Exclusion	Only one holder at a time

Lease	Time-limited lock	Auto-expires if holder crashes
Fencing Token	Monotonic counter	Prevents stale client from corrupting data
Redlock	Redis distributed lock	Majority quorum across Redis instances
TTL	Time To Live	Auto-expiration for leases
Heartbeat	Keep-alive signal	Extends lease while client is active
Raft/Paxos	Consensus algorithms	Replicated state machine for fault tolerance

Problem Statement

Design a distributed locking service that provides mutual exclusion across a cluster of servers.

Clarifying Questions to Ask

1. **Lock types**: Mutex, read-write, semaphore?
2. **Duration**: Short-lived or long-held locks?
3. **Fairness**: FIFO ordering or best-effort?
4. **Failure handling**: What if lock holder crashes?
5. **Scale**: How many locks? Lock operations per second?
6. **Clients**: In-datacenter or cross-region?
7. **Semantics**: Fencing tokens needed?

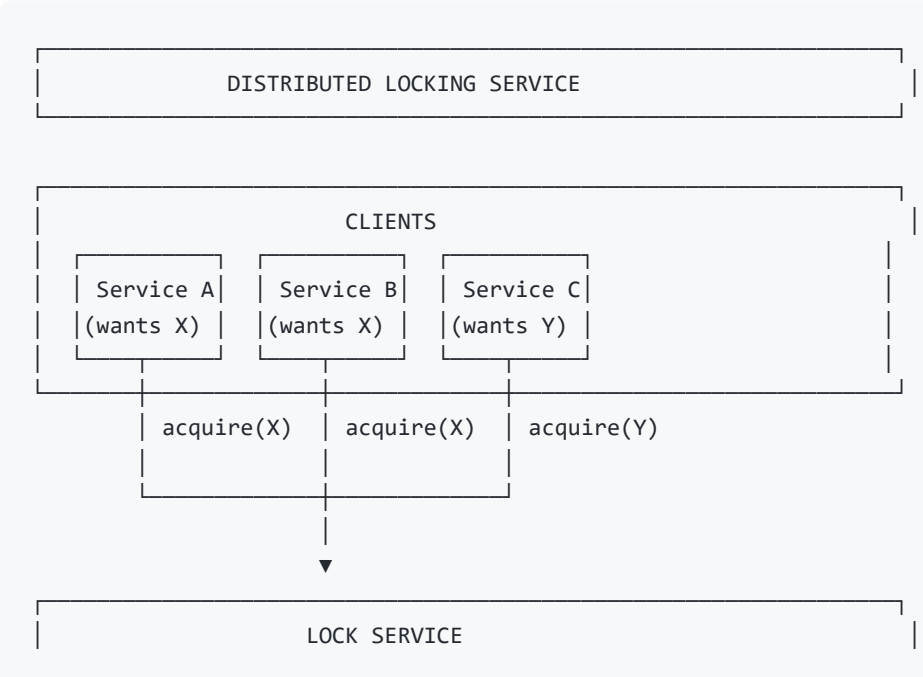
Design Reasoning

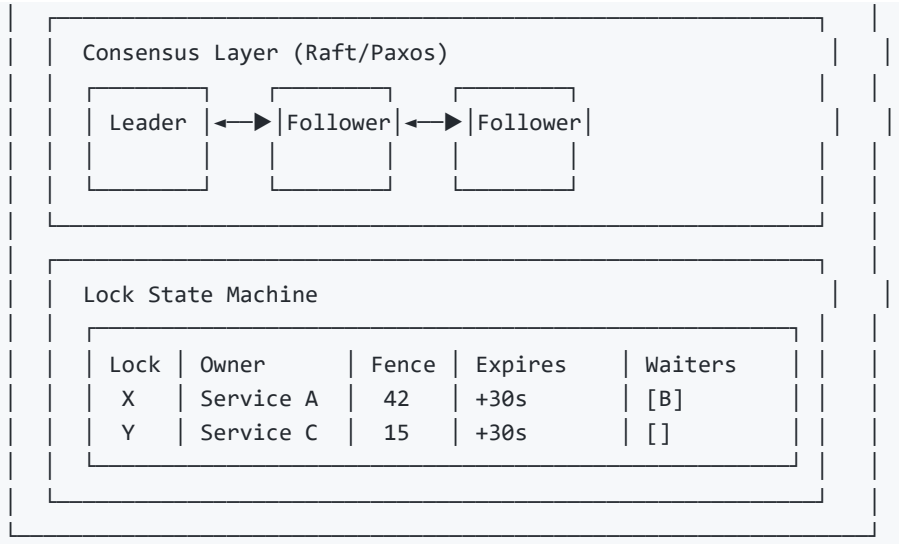
> **Core philosophy**: Use a consensus-backed replicated state machine for correctness — locks must survive node failures — and pair every lock with a fencing token so stale clients can never corrupt protected resources.

Decision	Choice	Why	Alternative Considered
Backend	Raft-based store (etcd)	Consensus is built-in; lock state survives leader failure without manual intervention	Redis (Redlock) — faster but no true consensus; clock drift can violate mutual exclusion

Safety mechanism	Fencing tokens (monotonic counter)	Prevents stale clients from writing after their lock expired and was re-granted	No fencing — a GC pause or network delay lets two clients believe they hold the lock
Lease management	TTL with client heartbeat	Auto-releases locks from crashed clients; heartbeat extends TTL for long operations	Infinite locks — a crashed holder blocks the resource forever
Wait strategy	Watch/event subscription	Waiters are notified immediately on release; no wasted polling traffic	Polling — wastes bandwidth and adds latency between release and next acquisition
Fairness	FIFO wait queue	Prevents starvation; callers acquire in order of request, which is predictable for operators	Best-effort — simpler but can starve long-waiting clients under contention
Lock partitioning	Hash-based sharding across groups	Distributes lock load across multiple Raft groups; no single group becomes a bottleneck	Single Raft group — simpler but limits throughput to one leader's capacity

High-Level Design





API Design

Lock APIs

Method	Endpoint	Description
`POST`	`/api/v1/locks/acquire`	Acquire a lock
`POST`	`/api/v1/locks/release`	Release a lock
`POST`	`/api/v1/locks/renew`	Renew lock TTL (heartbeat)
`GET`	`/api/v1/locks/{lockKey}`	Get lock status

Acquire Lock Request

```
POST /api/v1/locks/acquire
{
  "lock_key": "order:123:processing",
  "owner_id": "worker-456",
  "ttl_seconds": 30,
  "wait_timeout_ms": 5000   // 0 = no wait (try-lock)
}

Response: 200 OK
{
  "acquired": true,
  "lock_key": "order:123:processing",
  "fence_token": 42,        // monotonically increasing
  "expires_at": "2025-03-15T10:00:30Z"
}

Response: 409 Conflict
```

```
{
  "acquired": false,
  "current_owner": "worker-789",
  "retry_after_ms": 500
}
```

Release Lock Request

```
POST /api/v1/locks/release
{
  "lock_key": "order:123:processing",
  "owner_id": "worker-456",
  "fence_token": 42          // must match
}
```

Admin APIs

Method	Endpoint	Description
`GET`	`/api/v1/locks/active`	List all active locks
`DELETE`	`/api/v1/locks/{lockKey}/force`	Force-release a lock (admin)

Key Components

Component	Purpose	Technology
Consensus Layer	Replicated state machine	Raft (etcd) / Paxos (Chubby)
Lock Manager	Track lock ownership	State machine
Lease Manager	Auto-expire stale locks	TTL with heartbeat
Watch Service	Notify waiters	Event subscription
Fencing Token	Prevent stale operations	Monotonic counter

Lock Operations

```
ACQUIRE LOCK:
┌──────────────────────────────────────────────────────────────┐
│  1. Client sends: ACQUIRE(lock_id, client_id, ttl)           │
│  2. Leader checks lock state:                                 │
│     - If free: Grant lock, return fence_token                 │
```

```
│     - If held by same client: Extend TTL (reentrant)          │
│     - If held by other: Add to wait queue or reject           │
│  3. Replicate decision via Raft                                │
│  4. Return result to client                                    │
└────────────────────────────────────────────────────────────────┘

RELEASE LOCK:
┌────────────────────────────────────────────────────────────────┐
│  1. Client sends: RELEASE(lock_id, client_id, fence_token)    │
│  2. Leader verifies ownership and fence token                  │
│  3. If valid: Release lock, grant to next waiter              │
│  4. Replicate via Raft                                         │
└────────────────────────────────────────────────────────────────┘

HEARTBEAT (Keep-alive):
┌────────────────────────────────────────────────────────────────┐
│  Client periodically sends heartbeat while holding lock        │
│  Server extends TTL on heartbeat                               │
│  If no heartbeat within TTL: Auto-release lock                │
└────────────────────────────────────────────────────────────────┘
```

Fencing Tokens

```
PROBLEM: Client thinks it has lock, but lock expired
┌────────────────────────────────────────────────────────────────┐
│  Time 0: Client A acquires lock (token=42)                     │
│  Time 1: Client A pauses (GC, network delay)                   │
│  Time 2: Lock expires, Client B acquires (token=43)           │
│  Time 3: Client A resumes, thinks it has lock!                 │
│  Time 4: Both A and B write to resource → CORRUPTION          │
└────────────────────────────────────────────────────────────────┘

SOLUTION: Fencing tokens
┌────────────────────────────────────────────────────────────────┐
│  - Lock service returns monotonically increasing token        │
│  - Client includes token in all operations                     │
│  - Resource rejects operations with old tokens                 │
│                                                                │
│  Time 4: A sends write with token=42                           │
│          Resource sees token=43 was already used               │
│          Resource rejects A's write → SAFE                     │
└────────────────────────────────────────────────────────────────┘
```

Redis-Based Lock (Redlock)

```
REDLOCK ALGORITHM (for Redis without consensus):
┌────────────────────────────────────────────────────────────────┐
│  5 independent Redis instances                                 │
│                                                                │
│  Acquire:                                                      │
```

```
│  1. Get current time T1                                          │
│  2. Try SET lock_key client_id NX PX ttl on ALL 5 instances      │
│  3. Get current time T2                                          │
│  4. Lock acquired if:                                            │
│     - Majority (3+) instances returned OK                        │
│     - Total time (T2-T1) < ttl                                   │
│  5. If failed: Release on all instances                          │
│                                                                  │
│  Validity time = ttl - (T2-T1) - clock_drift                     │
└──────────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Partitioning**: Shard locks by key hash
- **Read scaling**: Serve reads from followers (for watch)
- **TTL tuning**: Balance between quick recovery and lease overhead
- **Batching**: Batch heartbeats for many locks
- **Caching**: Cache lock state locally with invalidation

Trade-offs

Decision	Option A	Option B	Recommendation
Backend	Redis	etcd/ZooKeeper	**etcd** - consensus built-in
Waiting	Block	Poll	**Watch** - efficient
Fairness	FIFO	Best-effort	**FIFO** - predictable
Fencing	Required	Optional	**Required** - safety

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Lock storage with Redlock algorithm for distributed mutual exclusion	Tech Guide
Consistent Hashing	Partitioning locks across nodes for scalability	Tech Guide
Raft Consensus	Ensuring lock state survives failures with replicated state machine	Tech Guide

How to Explain It Well

> **Opening**: "Distributed locks need consensus for correctness. I'll use Raft-based storage (etcd) with fencing tokens for safety."

> **Key talking points**:
>
> - "Raft consensus ensures lock state survives failures"
> - "TTL with heartbeat auto-releases locks from crashed clients"
> - "Fencing tokens prevent stale clients from corrupting resources"
> - "Watch API notifies waiters without polling"
> - "Redlock for Redis: majority quorum across independent instances"
